AI-Coding-Documentation-Assistant

User uploads Python file

↓

Python file contains code + error

↓

System reads/extracts the code

↓

User asks:

"Why am I getting this TypeError?"

↓

RAG searches your knowledge base

(Python docs + your coding/theory files)

↓

Relevant documentation is retrieved

↓

Gemini 2.5 Flash analyzes:

- uploaded code
- error
- retrieved documentation
- user's question

↓

AI generates:

- Error explanation
- Cause of error
- Corrected code

- Explanation of the fix
- Relevant source

↓

Frontend displays the answer

🚀 AI Coding Documentation Assistant — Complete Step-by-Step Plan

1. Project Objective

Build an AI assistant where a user can upload or provide technical documentation such as:

- Python documentation
- SQL notes
- Machine Learning notes
- Git documentation
- API documentation
- Programming tutorials
- Project documentation

Then the user can ask:

Explain this Python error.

How does INNER JOIN work?

Give me an example of RAG.

Explain this function.

What is the difference between REST and GraphQL?

The system will:

```
User Question
      ↓
Convert question to embedding
      ↓
Search Vector Database
      ↓
Retrieve relevant documentation
      ↓
Send question + retrieved context to Gemini
      ↓
Generate answer
      ↓
Display answer with sources
```

This is the core **RAG — Retrieval-Augmented Generation** architecture.

2. Final Project Architecture

Your project should eventually look like this:

```
AI-Coding-Documentation-Assistant/
│
├── Backend/
│   │
│   ├── app.py
│   ├── requirements.txt
│   ├── .env
│   │
│   ├── data/
│   │   ├── python/
│   │   ├── sql/
│   │   ├── machine_learning/
│   │   ├── git/
│   │   └── api/
│   │
│   └── src/
│       │
```

```
│       ├── document_loader.py
│       ├── text_cleaner.py
│       ├── chunker.py
│       ├── embeddings.py
│       ├── vector_store.py
│       ├── retriever.py
│       ├── llm.py
│       ├── rag_pipeline.py
│       └── github_loader.py
│
├── Frontend/
│   │
│   ├── src/
│   │   ├── App.jsx
│   │   ├── App.css
│   │   └── components/
│   │       ├── Chat.jsx
│   │       ├── Upload.jsx
│   │       ├── SourceCard.jsx
│   │       └── Sidebar.jsx
│   │
│   ├── package.json
│   └── vite.config.js
│
├── vector_db/
│
└── README.md
```

3. Technologies You Can Use

Backend

```
Python
FastAPI
Google Gemini 2.5 Flash
Sentence Transformers
FAISS / ChromaDB
```

PyPDF
python-docx
GitPython

Frontend

React.js
Vite
JavaScript
CSS / Tailwind CSS
Axios

AI/RAG

Gemini 2.5 Flash
Sentence Transformers
Embeddings
Vector Database
Semantic Search
RAG

Output:

Understood. For **now, we will work only on the Backend.** We will completely leave the React/Vite frontend untouched and build it later.

Current project scope

```
AI-Coding-Documentation-Assistant/
│
├── Backend/                    ← WORKING NOW
│   │
│   ├── app.py
│   ├── requirements.txt
│   ├── .env
│   │
│   ├── data/
│   │   ├── python/
│   │   ├── sql/
```

```
│   │   ├── machine_learning/
│   │   ├── git/
│   │   └── api/
│   │
│   └── src/
│       ├── document_loader.py
│       ├── text_cleaner.py
│       ├── chunker.py
│       ├── embeddings.py
│       ├── vector_store.py
│       ├── retriever.py
│       ├── llm.py
│       ├── rag_pipeline.py
│       └── github_loader.py
│
├── Frontend/                    ← DON'T TOUCH NOW
│
├── vector_db/                   ← WORKING NOW
│
└── README.md                    ← WORKING NOW
```

Backend technologies we'll use

Python
FastAPI
Gemini 2.5 Flash
Sentence Transformers
FAISS
PyPDF
python-docx
GitPython

For the RAG layer:

```
Documents
   ↓
Document Loader
   ↓
Text Cleaner
   ↓
```

Chunker
   ↓
Sentence Transformer
   ↓
Embeddings
   ↓
FAISS
   ↓
Retriever
   ↓
Relevant Context
   ↓
Gemini 2.5 Flash
   ↓
Answer

Backend implementation order

I recommend we complete the backend in this exact order:

Step 1 — document_loader.py

Read PDF, DOCX, TXT, Python and other documentation files.

Step 2 — text_cleaner.py

Clean extracted documentation and source code.

Step 3 — chunker.py

Split documentation into meaningful chunks.

Step 4 — embeddings.py

Convert chunks into embeddings using Sentence Transformers.

Step 5 — vector_store.py

Create and save:

```
vector_db/
├── index.faiss
└── metadata.pkl
```

Step 6 — retriever.py

Search the vector database using semantic similarity.

Step 7 — llm.py

Connect **Gemini 2.5 Flash**.

Step 8 — rag_pipeline.py

Combine:

```
Question
+
Retrieved documentation
+
Uploaded code
        ↓
Gemini
        ↓
Answer
```

Step 9 — github_loader.py

Later, support GitHub repositories.

Step 10 — app.py

Create the FastAPI endpoints.

Step 11 — Backend testing

Test everything through API requests before touching the frontend.

Only **after all of this works,** we'll start the React frontend.

So yes: **ignore Frontend/ completely for now.** Our next focus should be document_loader.py.

4. Step 1 — Create the Project

Open PowerShell:

```
mkdir AI-Coding-Documentation-Assistant
cd AI-Coding-Documentation-Assistant
```

Create:

```
mkdir Backend
mkdir Frontend
mkdir vector_db
```

Then:

```
cd Backend
mkdir src
mkdir data
```

5. Step 2 — Create Python Virtual Environment

Inside Backend:

```
python -m venv venv
```

Activate:

```
.\venv\Scripts\activate
```

You should see something similar to:

```
(venv) PS C:\...\AI-Coding-Documentation-Assistant\Backend>
```

6. Step 3 — Install Required Packages

Run:

```
pip install fastapi uvicorn python-dotenv
```

Install Gemini:

```
pip install google-genai
```

Install document processing:

```
pip install pypdf python-docx
```

Install embeddings:

```
pip install sentence-transformers
```

Install vector database:

```
pip install faiss-cpu
```

Install GitHub repository support:

```
pip install gitpython
```

Install multipart upload:

```
pip install python-multipart
```

You can save everything:

```
pip freeze > requirements.txt
```

7. Step 4 — Create `.env`

Inside:

```
Backend/.env
```

put:

```
GEMINI_API_KEY=YOUR_GEMINI_API_KEY
```

Important

Do **not** upload `.env` to GitHub.

Create:

```
.gitignore
```

with:

```
.env
venv/
__pycache__/
vector_db/
node_modules/
```

If you already exposed a Gemini API key publicly, **revoke that key and create a new one**.

8. Step 5 — Collect Your Knowledge Base

Create:

```
Backend/data/
```

Inside:

```
data/
│
├── python/
├── sql/
├── machine_learning/
├── git/
└── api/
```

For example:

```
python/
    python_basics.pdf
```

```
    python_functions.pdf
    python_exceptions.pdf

sql/
    sql_notes.pdf
    joins.pdf
    subqueries.pdf

machine_learning/
    ml_notes.pdf
    regression.pdf
    classification.pdf

git/
    git_commands.pdf
    git_branching.pdf

api/
    rest_api.pdf
    graphql.pdf
```

Your assistant will use these documents as its **knowledge base**.

9. Step 6 — Document Loading

Create:

`src/document_loader.py`

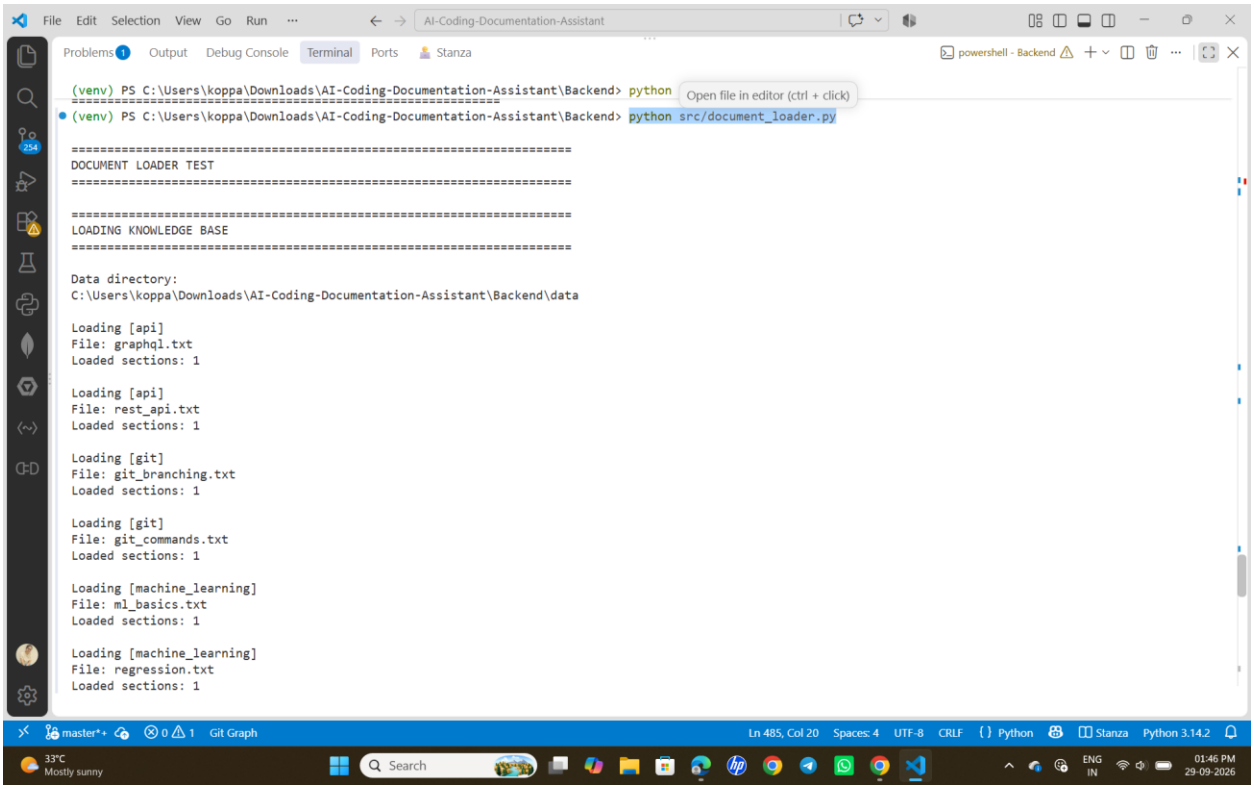
Its job:

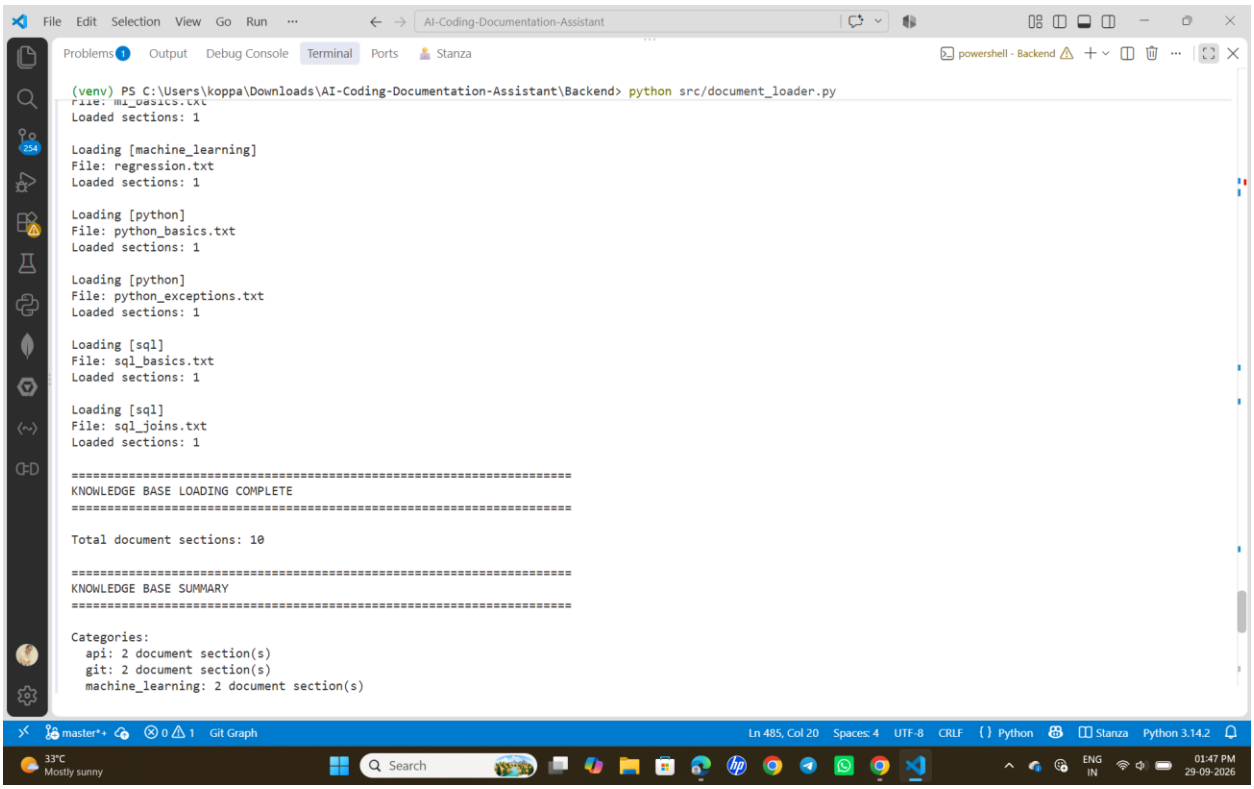
```
PDF
 ↓
Extract text
 ↓
Return documents
```

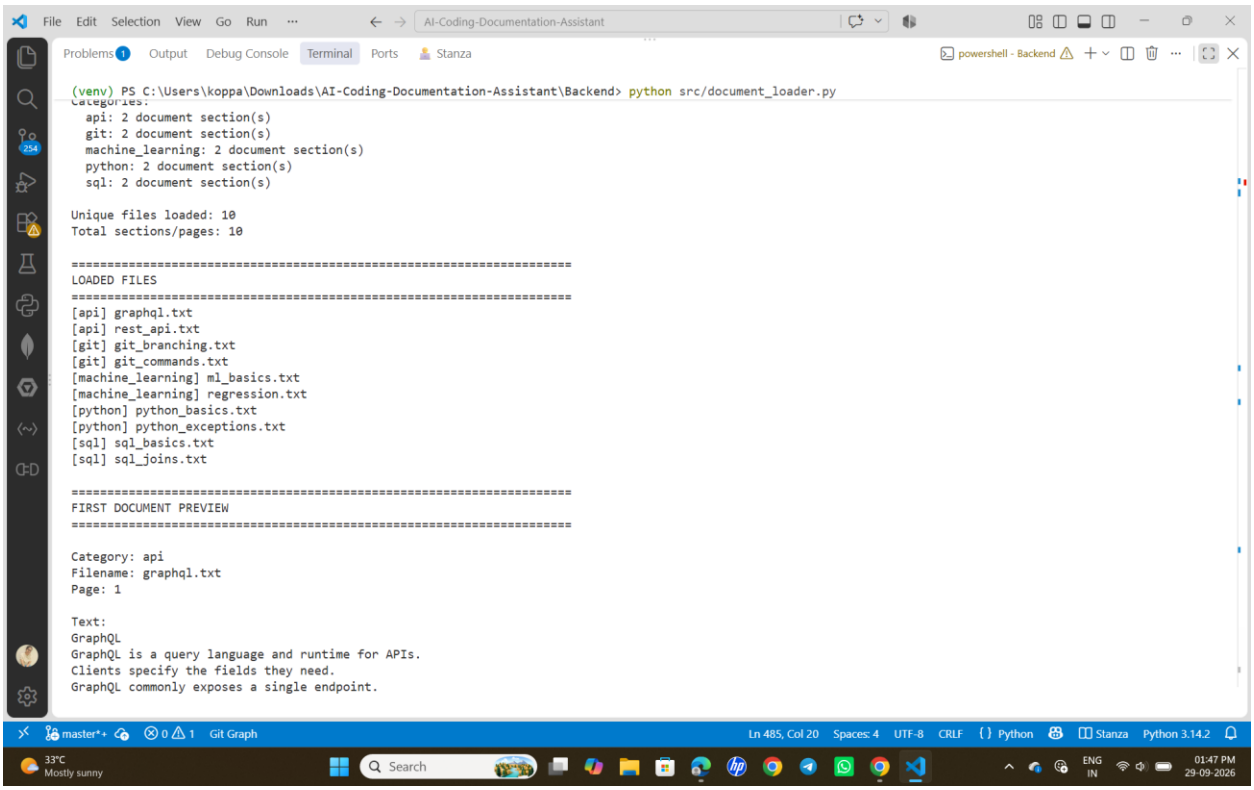
For example:

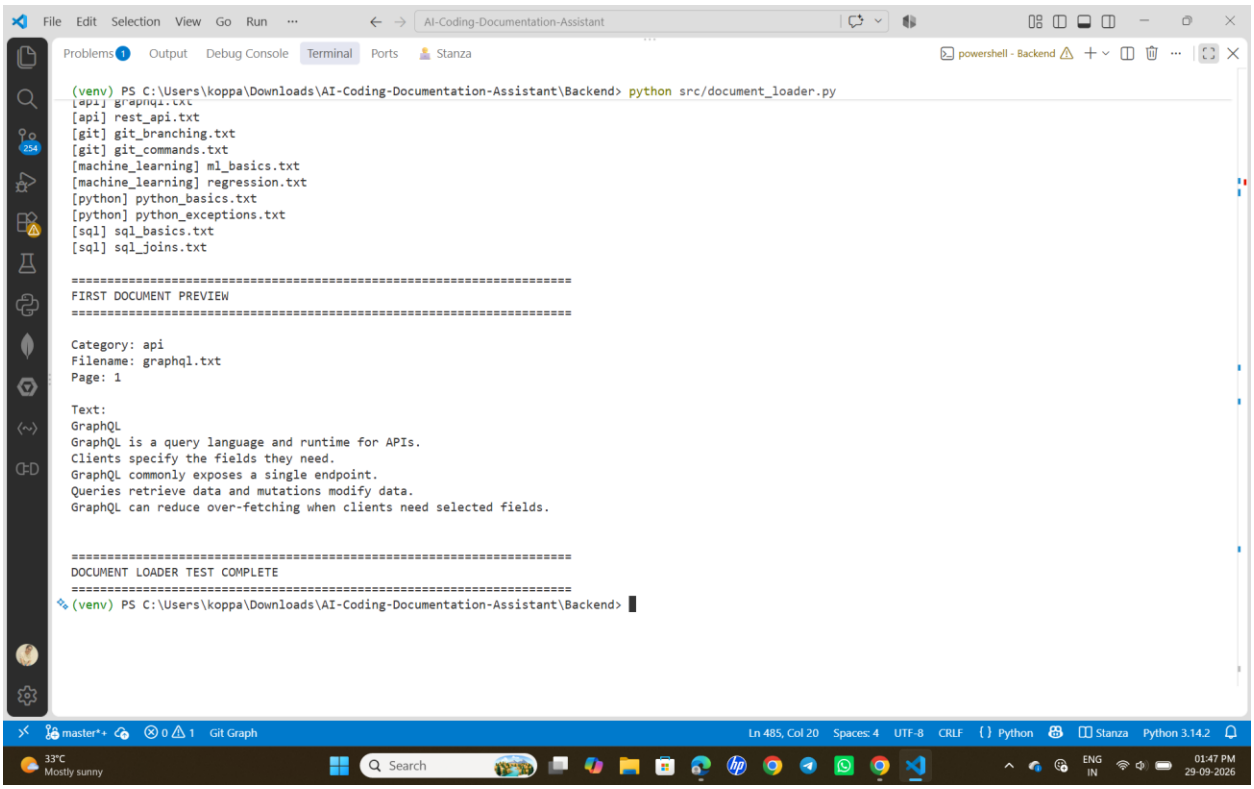
```
from pypdf import PdfReaderdef load_pdf(file_path):    reader = PdfReader(file_path)    documents = []    for page_number, page in enumerate(reader.pages, start=1):        text = page.extract_text()        if text:            documents.append({                "text": text,                "source": file_path,                "page": page_number            })    return documents
```

Output:




File Edit Selection View Go Run
AI-Coding-Documentation-Assistant
Problems 1 Output Debug Console Terminal Ports Stanza
powershell - Backend
(venv) PS C:\Users\koppa\Downloads\AI-Coding-Documentation-Assistant\Backend> python src/document_loader.py
File: ml_basics.txt
Loaded sections: 1
Loading [machine_learning]
File: regression.txt
Loaded sections: 1
Loading [python]
File: python_basics.txt
Loaded sections: 1
Loading [python]
File: python_exceptions.txt
Loaded sections: 1
Loading [sql]
File: sql_basics.txt
Loaded sections: 1
Loading [sql]
File: sql_joins.txt
Loaded sections: 1
KNOWLEDGE BASE LOADING COMPLETE
Total document sections: 10
KNOWLEDGE BASE SUMMARY
Categories:
api: 2 document section(s)
git: 2 document section(s)
machine_learning: 2 document section(s)
master*+ 0 1 Git Graph
Ln 485, Col 20 Spaces: 4 UTF-8 CRLF Python Stanza Python 3.14.2
33°C Mostly sunny
Search
ENG IN
01:47 PM 29-09-2026


File Edit Selection View Go Run
AI-Coding-Documentation-Assistant
Problems 1 Output Debug Console Terminal Ports Stanza
powershell - Backend
(venv) PS C:\Users\koppa\Downloads\AI-Coding-Documentation-Assistant\Backend> python src/document_loader.py
Categories:
api: 2 document section(s)
git: 2 document section(s)
machine_learning: 2 document section(s)
python: 2 document section(s)
sql: 2 document section(s)
Unique files loaded: 10
Total sections/pages: 10
LOADED FILES
[api] graphql.txt
[api] rest_api.txt
[git] git_branching.txt
[git] git_commands.txt
[machine_learning] ml_basics.txt
[machine_learning] regression.txt
[python] python_basics.txt
[python] python_exceptions.txt
[sql] sql_basics.txt
[sql] sql_joins.txt
FIRST DOCUMENT PREVIEW
Category: api
Filename: graphql.txt
Page: 1
Text:
GraphQL
GraphQL is a query language and runtime for APIs.
Clients specify the fields they need.
GraphQL commonly exposes a single endpoint.
master*+ 0 1 Git Graph
Ln 485, Col 20 Spaces: 4 UTF-8 CRLF Python Stanza Python 3.14.2
33°C Mostly sunny
Search
ENG IN
01:47 PM 29-09-2026



10. Step 7 — Text Cleaning

Create:

`src/text_cleaner.py`

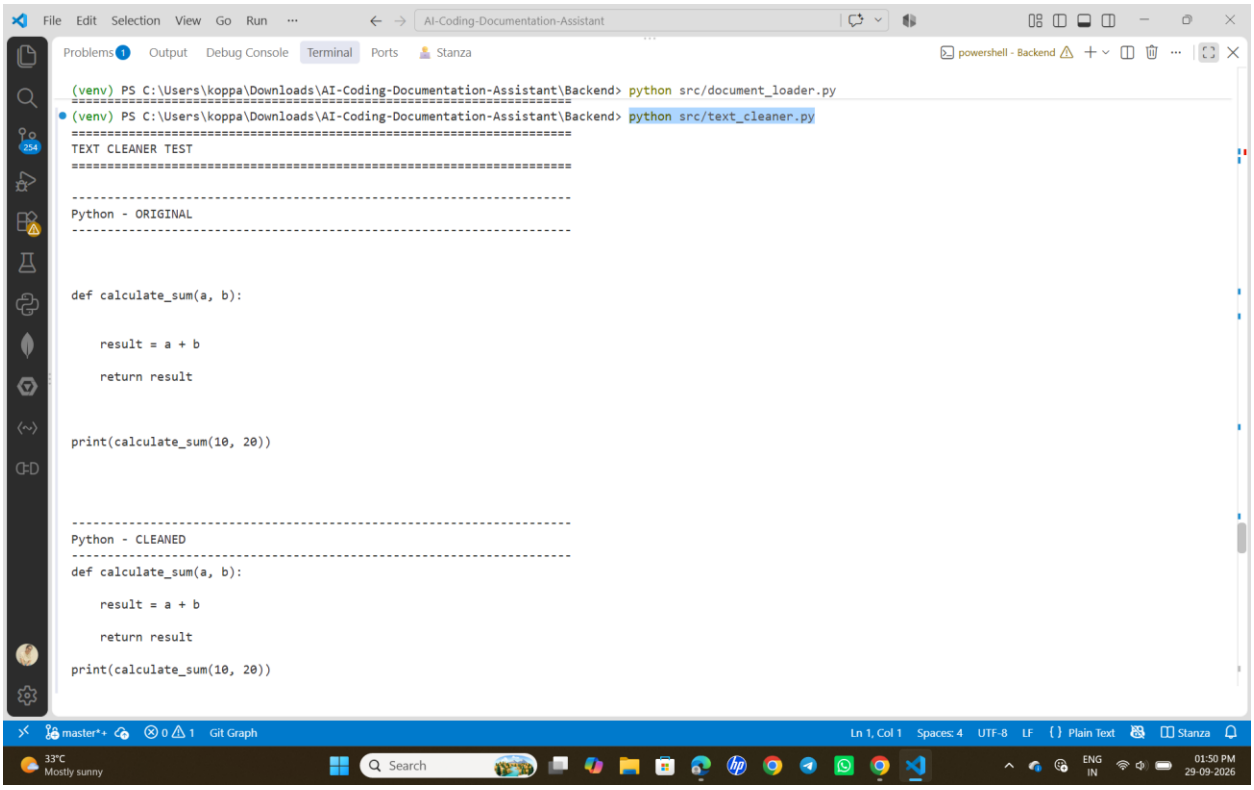
Purpose:

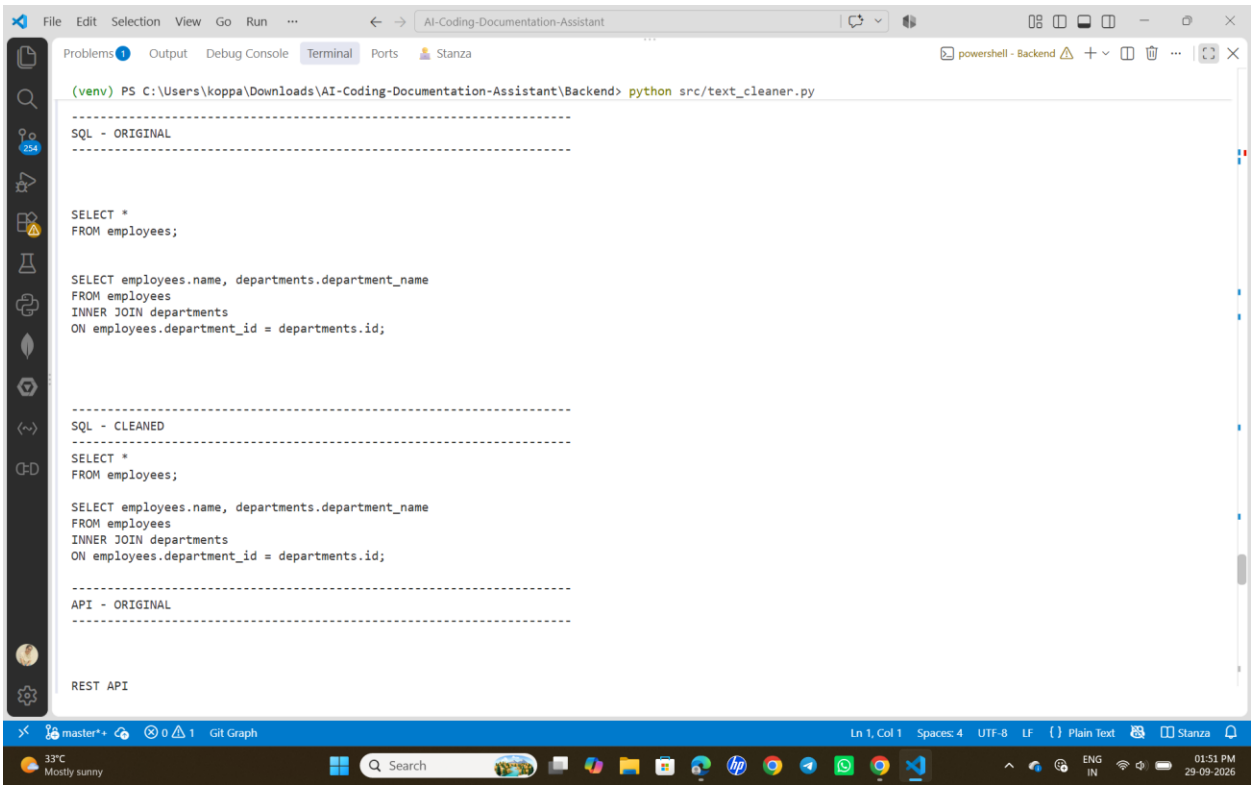
```
Raw text
   ↓
Remove unnecessary spaces
   ↓
Remove unwanted characters
   ↓
Clean text
```

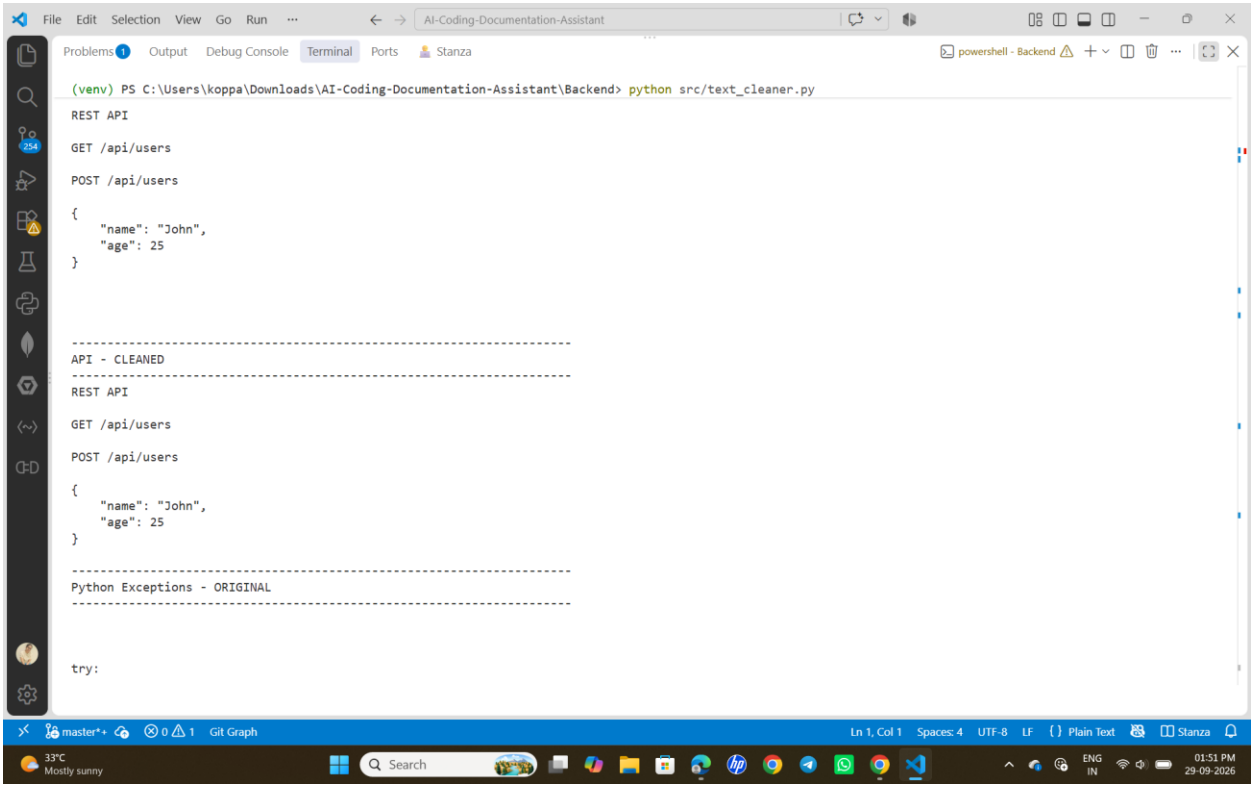
Example:

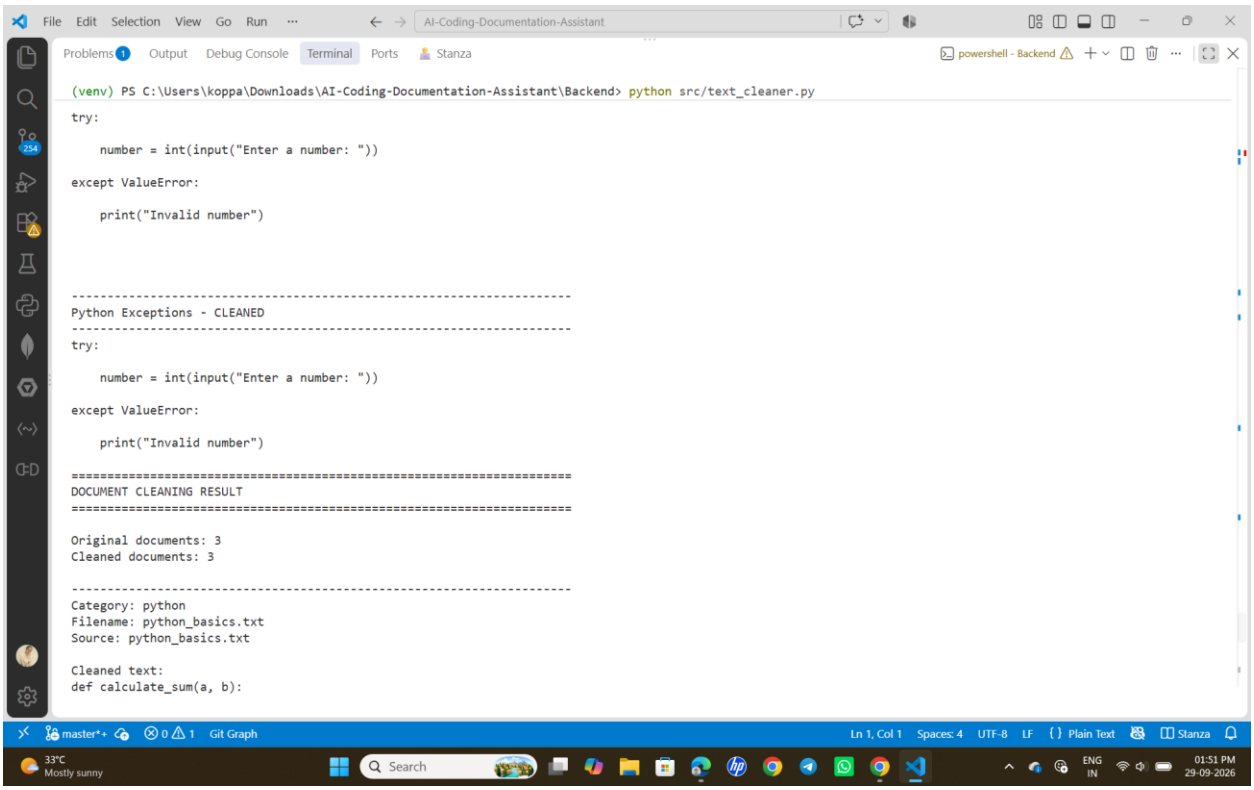
```
def clean_text(text):    text = text.replace("\n\n", "\n")    text = text.strip()    return text
```

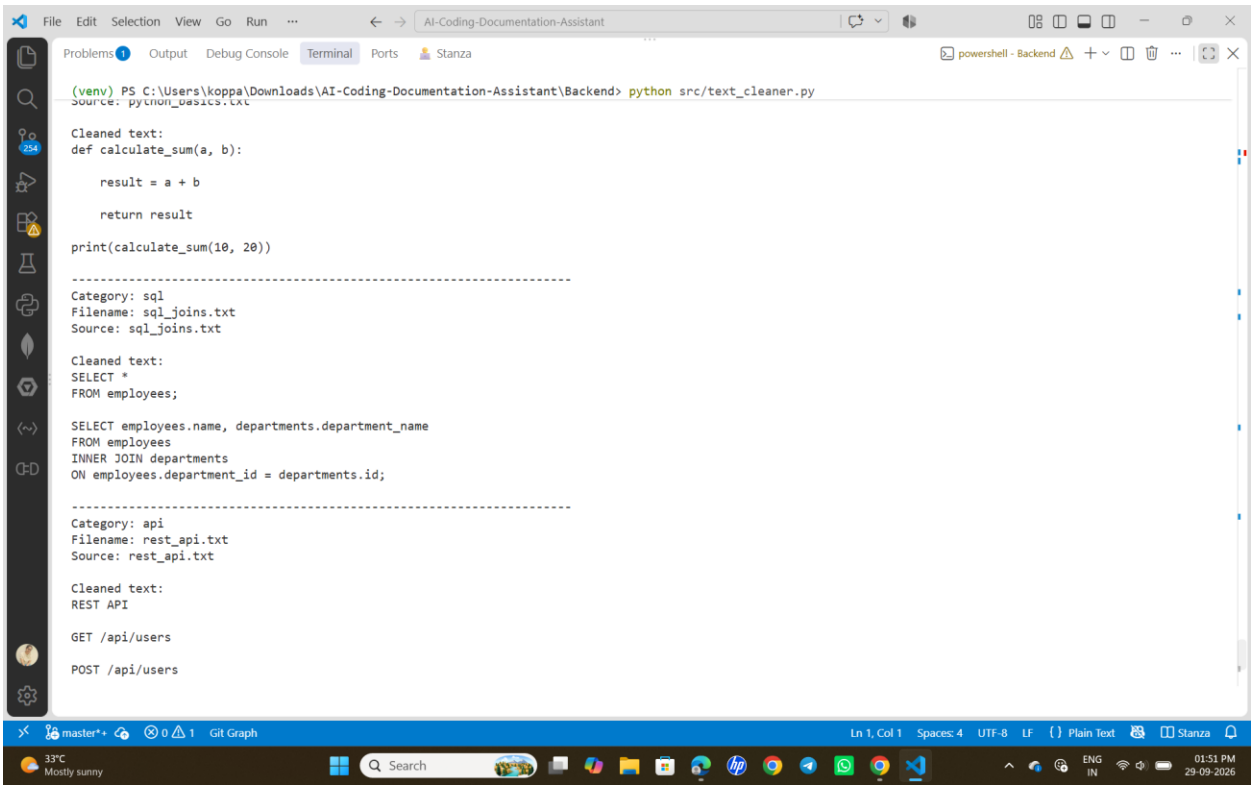
Later you can make this more advanced.

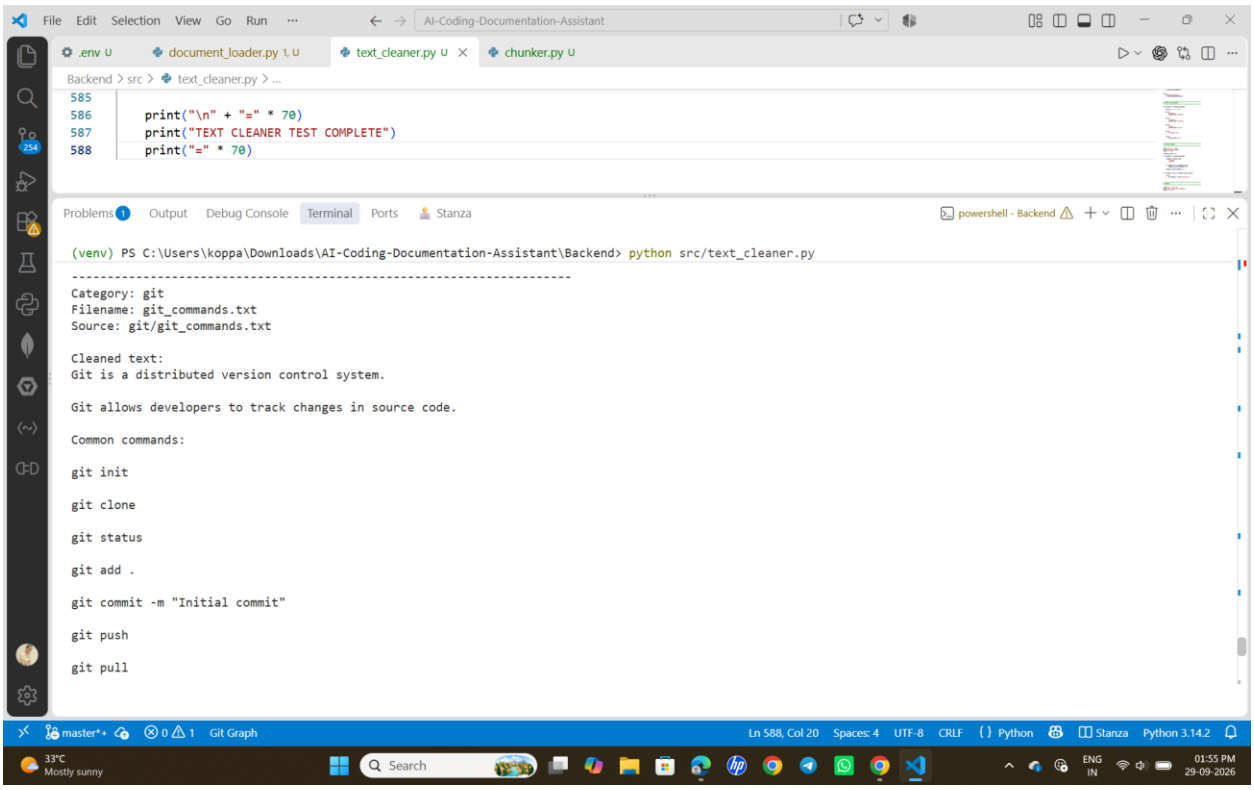


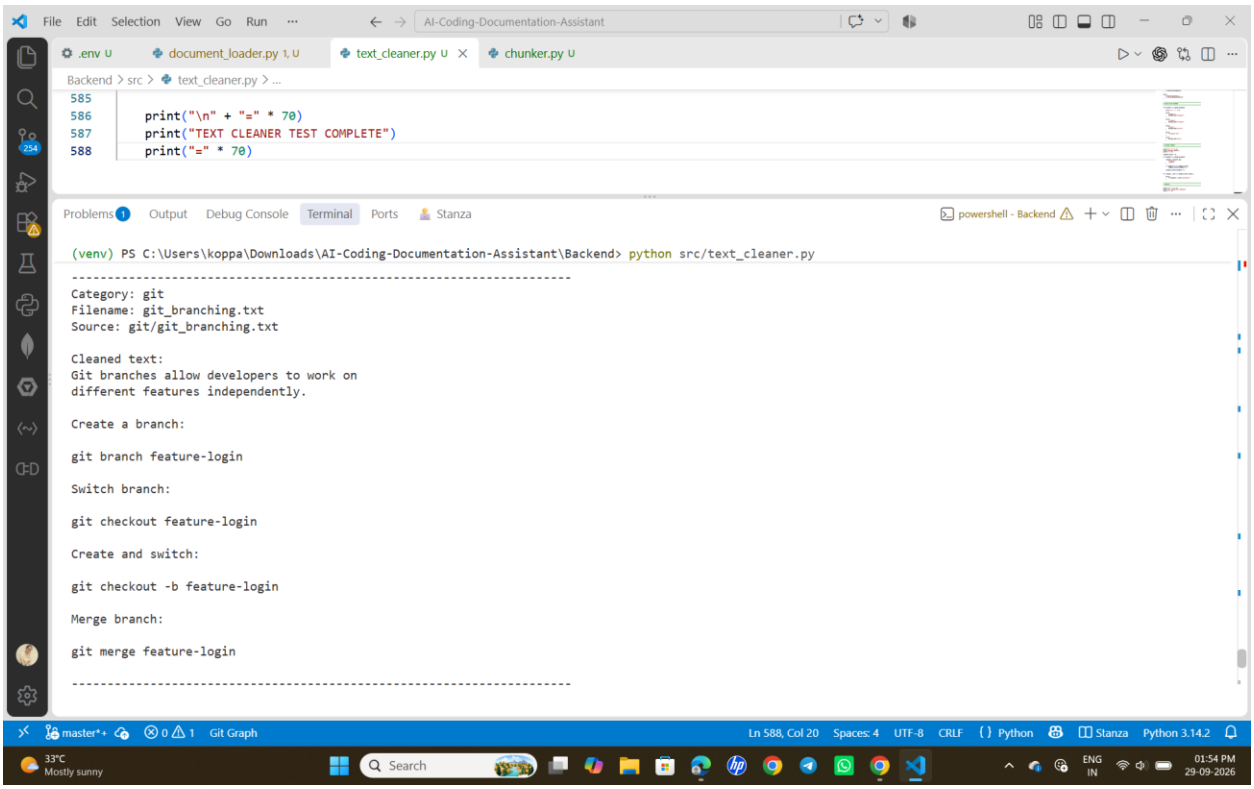

File Edit Selection View Go Run
AI-Coding-Documentation-Assistant
Problems 1 Output Debug Console Terminal Ports Stanza
powershell - Backend
(venv) PS C:\Users\koppa\Downloads\AI-Coding-Documentation-Assistant\Backend> python src/text_cleaner.py
SQL - ORIGINAL
SELECT *
FROM employees;
SELECT employees.name, departments.department_name
FROM employees
INNER JOIN departments
ON employees.department_id = departments.id;
SQL - CLEANED
SELECT *
FROM employees;
SELECT employees.name, departments.department_name
FROM employees
INNER JOIN departments
ON employees.department_id = departments.id;
API - ORIGINAL
REST API
master*+ 0 1 Git Graph
Ln 1, Col 1 Spaces: 4 UTF-8 LF Plain Text Stanza
33°C Mostly sunny
Search
ENG IN
01:51 PM 29-09-2026

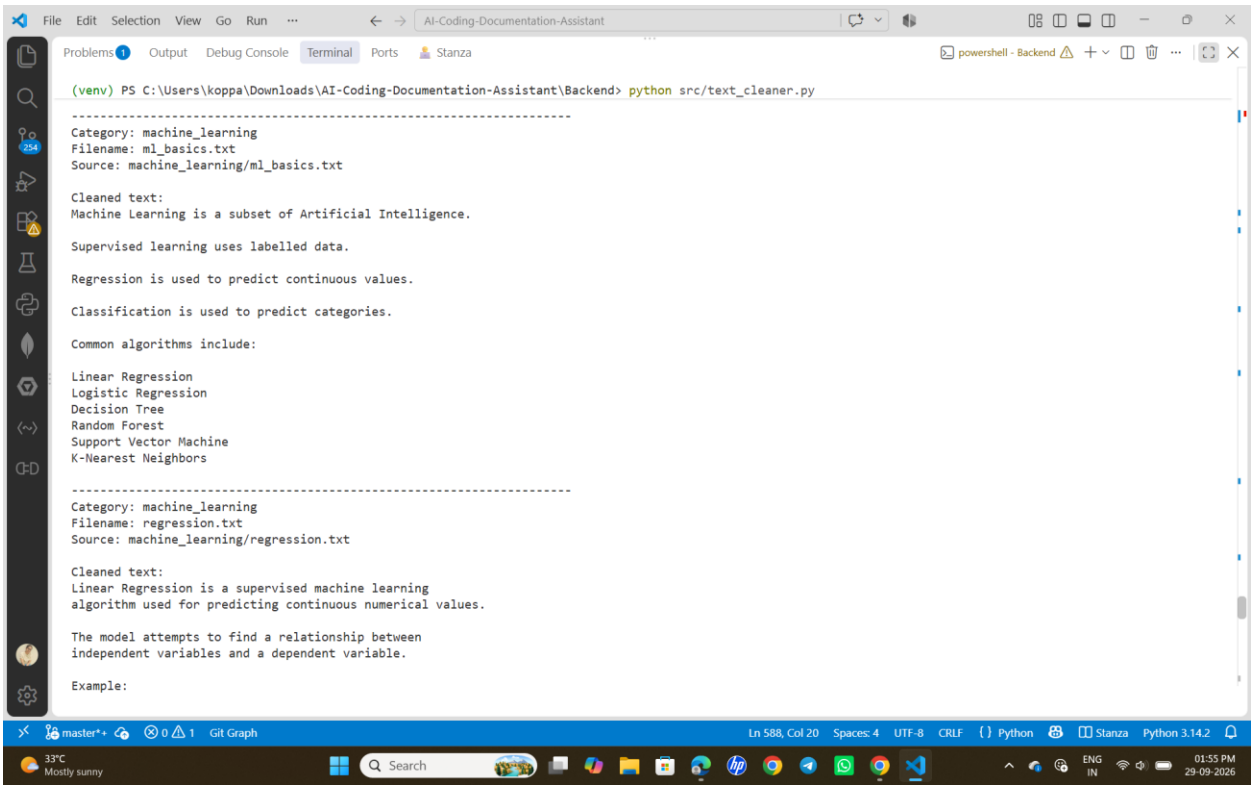

File Edit Selection View Go Run
AI-Coding-Documentation-Assistant
Problems 1 Output Debug Console Terminal Ports Stanza
powershell - Backend
(venv) PS C:\Users\koppa\Downloads\AI-Coding-Documentation-Assistant\Backend> python src/text_cleaner.py
REST API
GET /api/users
POST /api/users
{
"name": "John",
"age": 25
}
API - CLEANED
REST API
GET /api/users
POST /api/users
{
"name": "John",
"age": 25
}
Python Exceptions - ORIGINAL
try:
master*+ 0 1 Git Graph
Ln 1, Col 1 Spaces: 4 UTF-8 LF Plain Text Stanza
33°C Mostly sunny
Search
ENG IN
01:51 PM 29-09-2026

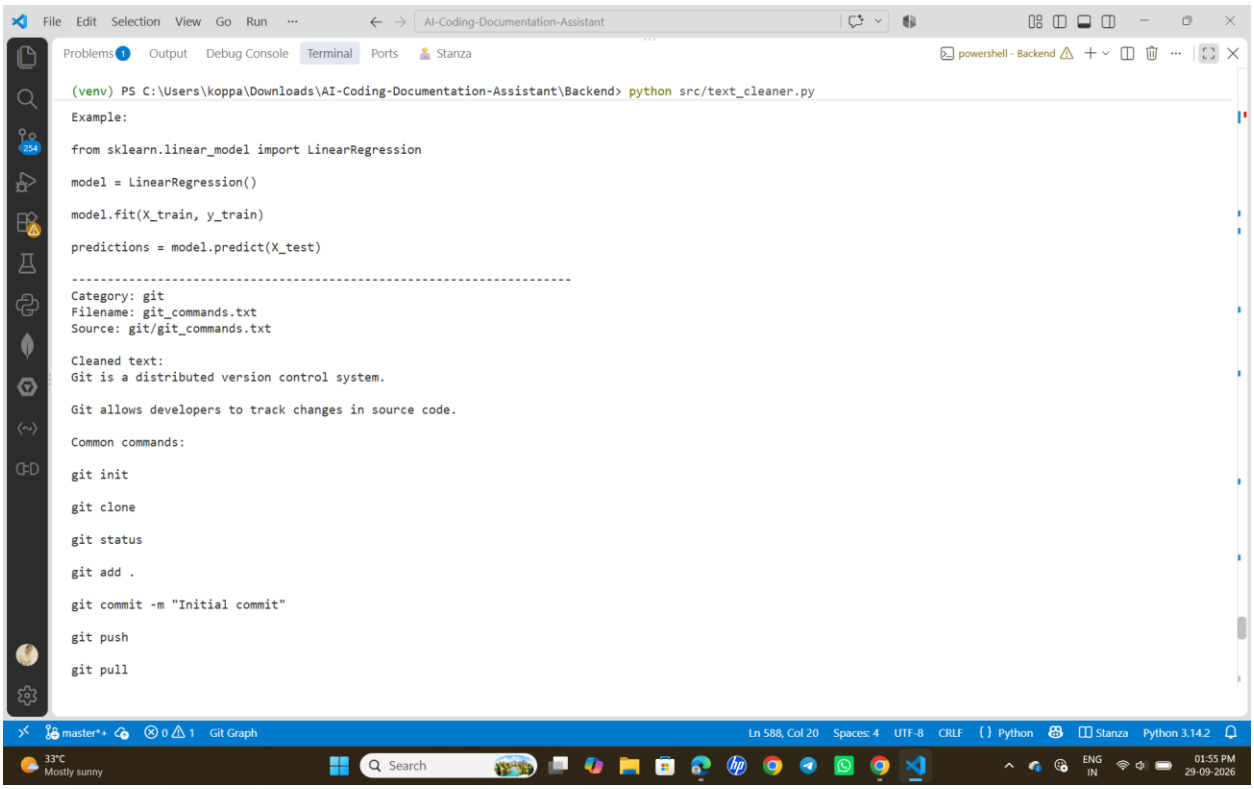

File Edit Selection View Go Run
AI-Coding-Documentation-Assistant
Problems 1 Output Debug Console Terminal Ports Stanza
powershell - Backend
(venv) PS C:\Users\koppa\Downloads\AI-Coding-Documentation-Assistant\Backend> python src/text_cleaner.py
try:
number = int(input("Enter a number: "))
except ValueError:
print("Invalid number")
Python Exceptions - CLEANED
try:
number = int(input("Enter a number: "))
except ValueError:
print("Invalid number")
DOCUMENT CLEANING RESULT
Original documents: 3
Cleaned documents: 3
Category: python
Filename: python_basics.txt
Source: python_basics.txt
Cleaned text:
def calculate_sum(a, b):
master*+ 0 1 Git Graph
Ln 1, Col 1 Spaces: 4 UTF-8 LF Plain Text Stanza
33°C
Mostly sunny
Search
ENG
IN
01:51 PM
29-09-2026

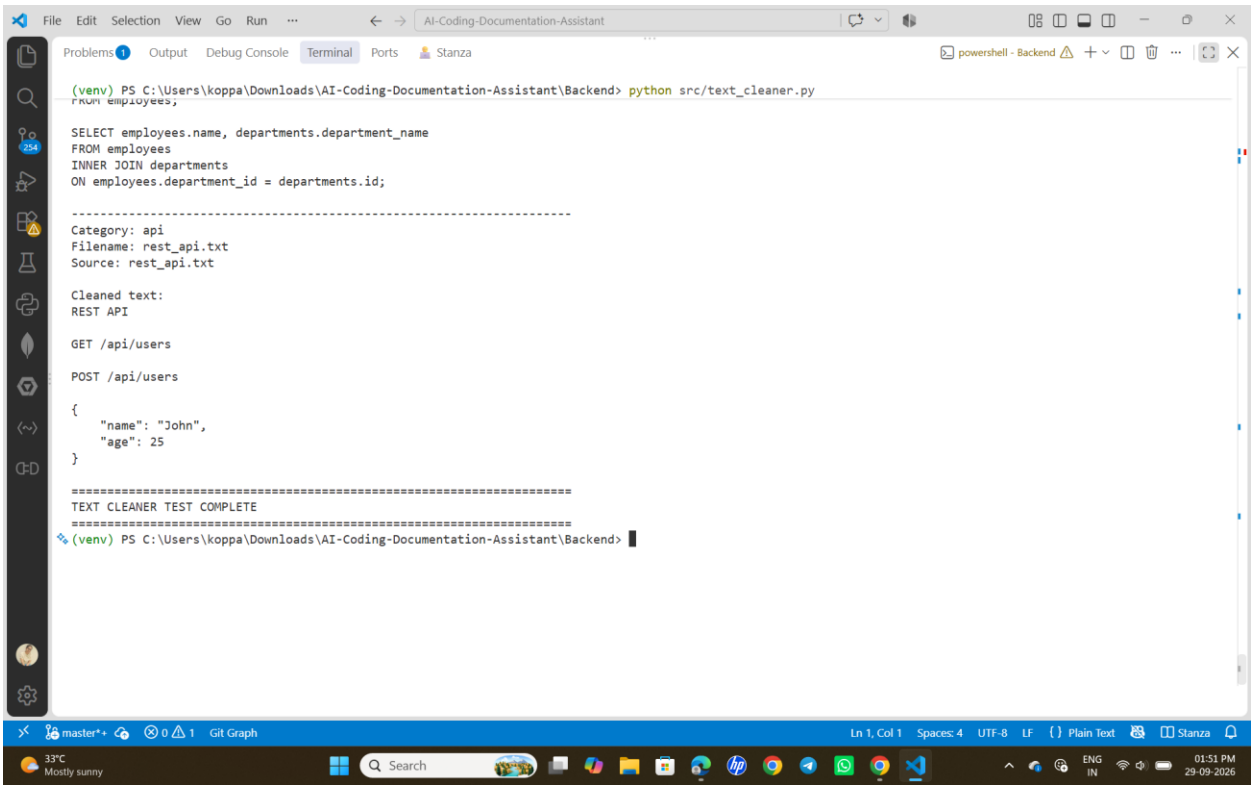

File Edit Selection View Go Run
AI-Coding-Documentation-Assistant
Problems 1 Output Debug Console Terminal Ports Stanza
powershell - Backend
(venv) PS C:\Users\koppa\Downloads\AI-Coding-Documentation-Assistant\Backend> python src/text_cleaner.py
Source: python_basics.txt
Cleaned text:
def calculate_sum(a, b):
result = a + b
return result
print(calculate_sum(10, 20))
Category: sql
Filename: sql_joins.txt
Source: sql_joins.txt
Cleaned text:
SELECT *
FROM employees;
SELECT employees.name, departments.department_name
FROM employees
INNER JOIN departments
ON employees.department_id = departments.id;
Category: api
Filename: rest_api.txt
Source: rest_api.txt
Cleaned text:
REST API
GET /api/users
POST /api/users
master*+ 0 1 Git Graph
Ln 1, Col 1 Spaces: 4 UTF-8 LF Plain Text Stanza
33°C Mostly sunny
Search
ENG IN
01:51 PM 29-09-2026


File Edit Selection View Go Run
AI-Coding-Documentation-Assistant
.env U document_loader.py 1, U text_cleaner.py U chunker.py U
Backend > src > text_cleaner.py > ...
print("\n" + "=" * 70)
print("TEXT CLEANER TEST COMPLETE")
print("=" * 70)
Problems 1 Output Debug Console Terminal Ports Stanza
powershell - Backend
(venv) PS C:\Users\koppa\Downloads\AI-Coding-Documentation-Assistant\Backend> python src/text_cleaner.py
Category: git
Filename: git_commands.txt
Source: git/git_commands.txt
Cleaned text:
Git is a distributed version control system.
Git allows developers to track changes in source code.
Common commands:
git init
git clone
git status
git add .
git commit -m "Initial commit"
git push
git pull
master*+ 0 1 Git Graph
Ln 588, Col 20 Spaces: 4 UTF-8 CRLF Python Stanza Python 3.14.2
33°C Mostly sunny
Search
ENG IN
01:55 PM 29-09-2026


File Edit Selection View Go Run
AI-Coding-Documentation-Assistant
.env
document_loader.py
text_cleaner.py
chunker.py
Backend > src > text_cleaner.py > ...
585
586 print("\n" + "=" * 70)
587 print("TEXT CLEANER TEST COMPLETE")
588 print("=" * 70)
Problems 1 Output Debug Console Terminal Ports Stanza
powershell - Backend
(venv) PS C:\Users\koppa\Downloads\AI-Coding-Documentation-Assistant\Backend> python src/text_cleaner.py
Category: git
Filename: git_branching.txt
Source: git/git_branching.txt
Cleaned text:
Git branches allow developers to work on
different features independently.
Create a branch:
git branch feature-login
Switch branch:
git checkout feature-login
Create and switch:
git checkout -b feature-login
Merge branch:
git merge feature-login
master*+ 0 1 Git Graph
Ln 588, Col 20 Spaces: 4 UTF-8 CRLF Python Stanza Python 3.14.2
33°C Mostly sunny
Search
ENG IN
01:54 PM 29-09-2026


File Edit Selection View Go Run
AI-Coding-Documentation-Assistant
Problems 1 Output Debug Console Terminal Ports Stanza
powershell - Backend
(venv) PS C:\Users\koppa\Downloads\AI-Coding-Documentation-Assistant\Backend> python src/text_cleaner.py
Category: machine_learning
Filename: ml_basics.txt
Source: machine_learning/ml_basics.txt
Cleaned text:
Machine Learning is a subset of Artificial Intelligence.
Supervised learning uses labelled data.
Regression is used to predict continuous values.
Classification is used to predict categories.
Common algorithms include:
Linear Regression
Logistic Regression
Decision Tree
Random Forest
Support Vector Machine
K-Nearest Neighbors
Category: machine_learning
Filename: regression.txt
Source: machine_learning/regression.txt
Cleaned text:
Linear Regression is a supervised machine learning
algorithm used for predicting continuous numerical values.
The model attempts to find a relationship between
independent variables and a dependent variable.
Example:
master*+ 0 1 Git Graph
Ln 588, Col 20 Spaces: 4 UTF-8 CRLF Python Stanza Python 3.14.2
33°C Mostly sunny
Search
ENG IN
01:55 PM 29-09-2026


File Edit Selection View Go Run
AI-Coding-Documentation-Assistant
Problems 1 Output Debug Console Terminal Ports Stanza
powershell - Backend
(venv) PS C:\Users\koppa\Downloads\AI-Coding-Documentation-Assistant\Backend> python src/text_cleaner.py
Example:
from sklearn.linear_model import LinearRegression
model = LinearRegression()
model.fit(X_train, y_train)
predictions = model.predict(X_test)
Category: git
Filename: git_commands.txt
Source: git/git_commands.txt
Cleaned text:
Git is a distributed version control system.
Git allows developers to track changes in source code.
Common commands:
git init
git clone
git status
git add .
git commit -m "Initial commit"
git push
git pull
master*+ 0 1 Git Graph
Ln 588, Col 20 Spaces: 4 UTF-8 CRLF Python Stanza Python 3.14.2
33°C Mostly sunny
Search
ENG IN
01:55 PM 29-09-2026


File Edit Selection View Go Run
AI-Coding-Documentation-Assistant
Problems 1 Output Debug Console Terminal Ports Stanza
powershell - Backend
(venv) PS C:\Users\koppa\Downloads\AI-Coding-Documentation-Assistant\Backend> python src/text_cleaner.py
SELECT employees.name, departments.department_name
FROM employees
INNER JOIN departments
ON employees.department_id = departments.id;
Category: api
Filename: rest_api.txt
Source: rest_api.txt
Cleaned text:
REST API
GET /api/users
POST /api/users
{
"name": "John",
"age": 25
}
TEXT CLEANER TEST COMPLETE
(venv) PS C:\Users\koppa\Downloads\AI-Coding-Documentation-Assistant\Backend>
master*+ 0 1 Git Graph
Ln 1, Col 1 Spaces: 4 UTF-8 LF Plain Text Stanza
33°C Mostly sunny
Search
ENG IN
01:51 PM 29-09-2026

11. Step 8 — Chunking

This is an important RAG step.

You shouldn't send an entire 100-page document to the LLM.

Instead:

```
Document
   ↓
Chunk 1
Chunk 2
Chunk 3
Chunk 4
...
```

Create:

```
src/chunker.py
```

Example:

```
def chunk_text(text, chunk_size=800, overlap=100):

    chunks = []

    start = 0

    while start < len(text):

        end = start + chunk_size

        chunk = text[start:end]

        chunks.append(chunk)

        start += chunk_size - overlap

    return chunks
```

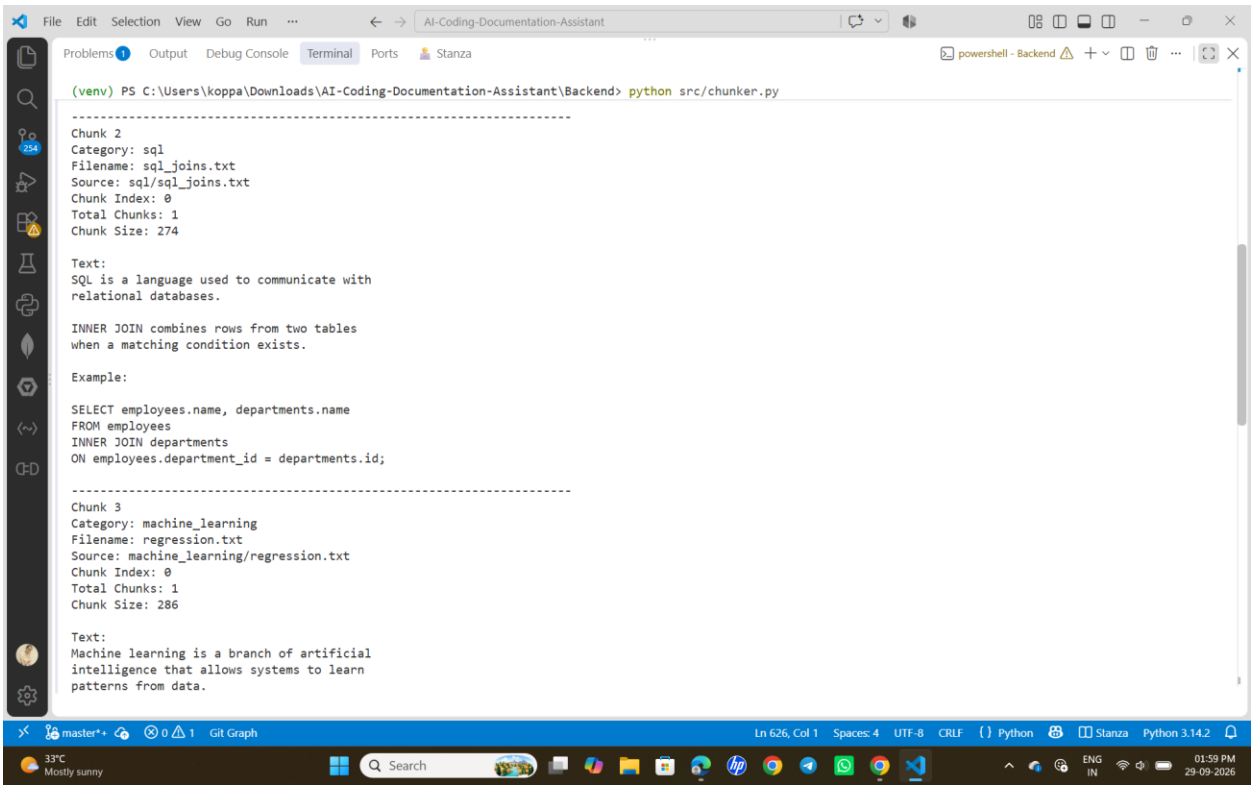
For your project, start with:

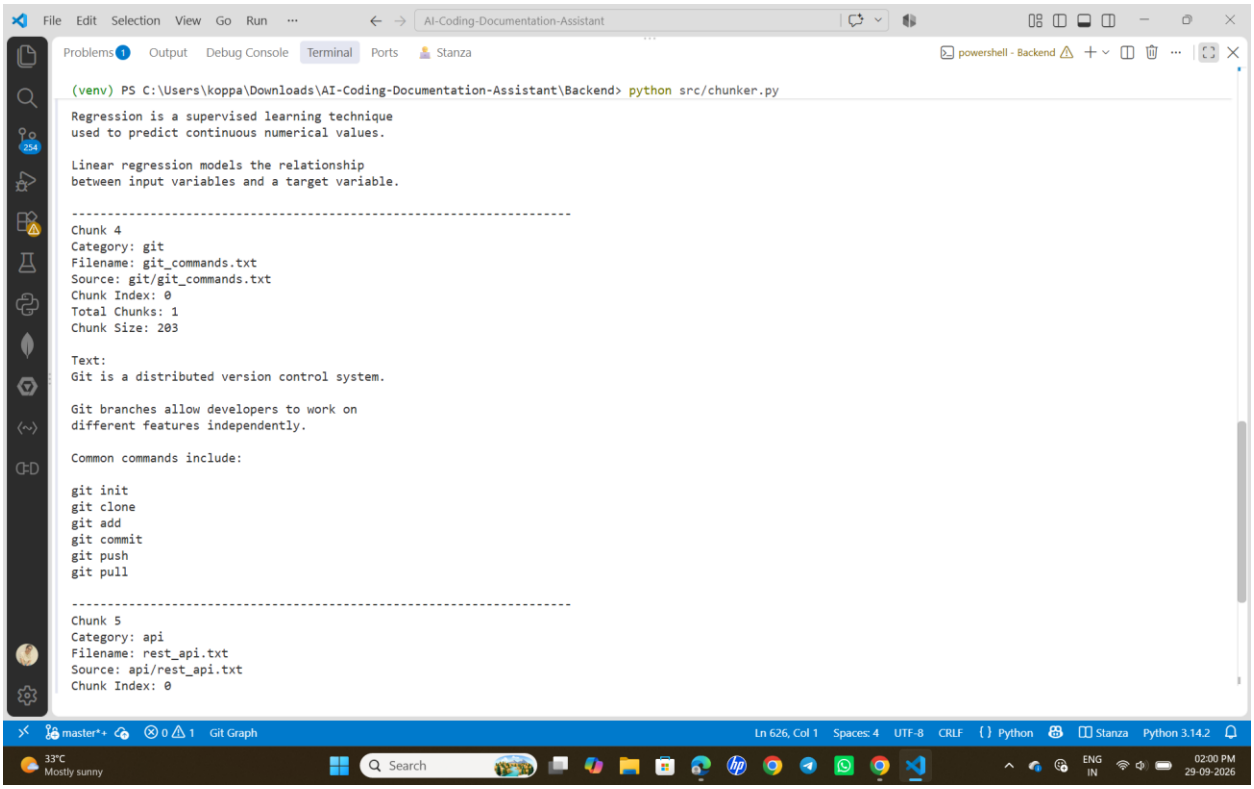
```
chunk_size = 800
overlap = 100
```

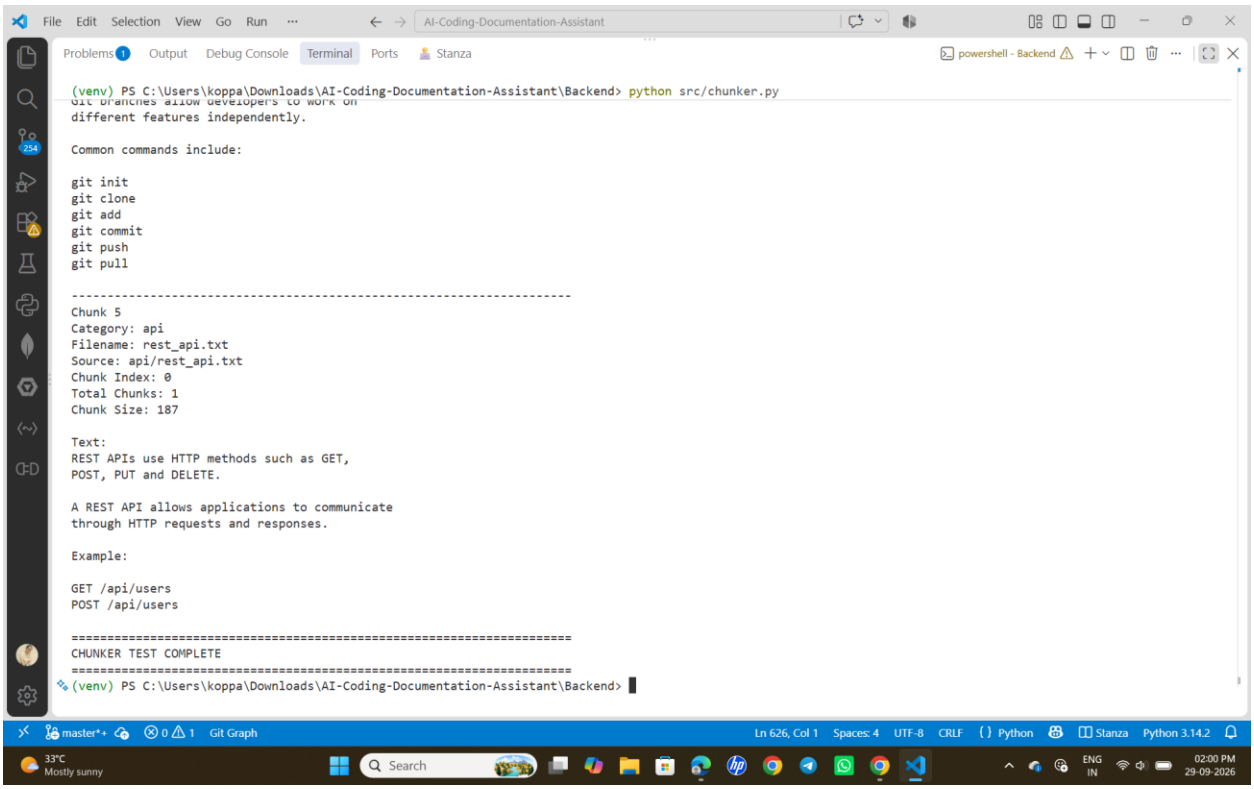
You can later experiment with:

```
500
800
1000
1200
```

```
Outputs:
```


File Edit Selection View Go Run
AI-Coding-Documentation-Assistant
Problems 1 Output Debug Console Terminal Ports Stanza
powershell - Backend
(venv) PS C:\Users\koppa\Downloads\AI-Coding-Documentation-Assistant\Backend> python src/chunker.py
Chunk 2
Category: sql
Filename: sql_joins.txt
Source: sql/sql_joins.txt
Chunk Index: 0
Total Chunks: 1
Chunk Size: 274
Text:
SQL is a language used to communicate with
relational databases.
INNER JOIN combines rows from two tables
when a matching condition exists.
Example:
SELECT employees.name, departments.name
FROM employees
INNER JOIN departments
ON employees.department_id = departments.id;
Chunk 3
Category: machine_learning
Filename: regression.txt
Source: machine_learning/regression.txt
Chunk Index: 0
Total Chunks: 1
Chunk Size: 286
Text:
Machine learning is a branch of artificial
intelligence that allows systems to learn
patterns from data.
master*+ 0 1 Git Graph
Ln 626, Col 1 Spaces: 4 UTF-8 CRLF Python Stanza Python 3.14.2
33°C Mostly sunny
Search
ENG IN
01:59 PM 29-09-2026


File Edit Selection View Go Run
AI-Coding-Documentation-Assistant
Problems 1 Output Debug Console Terminal Ports Stanza
powershell - Backend
(venv) PS C:\Users\koppa\Downloads\AI-Coding-Documentation-Assistant\Backend> python src/chunker.py
Regression is a supervised learning technique
used to predict continuous numerical values.
Linear regression models the relationship
between input variables and a target variable.
Chunk 4
Category: git
Filename: git_commands.txt
Source: git/git_commands.txt
Chunk Index: 0
Total Chunks: 1
Chunk Size: 203
Text:
Git is a distributed version control system.
Git branches allow developers to work on
different features independently.
Common commands include:
git init
git clone
git add
git commit
git push
git pull
Chunk 5
Category: api
Filename: rest_api.txt
Source: api/rest_api.txt
Chunk Index: 0
master*+ 0 1 Git Graph
Ln 626, Col 1 Spaces: 4 UTF-8 CRLF Python Stanza Python 3.14.2
33°C Mostly sunny
Search
ENG IN
02:00 PM 29-09-2026



12. Step 9 — Create Embeddings

Now convert each chunk into a numerical vector.

Example:

```
"Python lists are mutable"
              ↓
Embedding Model
              ↓
[0.021, -0.145, 0.781, ...]
```

Use:

```
sentence-transformers
```

A good starting model:

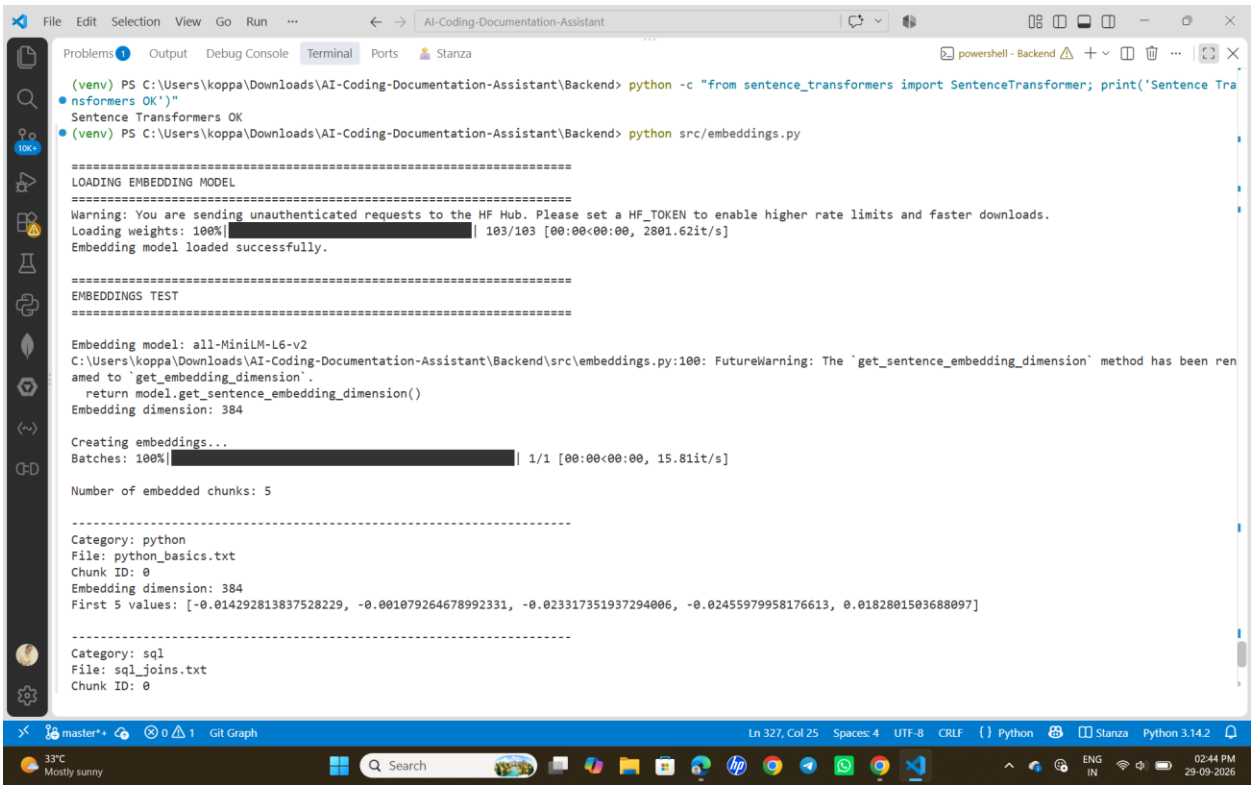
```
all-MiniLM-L6-v2
```

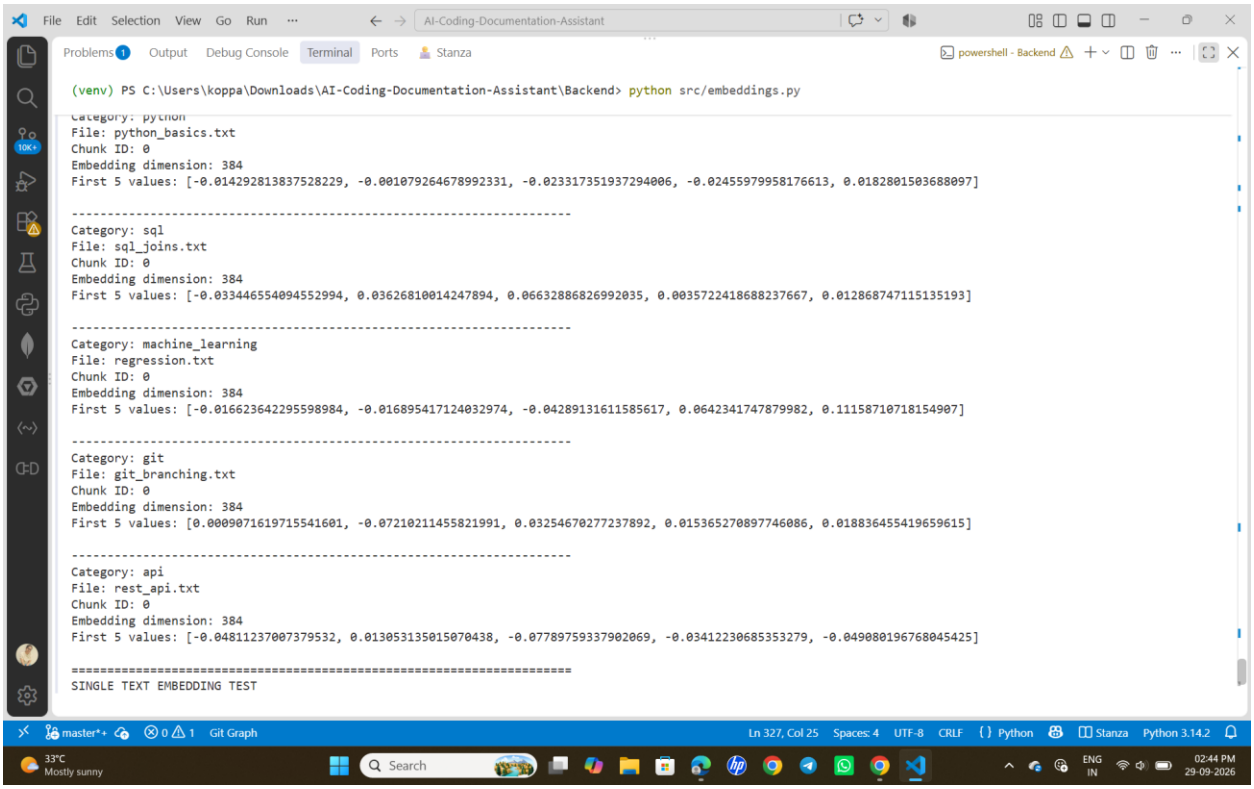
Create:

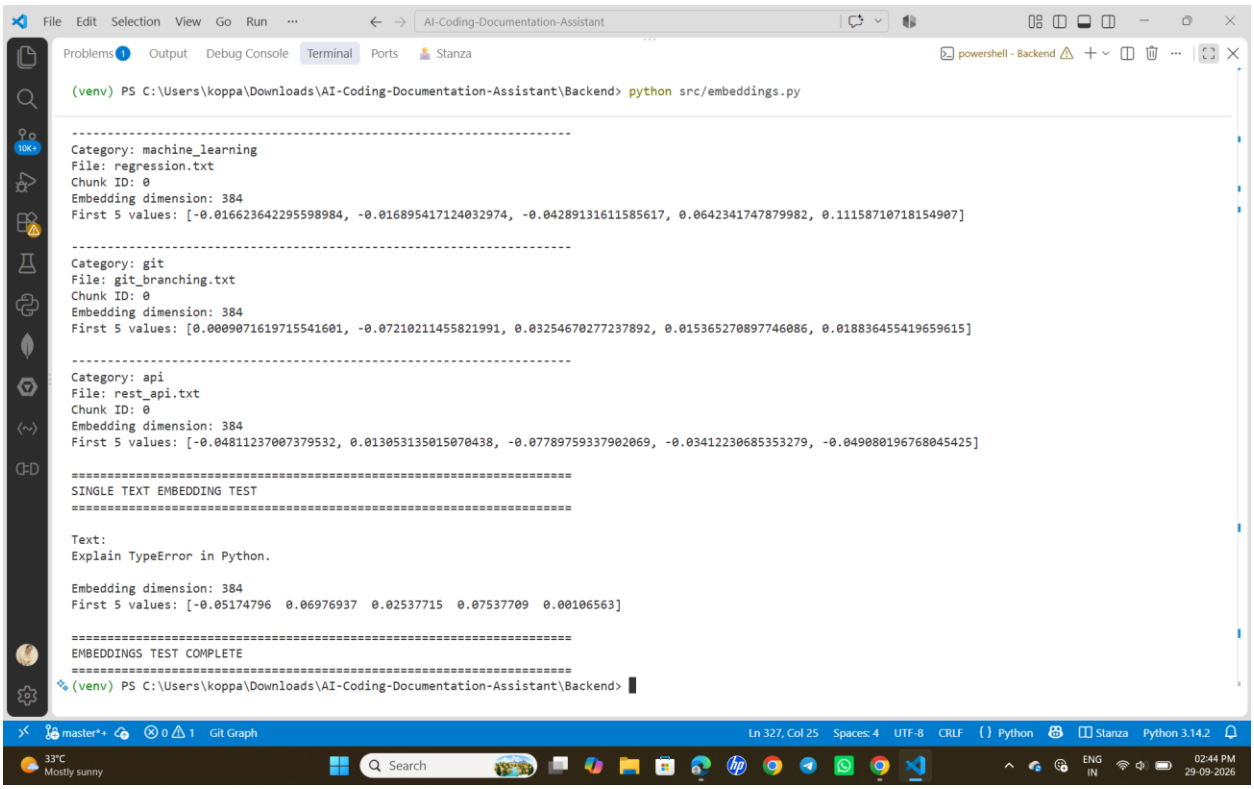
```
src/embeddings.py
```

Example:

```
from sentence_transformers import SentenceTransformermodel =
SentenceTransformer(    "all-MiniLM-L6-v2")def
create_embeddings(texts):    embeddings = model.encode(        texts,
convert_to_numpy=True    )    return embeddings
```


File Edit Selection View Go Run
AI-Coding-Documentation-Assistant
Problems 1 Output Debug Console Terminal Ports Stanza
powershell - Backend
(venv) PS C:\Users\koppa\Downloads\AI-Coding-Documentation-Assistant\Backend> python -c "from sentence_transformers import SentenceTransformer; print('Sentence Transformers OK')"
Sentence Transformers OK
(venv) PS C:\Users\koppa\Downloads\AI-Coding-Documentation-Assistant\Backend> python src/embeddings.py
LOADING EMBEDDING MODEL
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
Loading weights: 100%| 103/103 [00:00<00:00, 2801.62it/s]
Embedding model loaded successfully.
EMBEDDINGS TEST
Embedding model: all-MiniLM-L6-v2
C:\Users\koppa\Downloads\AI-Coding-Documentation-Assistant\Backend\src\embeddings.py:100: FutureWarning: The `get_sentence_embedding_dimension` method has been renamed to `get_embedding_dimension`.
return model.get_sentence_embedding_dimension()
Embedding dimension: 384
Creating embeddings...
Batches: 100%| 1/1 [00:00<00:00, 15.81it/s]
Number of embedded chunks: 5
Category: python
File: python_basics.txt
Chunk ID: 0
Embedding dimension: 384
First 5 values: [-0.014292813837528229, -0.001079264678992331, -0.023317351937294006, -0.02455979958176613, 0.0182801503688097]
Category: sql
File: sql_joins.txt
Chunk ID: 0
master*+ 0 1 Git Graph
Ln 327, Col 25 Spaces: 4 UTF-8 CRLF Python Stanza Python 3.14.2
33°C Mostly sunny
Search
ENG IN
02:44 PM 29-09-2026


File Edit Selection View Go Run
AI-Coding-Documentation-Assistant
Problems 1 Output Debug Console Terminal Ports Stanza
powershell - Backend
(venv) PS C:\Users\koppa\Downloads\AI-Coding-Documentation-Assistant\Backend> python src/embeddings.py
File: python_basics.txt
Chunk ID: 0
Embedding dimension: 384
First 5 values: [-0.014292813837528229, -0.001079264678992331, -0.023317351937294006, -0.02455979958176613, 0.0182801503688097]
Category: sql
File: sql_joins.txt
Chunk ID: 0
Embedding dimension: 384
First 5 values: [-0.033446554094552994, 0.03626810014247894, 0.06632886826992035, 0.0035722418688237667, 0.012868747115135193]
Category: machine_learning
File: regression.txt
Chunk ID: 0
Embedding dimension: 384
First 5 values: [-0.016623642295598984, -0.016895417124032974, -0.04289131611585617, 0.0642341747879982, 0.11158710718154907]
Category: git
File: git_branching.txt
Chunk ID: 0
Embedding dimension: 384
First 5 values: [0.0009071619715541601, -0.07210211455821991, 0.03254670277237892, 0.015365270897746086, 0.018836455419659615]
Category: api
File: rest_api.txt
Chunk ID: 0
Embedding dimension: 384
First 5 values: [-0.04811237007379532, 0.013053135015070438, -0.07789759337902069, -0.03412230685353279, -0.049080196768045425]
SINGLE TEXT EMBEDDING TEST
master*+ 0 1 Git Graph
Ln 327, Col 25 Spaces: 4 UTF-8 CRLF Python Stanza Python 3.14.2
33°C Mostly sunny
Search
ENG IN
02:44 PM 29-09-2026



13. Step 10 — Vector Database

Now store your embeddings.

You can use **FAISS**.

Create:

`src/vector_store.py`

Concept:

```
Chunk
 ↓
Embedding
 ↓
FAISS
```

For example:

```
import faiss
import numpy as np


class VectorStore:

    def __init__(self, dimension):

        self.index = faiss.IndexFlatL2(
            dimension
        )

        self.documents = []


    def add(self, embeddings, documents):

        embeddings = np.array(
            embeddings
        ).astype("float32")

        self.index.add(embeddings)

        self.documents.extend(
            documents
        )


    def search(self, query_embedding, k=5):

        query_embedding = np.array(
            [query_embedding]
        ).astype("float32")

        distances, indices = self.index.search(
            query_embedding,
            k
        )

        results = []
```

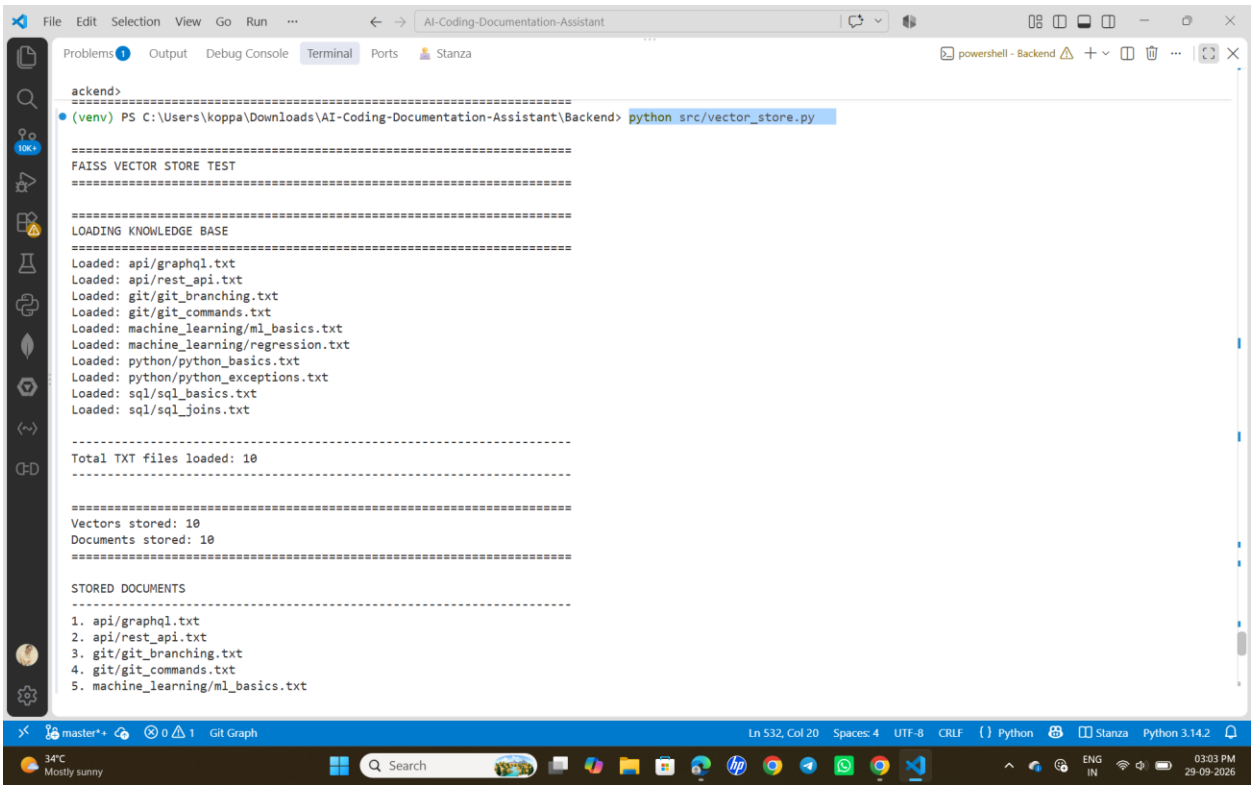
```
        for index in indices[0]:

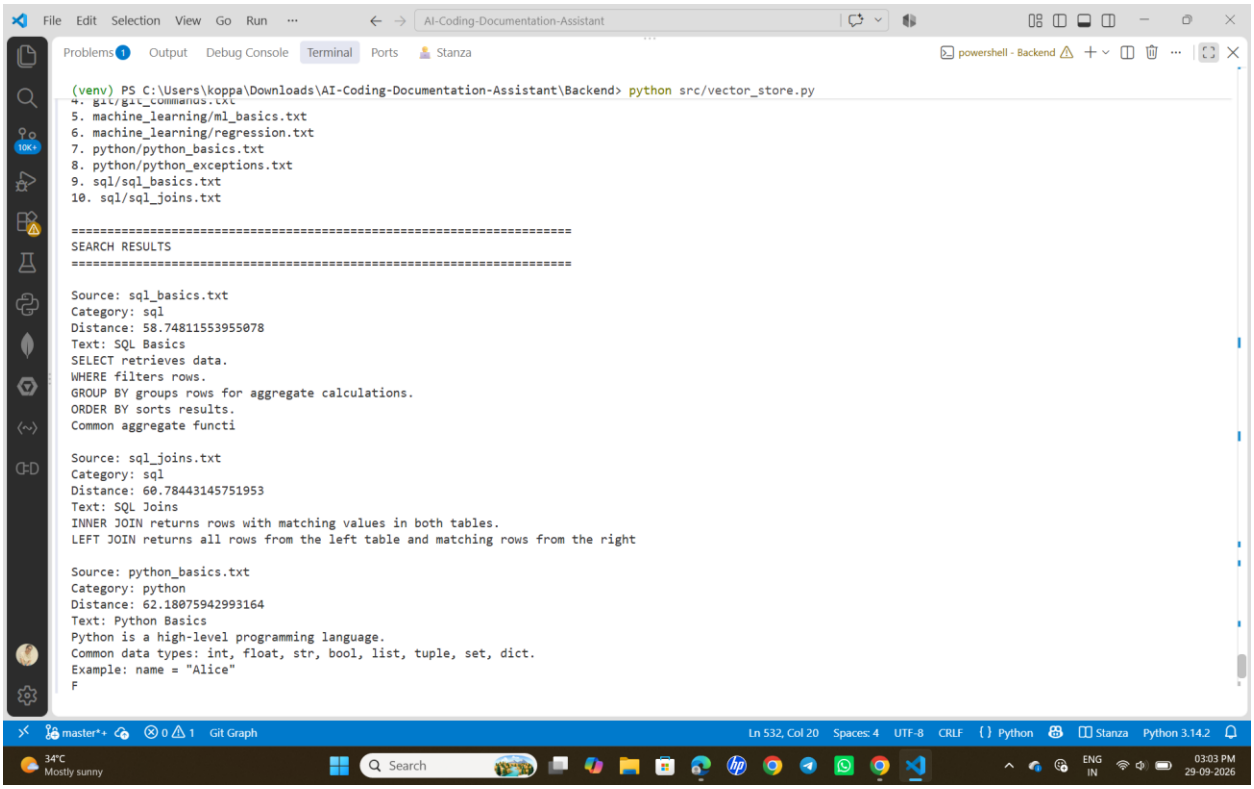
            if index < len(self.documents):

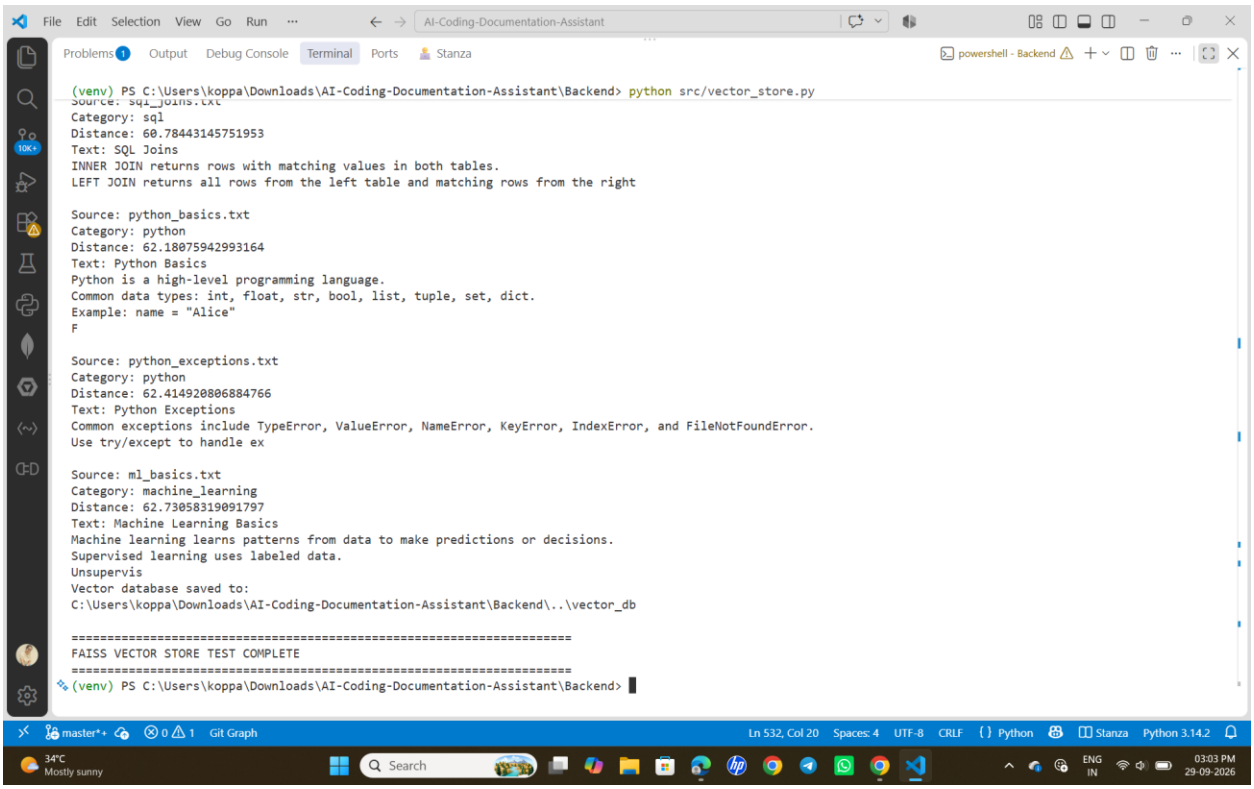
                results.append(
                    self.documents[index]
                )

        return results
```

Output:


File Edit Selection View Go Run
AI-Coding-Documentation-Assistant
Problems 1 Output Debug Console Terminal Ports Stanza
powershell - Backend
ackend>
(venv) PS C:\Users\koppa\Downloads\AI-Coding-Documentation-Assistant\Backend> python src/vector_store.py
FAISS VECTOR STORE TEST
LOADING KNOWLEDGE BASE
Loaded: api/graphql.txt
Loaded: api/rest_api.txt
Loaded: git/git_branching.txt
Loaded: git/git_commands.txt
Loaded: machine_learning/ml_basics.txt
Loaded: machine_learning/regression.txt
Loaded: python/python_basics.txt
Loaded: python/python_exceptions.txt
Loaded: sql/sql_basics.txt
Loaded: sql/sql_joins.txt
Total TXT files loaded: 10
Vectors stored: 10
Documents stored: 10
STORED DOCUMENTS
1. api/graphql.txt
2. api/rest_api.txt
3. git/git_branching.txt
4. git/git_commands.txt
5. machine_learning/ml_basics.txt
master*+ 0 1 Git Graph
Ln 532, Col 20 Spaces: 4 UTF-8 CRLF Python Stanza Python 3.14.2
34°C Mostly sunny
Search
ENG IN
03:03 PM 29-09-2026


File Edit Selection View Go Run
AI-Coding-Documentation-Assistant
Problems 1 Output Debug Console Terminal Ports Stanza
powershell - Backend
(venv) PS C:\Users\koppa\Downloads\AI-Coding-Documentation-Assistant\Backend> python src/vector_store.py
4. git/git_commands.txt
5. machine_learning/ml_basics.txt
6. machine_learning/regression.txt
7. python/python_basics.txt
8. python/python_exceptions.txt
9. sql/sql_basics.txt
10. sql/sql_joins.txt
SEARCH RESULTS
Source: sql_basics.txt
Category: sql
Distance: 58.74811553955078
Text: SQL Basics
SELECT retrieves data.
WHERE filters rows.
GROUP BY groups rows for aggregate calculations.
ORDER BY sorts results.
Common aggregate functi
Source: sql_joins.txt
Category: sql
Distance: 60.78443145751953
Text: SQL Joins
INNER JOIN returns rows with matching values in both tables.
LEFT JOIN returns all rows from the left table and matching rows from the right
Source: python_basics.txt
Category: python
Distance: 62.18075942993164
Text: Python Basics
Python is a high-level programming language.
Common data types: int, float, str, bool, list, tuple, set, dict.
Example: name = "Alice"
F
master*+ 0 1 Git Graph
Ln 532, Col 20 Spaces: 4 UTF-8 CRLF Python Stanza Python 3.14.2
34°C Mostly sunny
Search
ENG IN
03:03 PM 29-09-2026



14. Step 11 — Build the Retriever

Create:

`src/retriever.py`

The retriever does:

```
User Question
      ↓
Question Embedding
      ↓
Vector Search
      ↓
Top 5 relevant chunks
```

Example:

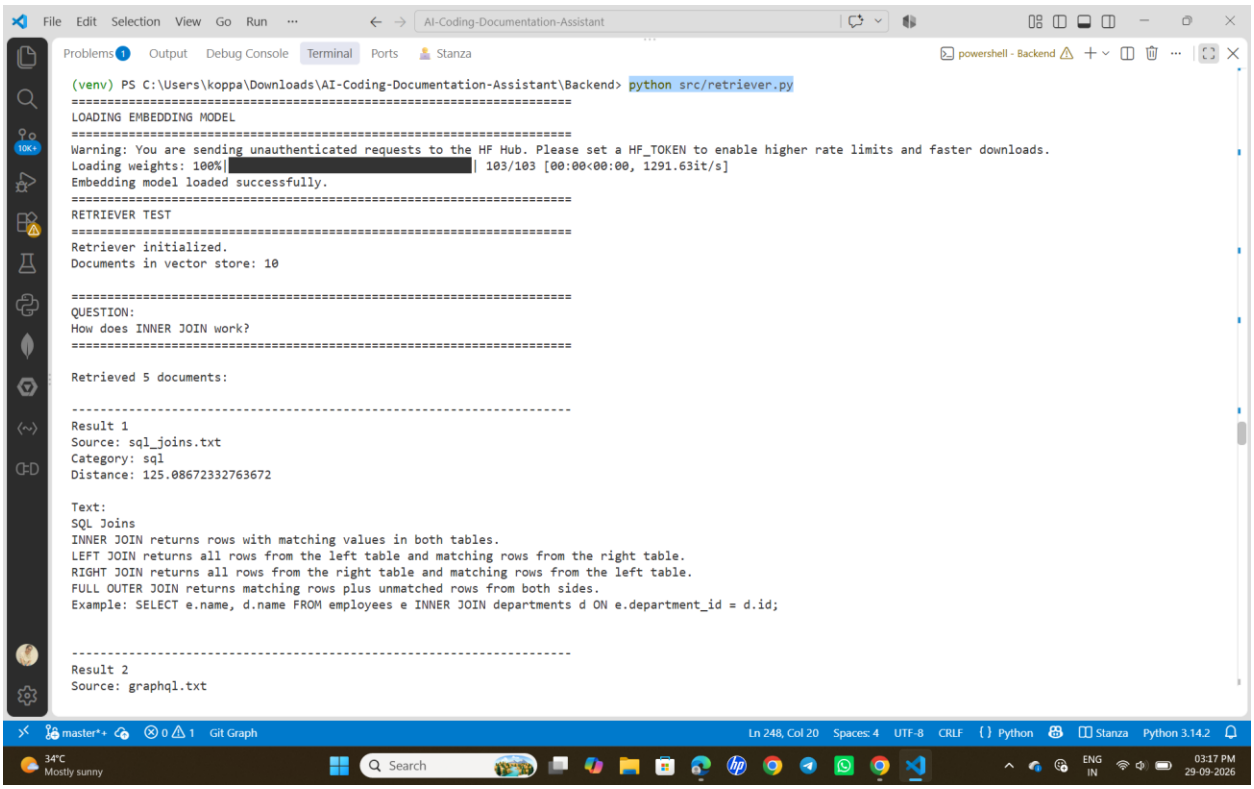
User asks:

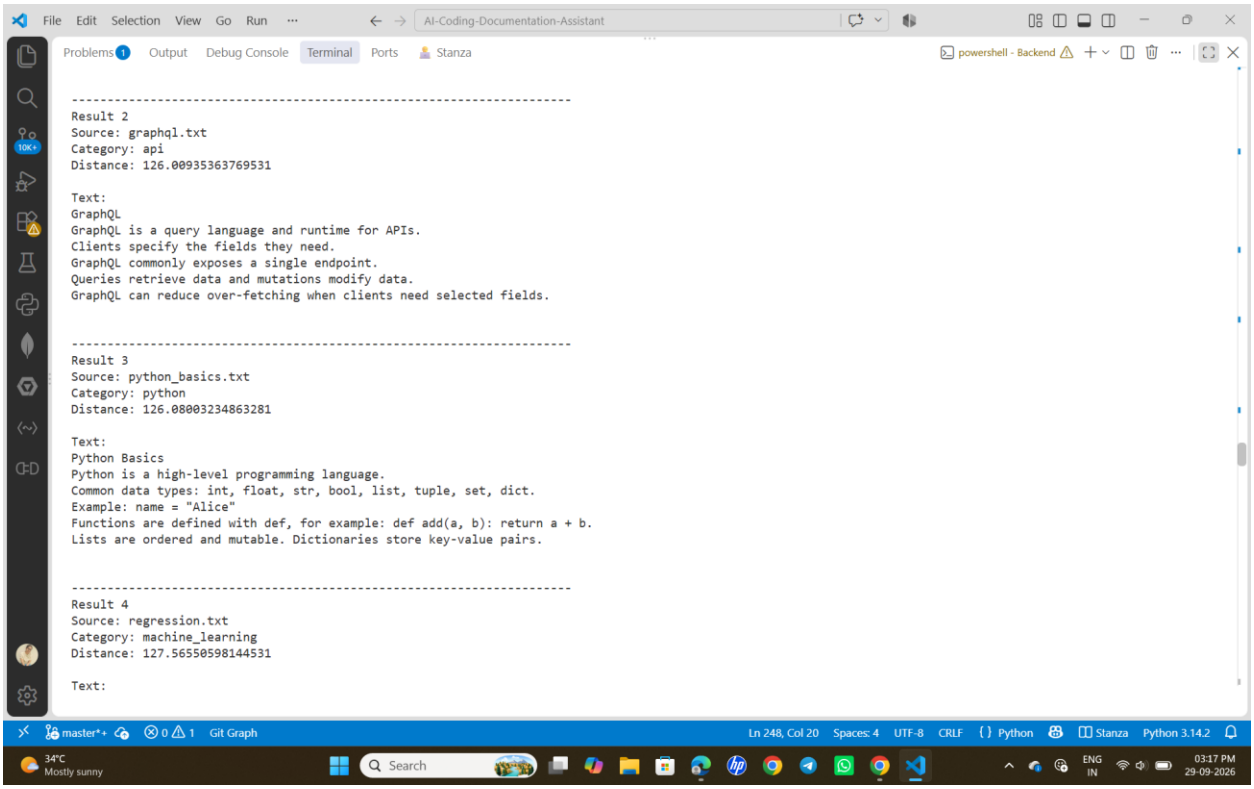
```
How does INNER JOIN work?
```

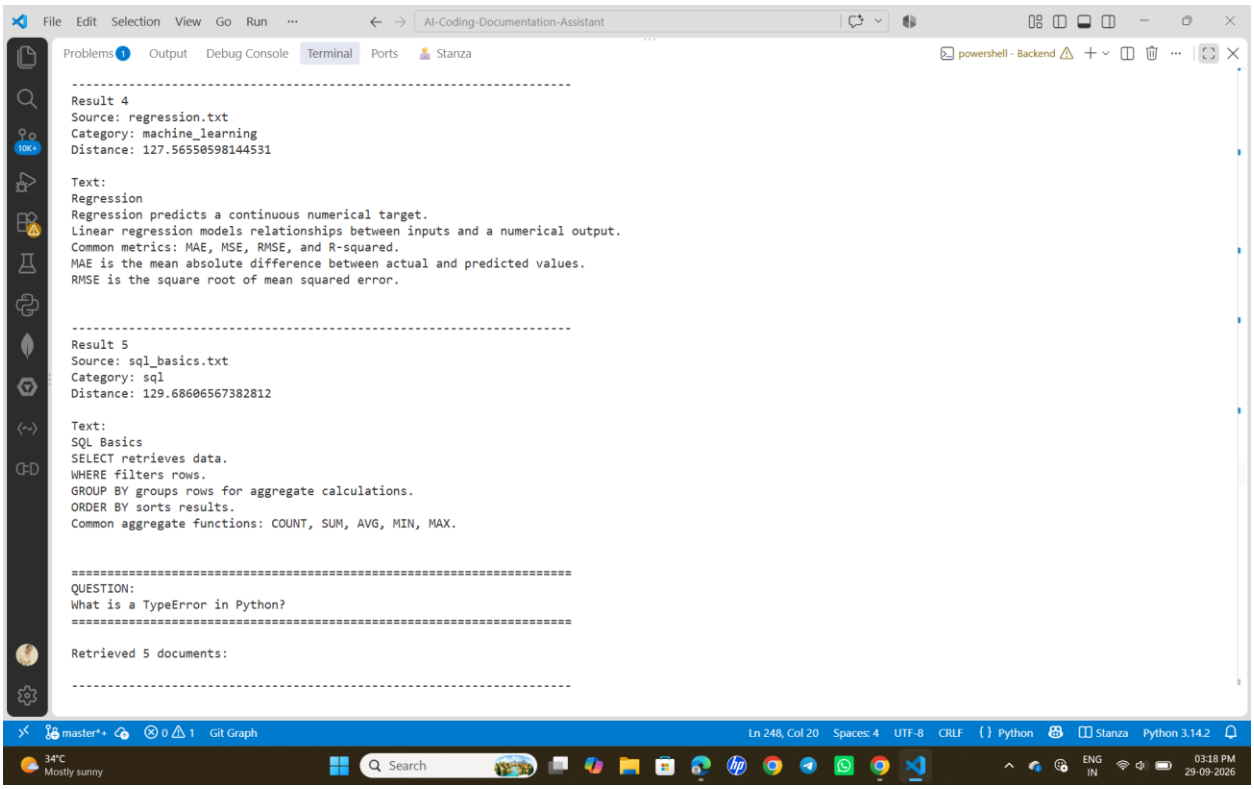
The vector search may retrieve:

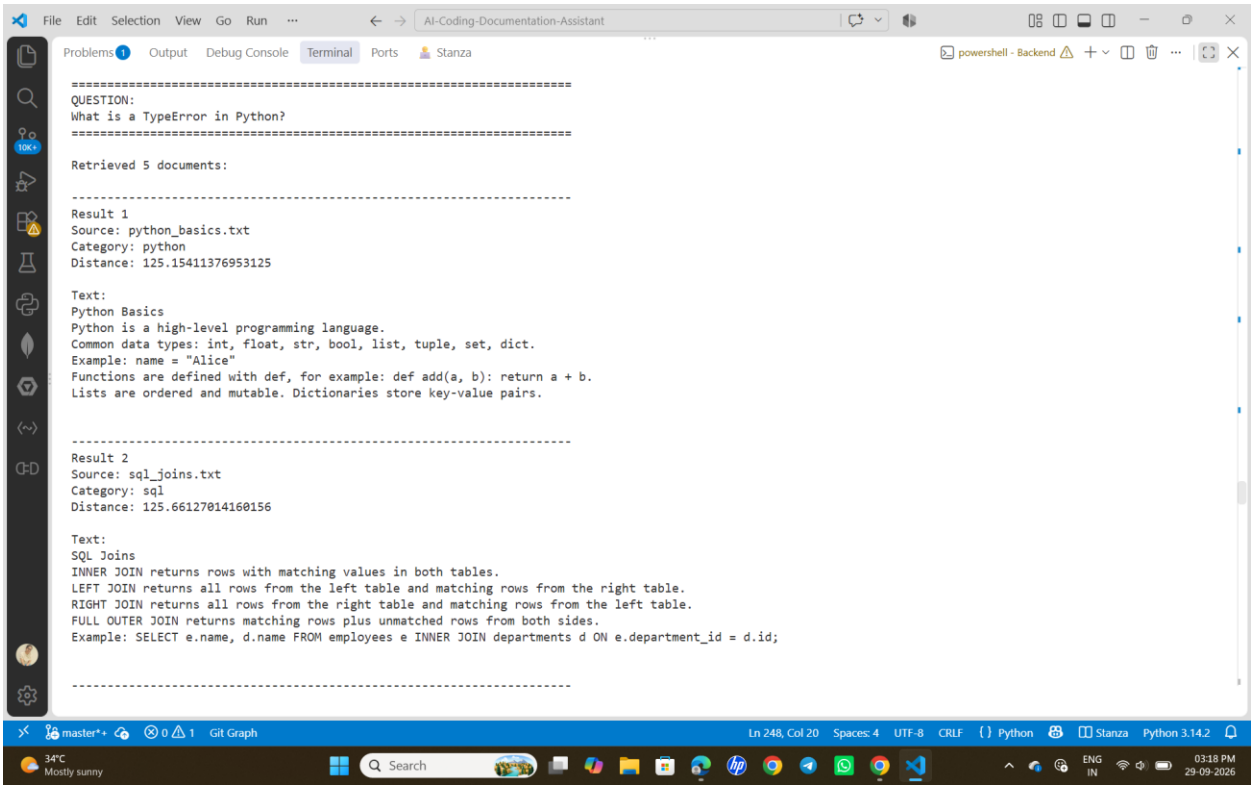
```
SQL Joins documentation
SQL INNER JOIN examples
SQL relational database notes
```

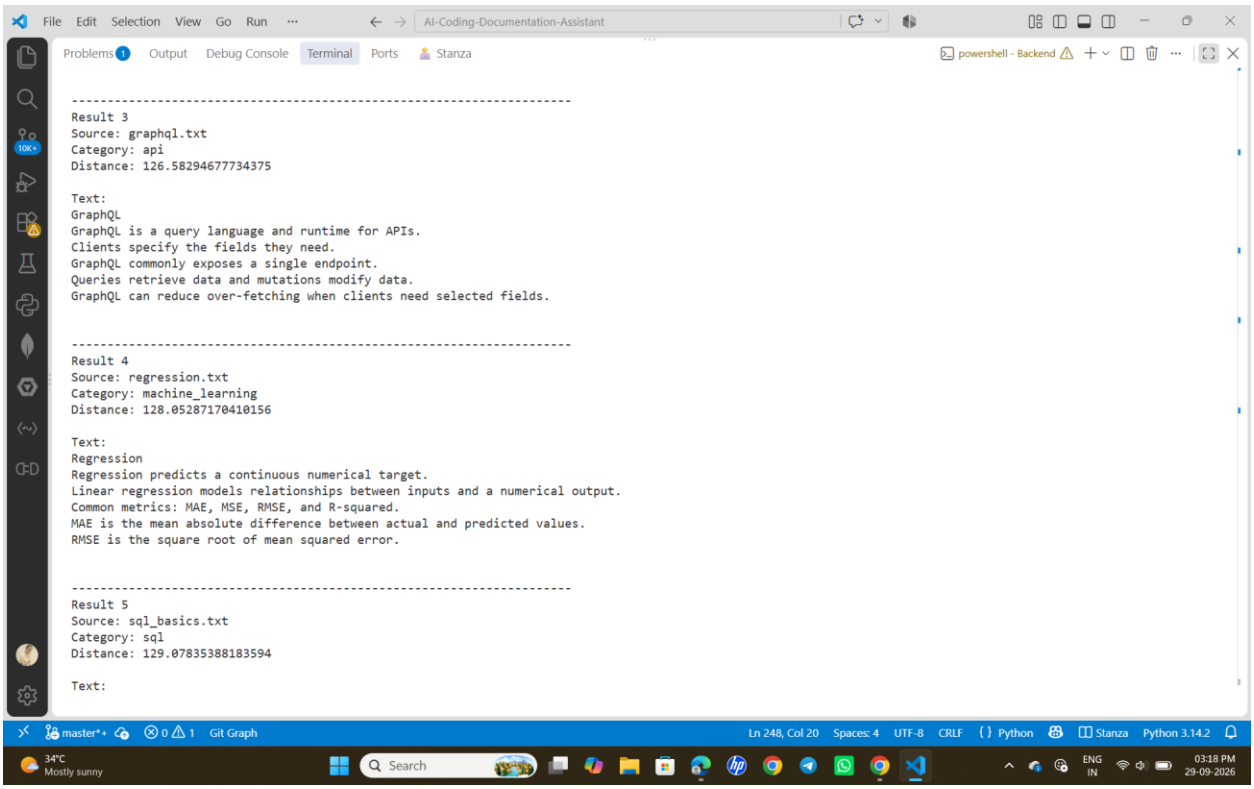
instead of unrelated Python documentation.

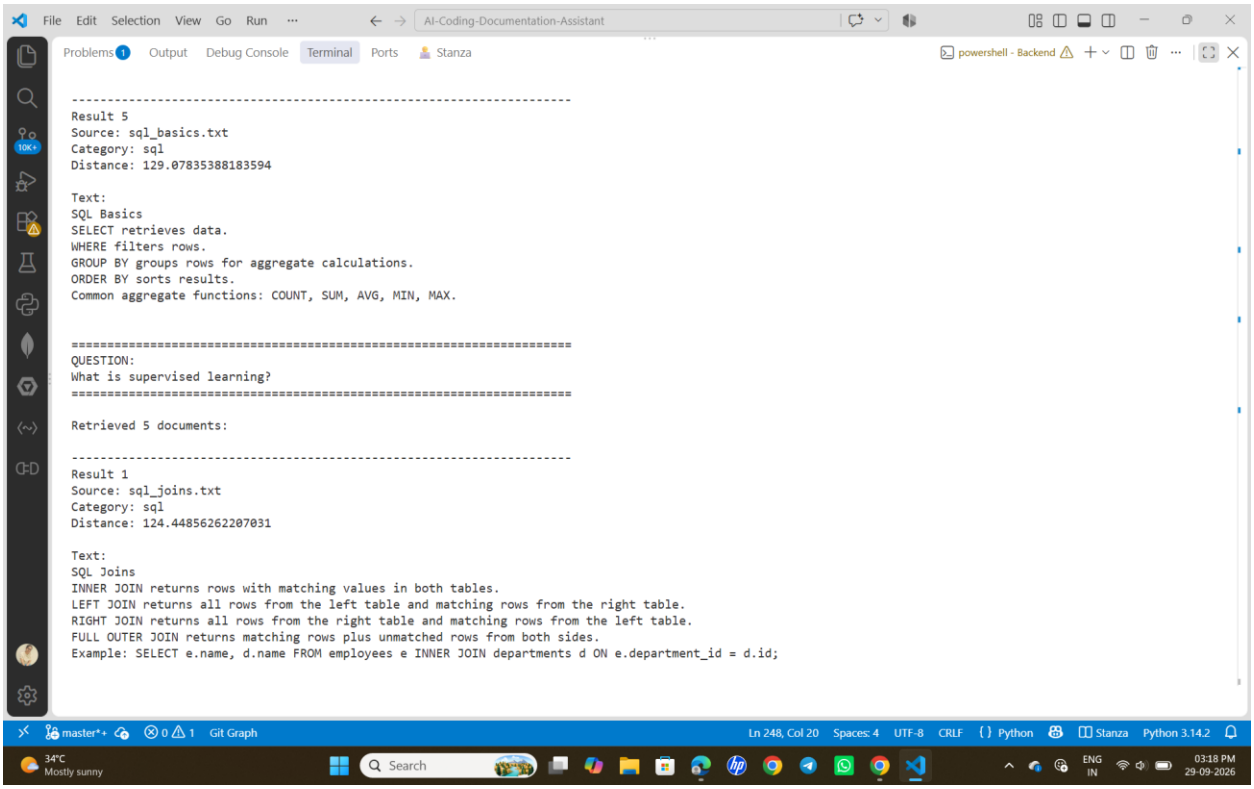

File Edit Selection View Go Run ···
AI-Coding-Documentation-Assistant
Problems 1 Output Debug Console Terminal Ports Stanza
powershell - Backend
(venv) PS C:\Users\koppa\Downloads\AI-Coding-Documentation-Assistant\Backend> python src/retriever.py
LOADING EMBEDDING MODEL
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
Loading weights: 100%| 103/103 [00:00<00:00, 1291.63it/s]
Embedding model loaded successfully.
RETRIEVER TEST
Retriever initialized.
Documents in vector store: 10
QUESTION:
How does INNER JOIN work?
Retrieved 5 documents:
Result 1
Source: sql_joins.txt
Category: sql
Distance: 125.08672332763672
Text:
SQL Joins
INNER JOIN returns rows with matching values in both tables.
LEFT JOIN returns all rows from the left table and matching rows from the right table.
RIGHT JOIN returns all rows from the right table and matching rows from the left table.
FULL OUTER JOIN returns matching rows plus unmatched rows from both sides.
Example: SELECT e.name, d.name FROM employees e INNER JOIN departments d ON e.department_id = d.id;
Result 2
Source: graphql.txt
master*+ 0 1 Git Graph Ln 248, Col 20 Spaces: 4 UTF-8 CRLF Python Stanza Python 3.14.2
34°C Mostly sunny
Search
ENG IN
03:17 PM 29-09-2026

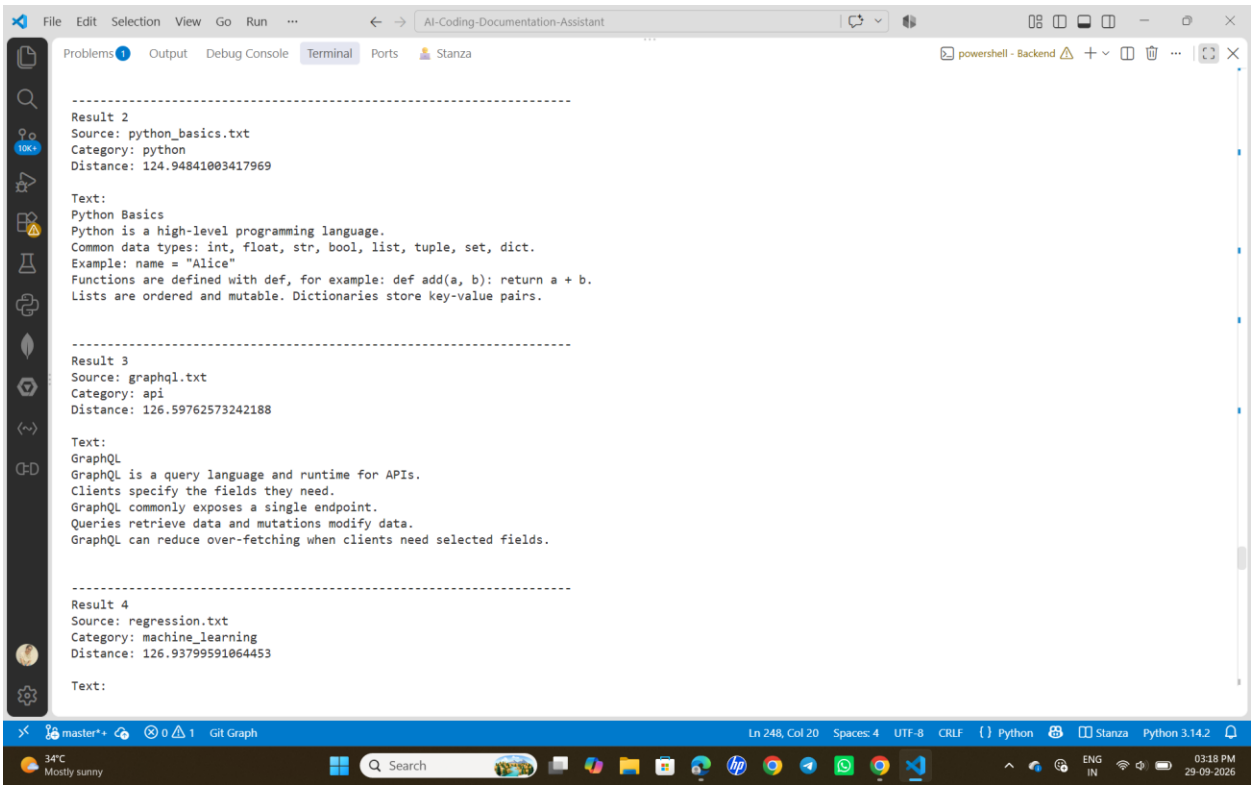

File Edit Selection View Go Run ···
AI-Coding-Documentation-Assistant
Problems 1 Output Debug Console Terminal Ports Stanza
powershell - Backend
Result 2
Source: graphql.txt
Category: api
Distance: 126.00935363769531
Text:
GraphQL
GraphQL is a query language and runtime for APIs.
Clients specify the fields they need.
GraphQL commonly exposes a single endpoint.
Queries retrieve data and mutations modify data.
GraphQL can reduce over-fetching when clients need selected fields.
Result 3
Source: python_basics.txt
Category: python
Distance: 126.08003234863281
Text:
Python Basics
Python is a high-level programming language.
Common data types: int, float, str, bool, list, tuple, set, dict.
Example: name = "Alice"
Functions are defined with def, for example: def add(a, b): return a + b.
Lists are ordered and mutable. Dictionaries store key-value pairs.
Result 4
Source: regression.txt
Category: machine_learning
Distance: 127.56550598144531
Text:
master*+ 0 1 Git Graph Ln 248, Col 20 Spaces: 4 UTF-8 CRLF Python Stanza Python 3.14.2
34°C Mostly sunny
Search
ENG IN
03:17 PM 29-09-2026

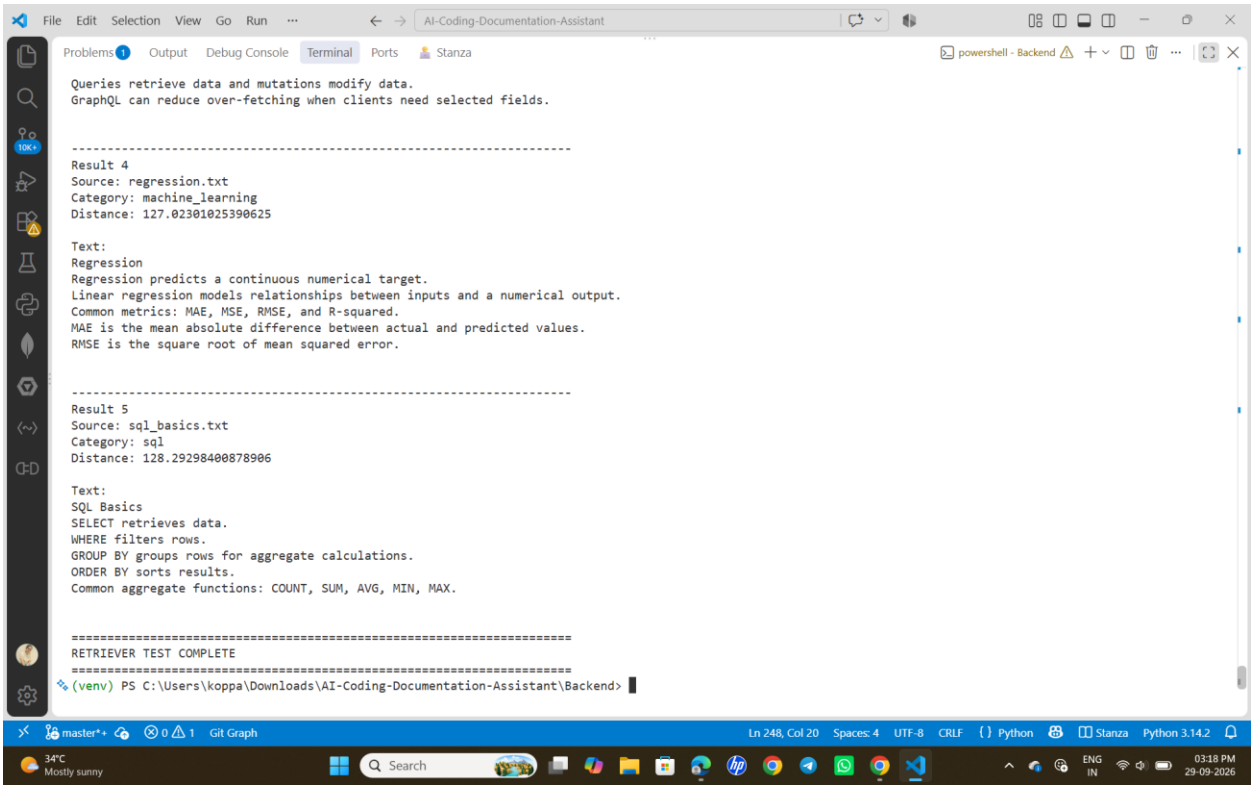

File Edit Selection View Go Run
AI-Coding-Documentation-Assistant
Problems 1 Output Debug Console Terminal Ports Stanza
powershell - Backend
Result 4
Source: regression.txt
Category: machine_learning
Distance: 127.56550598144531
Text:
Regression
Regression predicts a continuous numerical target.
Linear regression models relationships between inputs and a numerical output.
Common metrics: MAE, MSE, RMSE, and R-squared.
MAE is the mean absolute difference between actual and predicted values.
RMSE is the square root of mean squared error.
Result 5
Source: sql_basics.txt
Category: sql
Distance: 129.68606567382812
Text:
SQL Basics
SELECT retrieves data.
WHERE filters rows.
GROUP BY groups rows for aggregate calculations.
ORDER BY sorts results.
Common aggregate functions: COUNT, SUM, AVG, MIN, MAX.
QUESTION:
What is a TypeError in Python?
Retrieved 5 documents:
master*+ 0 1 Git Graph
Ln 248, Col 20 Spaces: 4 UTF-8 CRLF Python Stanza Python 3.14.2
34°C Mostly sunny
Search
ENG IN
03:18 PM 29-09-2026


File Edit Selection View Go Run
AI-Coding-Documentation-Assistant
Problems 1 Output Debug Console Terminal Ports Stanza
powershell - Backend
QUESTION:
What is a TypeError in Python?
Retrieved 5 documents:
Result 1
Source: python_basics.txt
Category: python
Distance: 125.15411376953125
Text:
Python Basics
Python is a high-level programming language.
Common data types: int, float, str, bool, list, tuple, set, dict.
Example: name = "Alice"
Functions are defined with def, for example: def add(a, b): return a + b.
Lists are ordered and mutable. Dictionaries store key-value pairs.
Result 2
Source: sql_joins.txt
Category: sql
Distance: 125.66127014160156
Text:
SQL Joins
INNER JOIN returns rows with matching values in both tables.
LEFT JOIN returns all rows from the left table and matching rows from the right table.
RIGHT JOIN returns all rows from the right table and matching rows from the left table.
FULL OUTER JOIN returns matching rows plus unmatched rows from both sides.
Example: SELECT e.name, d.name FROM employees e INNER JOIN departments d ON e.department_id = d.id;
master*+ 0 1 Git Graph
Ln 248, Col 20 Spaces: 4 UTF-8 CRLF Python Stanza Python 3.14.2
34°C Mostly sunny
Search
ENG IN
03:18 PM 29-09-2026


File Edit Selection View Go Run
AI-Coding-Documentation-Assistant
Problems 1 Output Debug Console Terminal Ports Stanza
powershell - Backend
Result 3
Source: graphql.txt
Category: api
Distance: 126.58294677734375
Text:
GraphQL
GraphQL is a query language and runtime for APIs.
Clients specify the fields they need.
GraphQL commonly exposes a single endpoint.
Queries retrieve data and mutations modify data.
GraphQL can reduce over-fetching when clients need selected fields.
Result 4
Source: regression.txt
Category: machine_learning
Distance: 128.05287170410156
Text:
Regression
Regression predicts a continuous numerical target.
Linear regression models relationships between inputs and a numerical output.
Common metrics: MAE, MSE, RMSE, and R-squared.
MAE is the mean absolute difference between actual and predicted values.
RMSE is the square root of mean squared error.
Result 5
Source: sql_basics.txt
Category: sql
Distance: 129.07835388183594
Text:
master*+ 0 1 Git Graph
Ln 248, Col 20 Spaces: 4 UTF-8 CRLF Python Stanza Python 3.14.2
34°C Mostly sunny
Search
ENG IN
03:18 PM 29-09-2026


File Edit Selection View Go Run
AI-Coding-Documentation-Assistant
Problems 1 Output Debug Console Terminal Ports Stanza
powershell - Backend
Result 5
Source: sql_basics.txt
Category: sql
Distance: 129.07835388183594
Text:
SQL Basics
SELECT retrieves data.
WHERE filters rows.
GROUP BY groups rows for aggregate calculations.
ORDER BY sorts results.
Common aggregate functions: COUNT, SUM, AVG, MIN, MAX.
QUESTION:
What is supervised learning?
Retrieved 5 documents:
Result 1
Source: sql_joins.txt
Category: sql
Distance: 124.44856262207031
Text:
SQL Joins
INNER JOIN returns rows with matching values in both tables.
LEFT JOIN returns all rows from the left table and matching rows from the right table.
RIGHT JOIN returns all rows from the right table and matching rows from the left table.
FULL OUTER JOIN returns matching rows plus unmatched rows from both sides.
Example: SELECT e.name, d.name FROM employees e INNER JOIN departments d ON e.department_id = d.id;
master*+ 0 1 Git Graph
Ln 248, Col 20 Spaces: 4 UTF-8 CRLF Python Stanza Python 3.14.2
34°C Mostly sunny
Search
ENG IN
03:18 PM 29-09-2026


File Edit Selection View Go Run
AI-Coding-Documentation-Assistant
Problems 1 Output Debug Console Terminal Ports Stanza
powershell - Backend
Result 2
Source: python_basics.txt
Category: python
Distance: 124.94841003417969
Text:
Python Basics
Python is a high-level programming language.
Common data types: int, float, str, bool, list, tuple, set, dict.
Example: name = "Alice"
Functions are defined with def, for example: def add(a, b): return a + b.
Lists are ordered and mutable. Dictionaries store key-value pairs.
Result 3
Source: graphql.txt
Category: api
Distance: 126.59762573242188
Text:
GraphQL
GraphQL is a query language and runtime for APIs.
Clients specify the fields they need.
GraphQL commonly exposes a single endpoint.
Queries retrieve data and mutations modify data.
GraphQL can reduce over-fetching when clients need selected fields.
Result 4
Source: regression.txt
Category: machine_learning
Distance: 126.93799591064453
Text:
master*+ 0 1 Git Graph
Ln 248, Col 20 Spaces: 4 UTF-8 CRLF Python Stanza Python 3.14.2
34°C Mostly sunny
Search
ENG IN
03:18 PM 29-09-2026


File Edit Selection View Go Run
AI-Coding-Documentation-Assistant
Problems 1 Output Debug Console Terminal Ports Stanza
powershell - Backend
Queries retrieve data and mutations modify data.
GraphQL can reduce over-fetching when clients need selected fields.
Result 4
Source: regression.txt
Category: machine_learning
Distance: 127.02301025390625
Text:
Regression
Regression predicts a continuous numerical target.
Linear regression models relationships between inputs and a numerical output.
Common metrics: MAE, MSE, RMSE, and R-squared.
MAE is the mean absolute difference between actual and predicted values.
RMSE is the square root of mean squared error.
Result 5
Source: sql_basics.txt
Category: sql
Distance: 128.29298400878906
Text:
SQL Basics
SELECT retrieves data.
WHERE filters rows.
GROUP BY groups rows for aggregate calculations.
ORDER BY sorts results.
Common aggregate functions: COUNT, SUM, AVG, MIN, MAX.
RETRIEVER TEST COMPLETE
(venv) PS C:\Users\koppa\Downloads\AI-Coding-Documentation-Assistant\Backend>
master*+ 0 1 Git Graph
Ln 248, Col 20 Spaces: 4 UTF-8 CRLF Python Stanza Python 3.14.2
34°C Mostly sunny
Search
ENG IN
03:18 PM 29-09-2026

15. Step 12 — Gemini 2.5 Flash

Since you specifically want:

`gemini-2.5-flash`

keep your LLM file focused on that model.

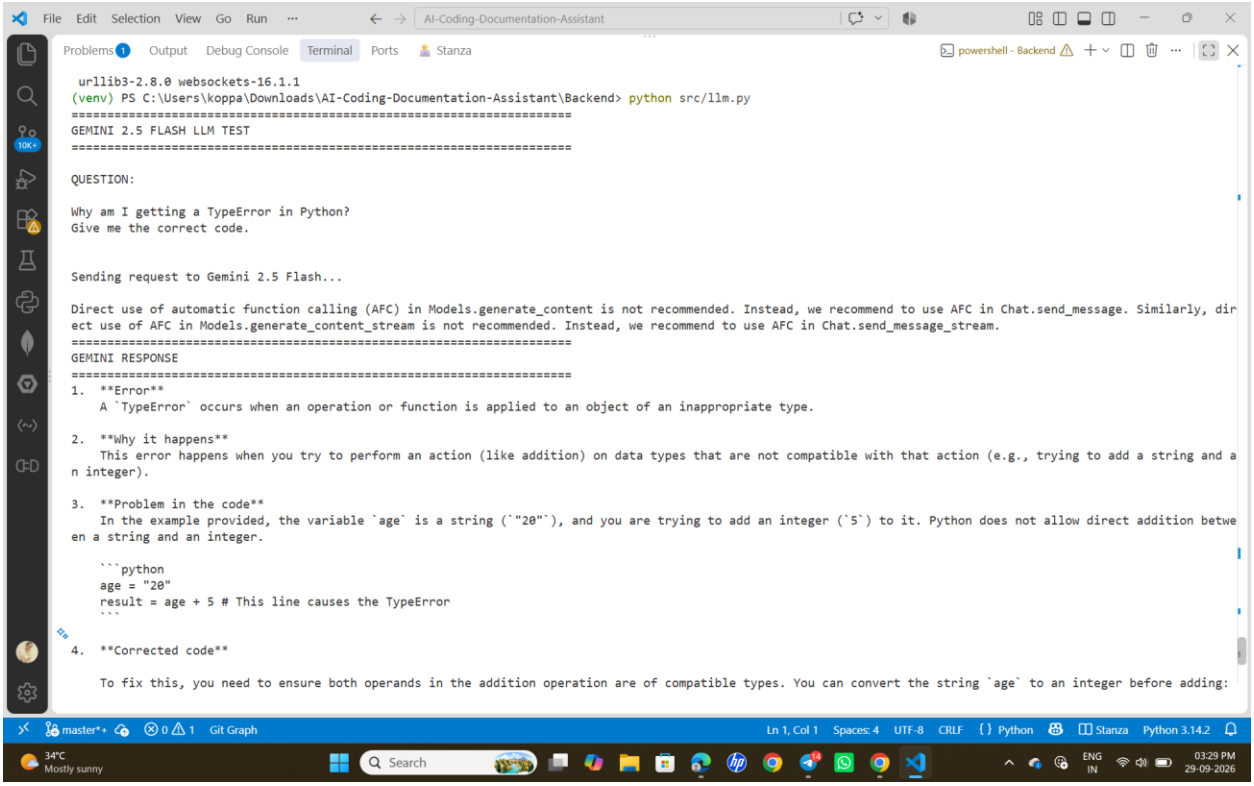
Create:

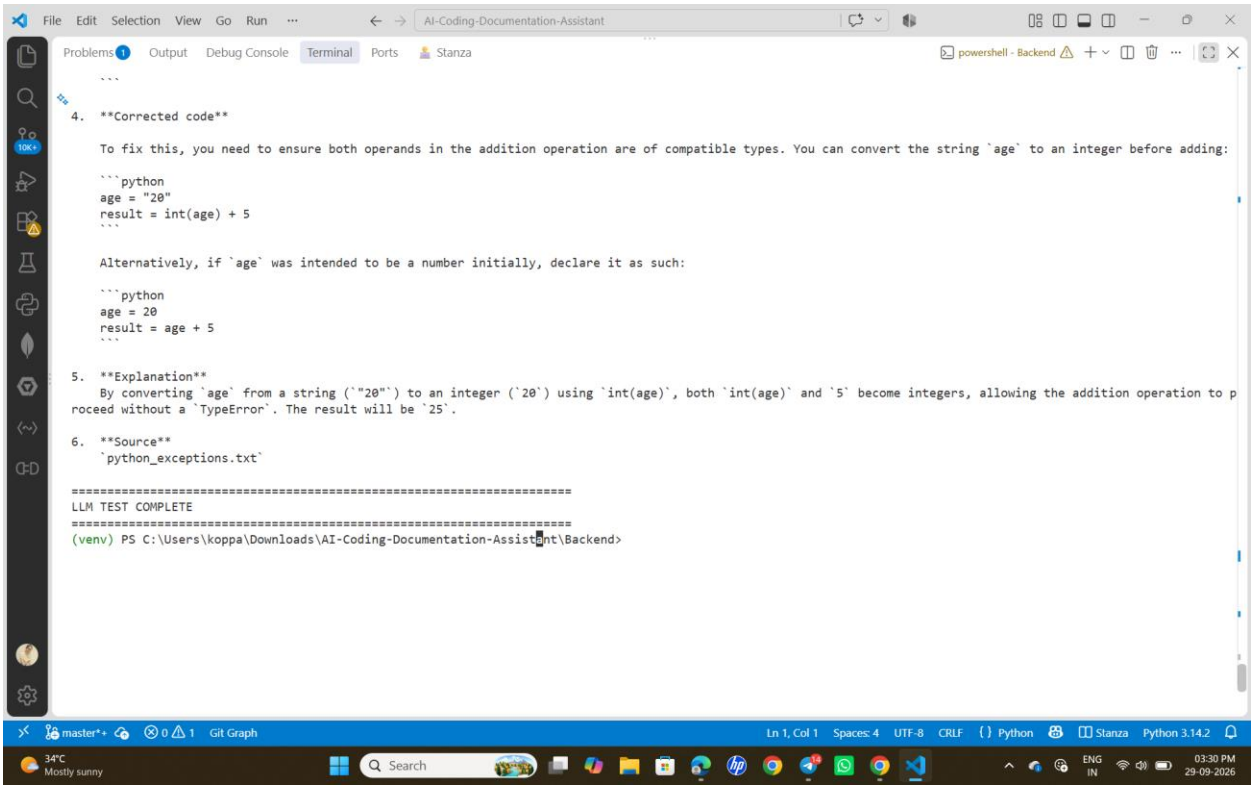
`src/llm.py`

Use:

```
import osfrom dotenv import load_dotenvfrom google import
genaiload_dotenv()GEMINI_API_KEY = os.getenv("GEMINI_API_KEY")if not
GEMINI_API_KEY:    raise ValueError(        "GEMINI_API_KEY not
found"    )client = genai.Client(    api_key=GEMINI_API_KEY)MODEL_NAME
= "gemini-2.5-flash"def generate_answer(    question,
retrieved_documents):    context =
"\n\n".join(        document["text"]        for document in
retrieved_documents    )    prompt = f"""You are an AI Coding
Documentation Assistant.Answer the question using ONLY theprovided
documentation.
```

Output:


File Edit Selection View Go Run
AI-Coding-Documentation-Assistant
Problems 1 Output Debug Console Terminal Ports Stanza
powershell - Backend
urllib3-2.8.0 websockets-16.1.1
(venv) PS C:\Users\koppa\Downloads\AI-Coding-Documentation-Assistant\Backend> python src/llm.py
GEMINI 2.5 FLASH LLM TEST
QUESTION:
Why am I getting a TypeError in Python?
Give me the correct code.
Sending request to Gemini 2.5 Flash...
Direct use of automatic function calling (AFC) in Models.generate_content is not recommended. Instead, we recommend to use AFC in Chat.send_message. Similarly, direct use of AFC in Models.generate_content_stream is not recommended. Instead, we recommend to use AFC in Chat.send_message_stream.
GEMINI RESPONSE
1. **Error**
A `TypeError` occurs when an operation or function is applied to an object of an inappropriate type.
2. **Why it happens**
This error happens when you try to perform an action (like addition) on data types that are not compatible with that action (e.g., trying to add a string and an integer).
3. **Problem in the code**
In the example provided, the variable `age` is a string (`"20"`), and you are trying to add an integer (`5`) to it. Python does not allow direct addition between a string and an integer.
```python
age = "20"
result = age + 5 # This line causes the TypeError
```
4. **Corrected code**
To fix this, you need to ensure both operands in the addition operation are of compatible types. You can convert the string `age` to an integer before adding:
master*+ 0 1 Git Graph
Ln 1, Col 1 Spaces: 4 UTF-8 CRLF Python Stanza Python 3.14.2
34°C Mostly sunny
Search
ENG IN
03:29 PM 29-09-2026


File Edit Selection View Go Run
AI-Coding-Documentation-Assistant
Problems 1 Output Debug Console Terminal Ports Stanza
powershell - Backend
```
4. **Corrected code**
To fix this, you need to ensure both operands in the addition operation are of compatible types. You can convert the string `age` to an integer before adding:
```python
age = "20"
result = int(age) + 5
```
Alternatively, if `age` was intended to be a number initially, declare it as such:
```python
age = 20
result = age + 5
```
5. **Explanation**
By converting `age` from a string (`"20"`) to an integer (`20`) using `int(age)`, both `int(age)` and `5` become integers, allowing the addition operation to proceed without a `TypeError`. The result will be `25`.
6. **Source**
`python_exceptions.txt`
LLM TEST COMPLETE
(venv) PS C:\Users\koppa\Downloads\AI-Coding-Documentation-Assistant\Backend>
master*+ 0 1 Git Graph
Ln 1, Col 1 Spaces: 4 UTF-8 CRLF Python Stanza Python 3.14.2
34°C Mostly sunny
Search
ENG IN
03:30 PM 29-09-2026
```

16. Important Gemini Issue You Had Earlier

Your screenshot showed:

```
429 RESOURCE_EXHAUSTED
```

and:

```
You exceeded your current quota
```

That is **not a Python import problem**.

Your code successfully reached Gemini.

The problem was API quota/rate limits.

So don't keep changing:

```
MODEL_NAME
```

randomly.

For this project, keep:

```
MODEL_NAME = "gemini-2.5-flash"
```

and make sure the API key has available quota.

Also don't put the API key directly into Python code.

Use:

```
GEMINI_API_KEY=...
```

17. Step 13 — Build the RAG Pipeline

This is the most important file.

Create:

src/rag_pipeline.py

The pipeline should be:

```
Question
   ↓
Create question embedding
   ↓
Search FAISS
   ↓
Retrieve top K documents
   ↓
Build context
   ↓
Gemini 2.5 Flash
   ↓
Answer
```

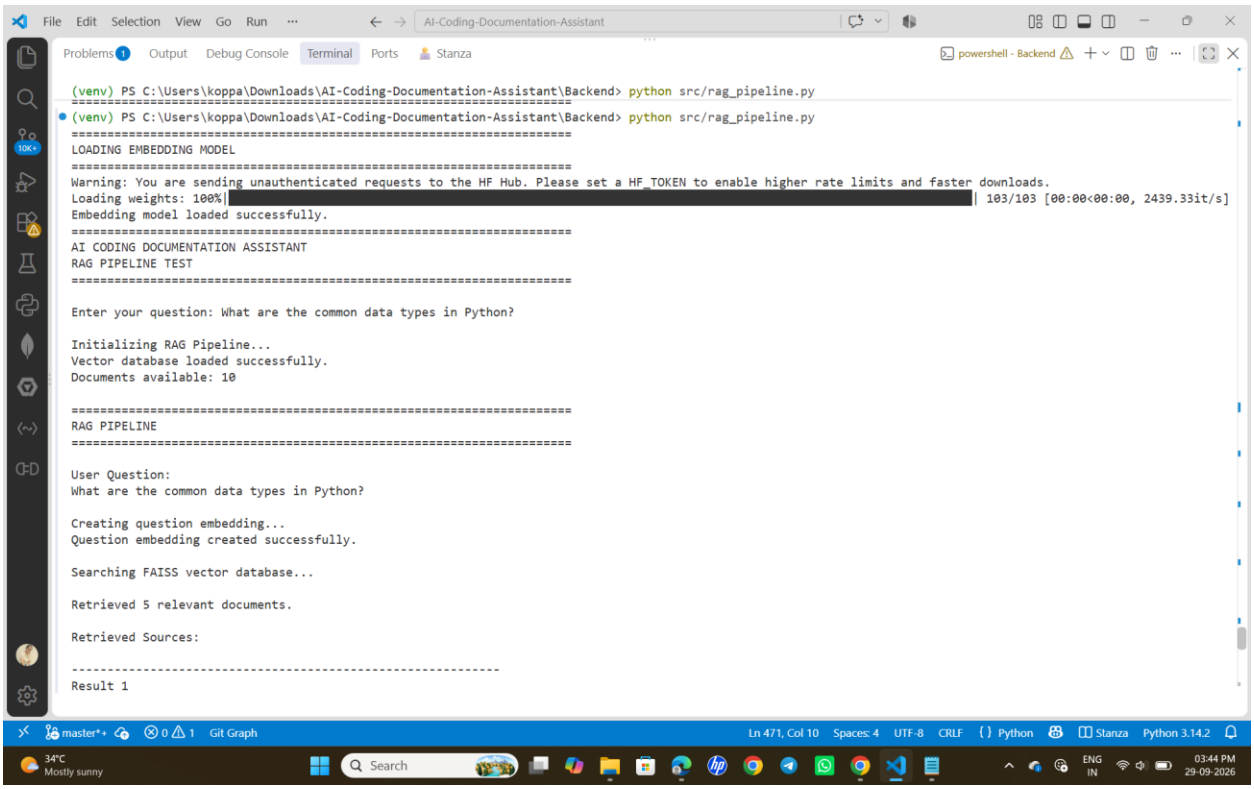
Conceptually:

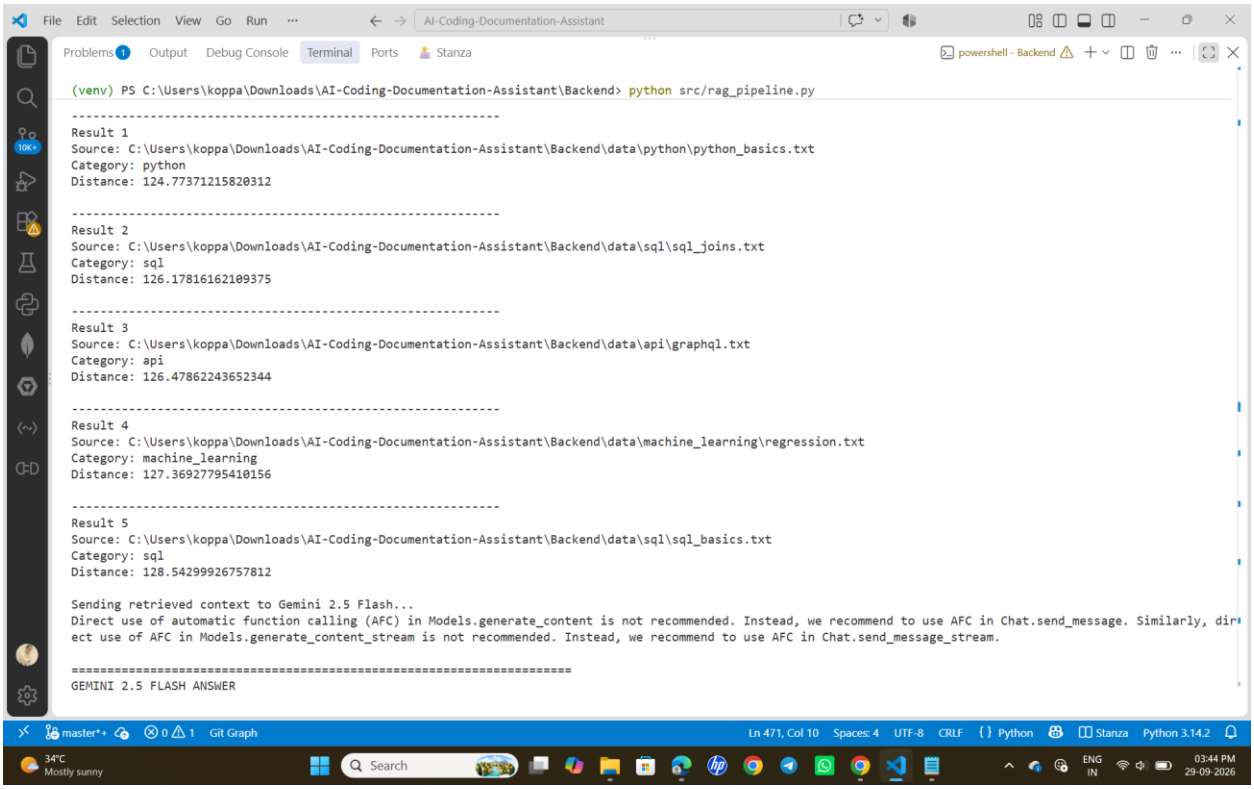
```
def run_rag_pipeline(question):    query_embedding =
create_embeddings(        [question]    )[0]    retrieved_documents =
vector_store.search(        query_embedding,        k=5    )    answer
= generate_answer(        question,        retrieved_documents    )
return {        "question": question,        "answer": answer,
"sources": retrieved_documents    }
```

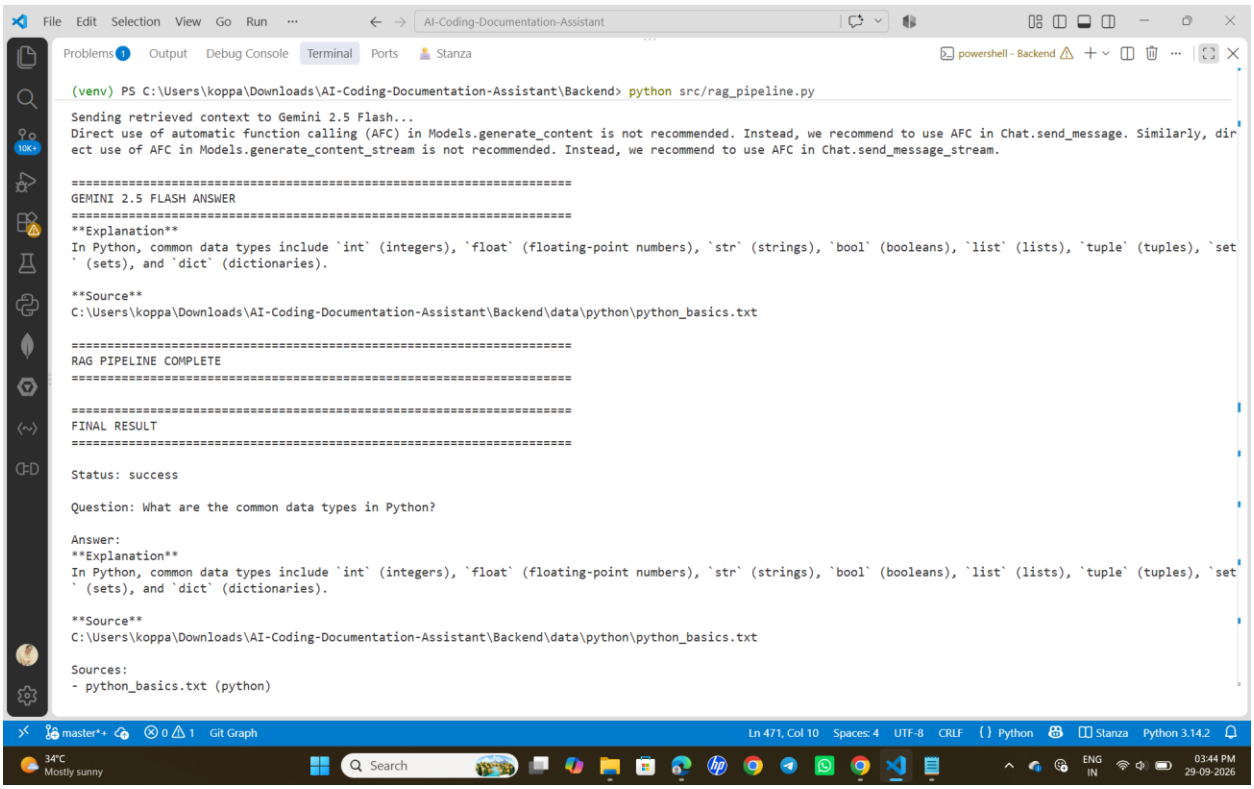
Output:

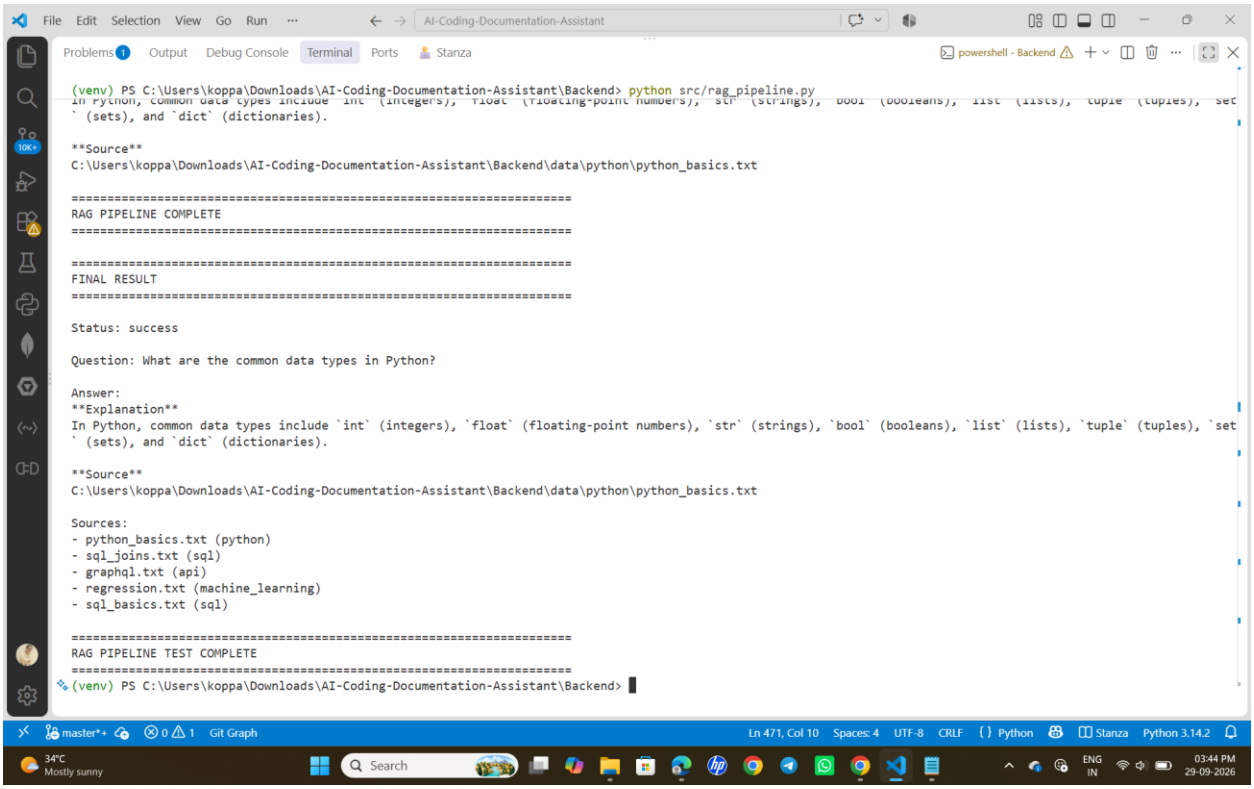
Yes. Based on the `document_loader.py → text_cleaner.py → chunker.py → embeddings.py → vector_store.py → retriever.py → llm.py` structure we have been building, your `rag_pipeline.py` should connect all the pieces.

What are the common data types in Python? -> i asked this question


File Edit Selection View Go Run
AI-Coding-Documentation-Assistant
Problems 1 Output Debug Console Terminal Ports Stanza
powershell - Backend
(venv) PS C:\Users\koppa\Downloads\AI-Coding-Documentation-Assistant\Backend> python src/rag_pipeline.py
(venv) PS C:\Users\koppa\Downloads\AI-Coding-Documentation-Assistant\Backend> python src/rag_pipeline.py
LOADING EMBEDDING MODEL
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
Loading weights: 100%| 103/103 [00:00<00:00, 2439.33it/s]
Embedding model loaded successfully.
AI CODING DOCUMENTATION ASSISTANT
RAG PIPELINE TEST
Enter your question: What are the common data types in Python?
Initializing RAG Pipeline...
Vector database loaded successfully.
Documents available: 10
RAG PIPELINE
User Question:
What are the common data types in Python?
Creating question embedding...
Question embedding created successfully.
Searching FAISS vector database...
Retrieved 5 relevant documents.
Retrieved Sources:
Result 1
master*+ 0 1 Git Graph
Ln 471, Col 10 Spaces: 4 UTF-8 CRLF Python Stanza Python 3.14.2
34°C Mostly sunny
Search
ENG IN
03:44 PM 29-09-2026


File Edit Selection View Go Run
AI-Coding-Documentation-Assistant
Problems 1 Output Debug Console Terminal Ports Stanza
powershell - Backend
(venv) PS C:\Users\koppa\Downloads\AI-Coding-Documentation-Assistant\Backend> python src/rag_pipeline.py
Result 1
Source: C:\Users\koppa\Downloads\AI-Coding-Documentation-Assistant\Backend\data\python\python_basics.txt
Category: python
Distance: 124.77371215820312
Result 2
Source: C:\Users\koppa\Downloads\AI-Coding-Documentation-Assistant\Backend\data\sql\sql_joins.txt
Category: sql
Distance: 126.17816162109375
Result 3
Source: C:\Users\koppa\Downloads\AI-Coding-Documentation-Assistant\Backend\data\api\graphql.txt
Category: api
Distance: 126.47862243652344
Result 4
Source: C:\Users\koppa\Downloads\AI-Coding-Documentation-Assistant\Backend\data\machine_learning\regression.txt
Category: machine_learning
Distance: 127.36927795410156
Result 5
Source: C:\Users\koppa\Downloads\AI-Coding-Documentation-Assistant\Backend\data\sql\sql_basics.txt
Category: sql
Distance: 128.54299926757812
Sending retrieved context to Gemini 2.5 Flash...
Direct use of automatic function calling (AFC) in Models.generate_content is not recommended. Instead, we recommend to use AFC in Chat.send_message. Similarly, dir
ect use of AFC in Models.generate_content_stream is not recommended. Instead, we recommend to use AFC in Chat.send_message_stream.
GEMINI 2.5 FLASH ANSWER
master*+ 0 1 Git Graph
Ln 471, Col 10 Spaces: 4 UTF-8 CRLF Python Stanza Python 3.14.2
34°C Mostly sunny
Search
ENG IN
03:44 PM 29-09-2026


File Edit Selection View Go Run
AI-Coding-Documentation-Assistant
Problems 1 Output Debug Console Terminal Ports Stanza
powershell - Backend
(venv) PS C:\Users\koppa\Downloads\AI-Coding-Documentation-Assistant\Backend> python src/rag_pipeline.py
Sending retrieved context to Gemini 2.5 Flash...
Direct use of automatic function calling (AFC) in Models.generate_content is not recommended. Instead, we recommend to use AFC in Chat.send_message. Similarly, dir
ect use of AFC in Models.generate_content_stream is not recommended. Instead, we recommend to use AFC in Chat.send_message_stream.
GEMINI 2.5 FLASH ANSWER
Explanation
In Python, common data types include `int` (integers), `float` (floating-point numbers), `str` (strings), `bool` (booleans), `list` (lists), `tuple` (tuples), `set
` (sets), and `dict` (dictionaries).
Source
C:\Users\koppa\Downloads\AI-Coding-Documentation-Assistant\Backend\data\python\python_basics.txt
RAG PIPELINE COMPLETE
FINAL RESULT
Status: success
Question: What are the common data types in Python?
Answer:
Explanation
In Python, common data types include `int` (integers), `float` (floating-point numbers), `str` (strings), `bool` (booleans), `list` (lists), `tuple` (tuples), `set
` (sets), and `dict` (dictionaries).
Source
C:\Users\koppa\Downloads\AI-Coding-Documentation-Assistant\Backend\data\python\python_basics.txt
Sources:
- python_basics.txt (python)
master*+ 0 1 Git Graph
Ln 471, Col 10 Spaces: 4 UTF-8 CRLF Python Stanza Python 3.14.2
34°C Mostly sunny
Search
ENG IN
03:44 PM 29-09-2026


File Edit Selection View Go Run
AI-Coding-Documentation-Assistant
Problems 1 Output Debug Console Terminal Ports Stanza
powershell - Backend
(venv) PS C:\Users\koppa\Downloads\AI-Coding-Documentation-Assistant\Backend> python src/rag_pipeline.py
` (sets), and `dict` (dictionaries).
Source
C:\Users\koppa\Downloads\AI-Coding-Documentation-Assistant\Backend\data\python\python_basics.txt
RAG PIPELINE COMPLETE
FINAL RESULT
Status: success
Question: What are the common data types in Python?
Answer:
Explanation
In Python, common data types include `int` (integers), `float` (floating-point numbers), `str` (strings), `bool` (booleans), `list` (lists), `tuple` (tuples), `set
` (sets), and `dict` (dictionaries).
Source
C:\Users\koppa\Downloads\AI-Coding-Documentation-Assistant\Backend\data\python\python_basics.txt
Sources:
- python_basics.txt (python)
- sql_joins.txt (sql)
- graphql.txt (api)
- regression.txt (machine_learning)
- sql_basics.txt (sql)
RAG PIPELINE TEST COMPLETE
(venv) PS C:\Users\koppa\Downloads\AI-Coding-Documentation-Assistant\Backend>
master*+ 0 1 Git Graph
Ln 471, Col 10 Spaces: 4 UTF-8 CRLF Python Stanza Python 3.14.2
34°C Mostly sunny
Search
ENG IN
03:44 PM 29-09-2026

18. Step 14 — Create FastAPI Backend

Your:

`app.py`

will expose APIs.

For example:

`POST /api/ask`

Request:

```
{
    "question": "How does INNER JOIN work?"
}
```

Response:

```
{
    "status": "success",
    "question": "How does INNER JOIN work?",
    "answer": "An INNER JOIN returns rows that have matching values in
both tables...",
    "sources": [
        {
            "source": "joins.pdf",
            "page": 3
        }
    ]
}
```

19. Step 15 — Add Health Endpoint

Create:

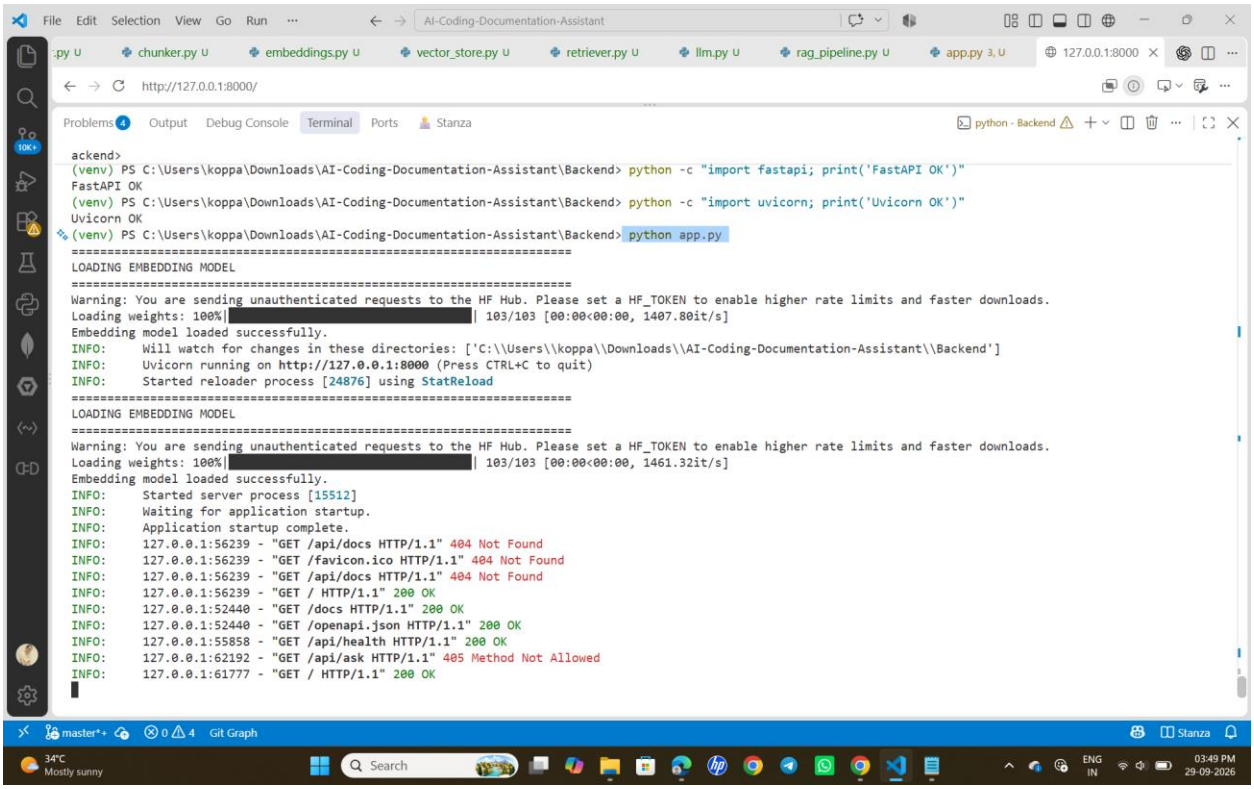
`GET /api/health`

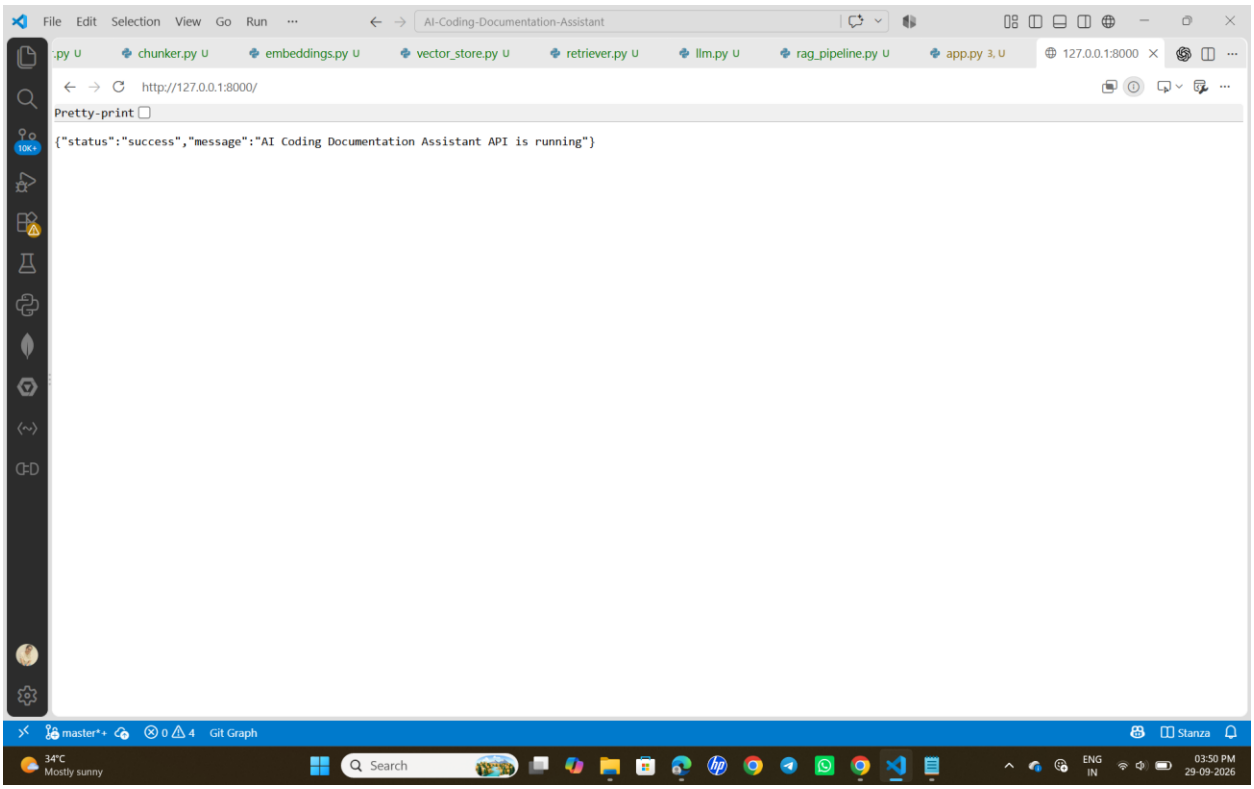
Response:

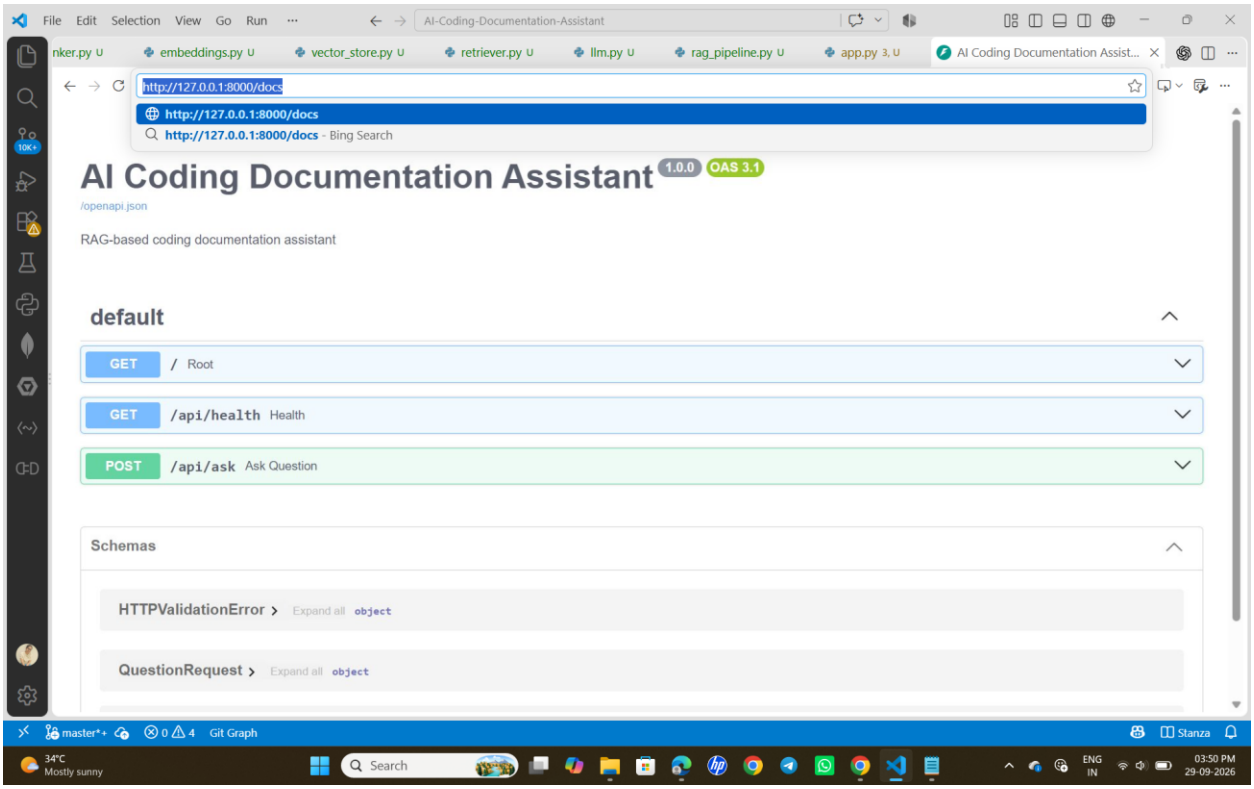
```
{
    "status": "success",
    "message": "AI Coding Documentation Assistant is running"
}
```

You can test:

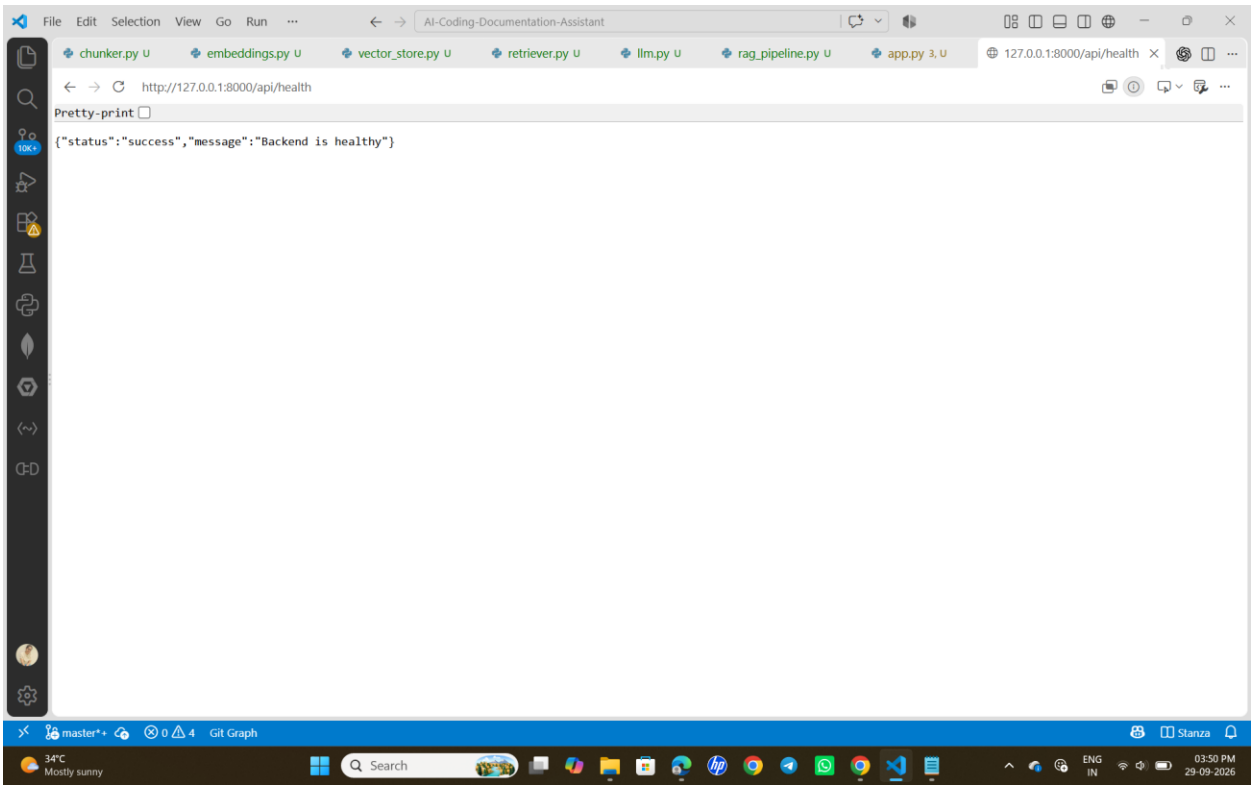
http://127.0.0.1:8000/api/health







http://127.0.0.1:8000/api/health



http://127.0.0.1:8000/api/ask
http://127.0.0.1:8000/docs

Open:

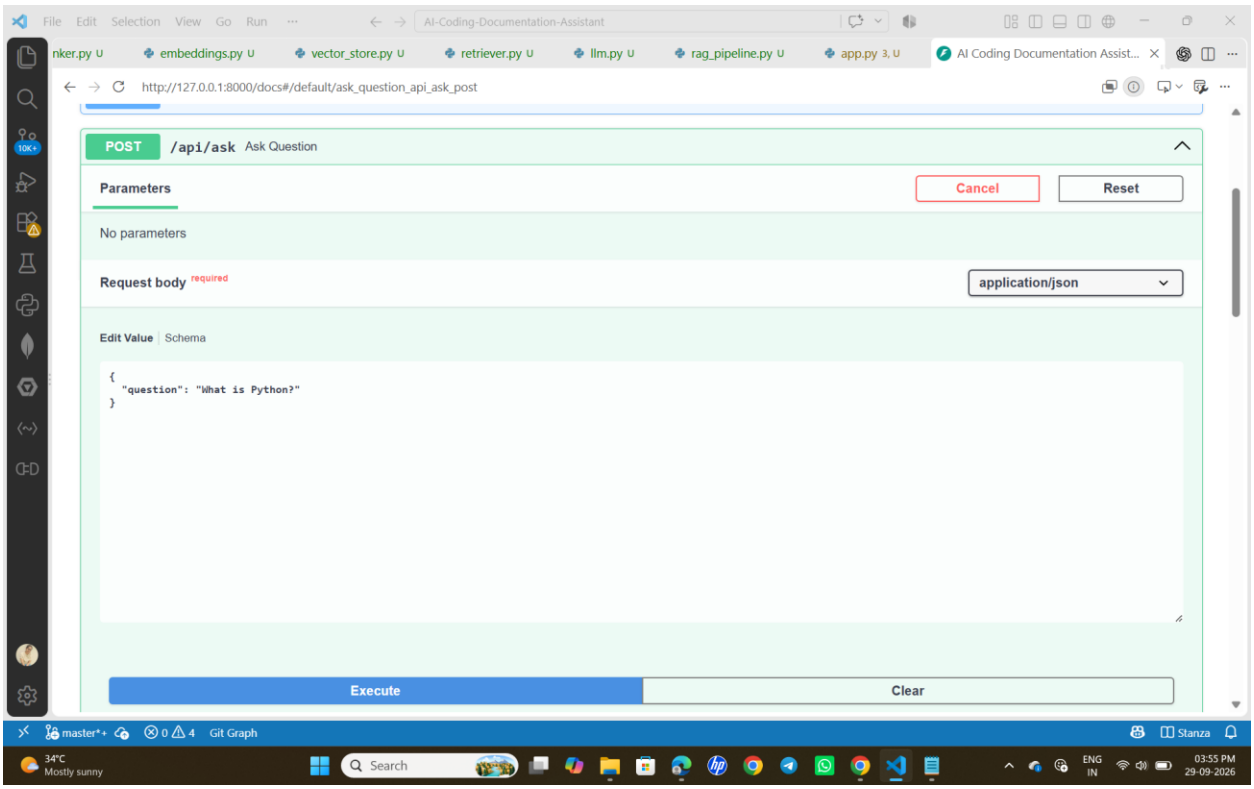
`http://127.0.0.1:8000/docs`

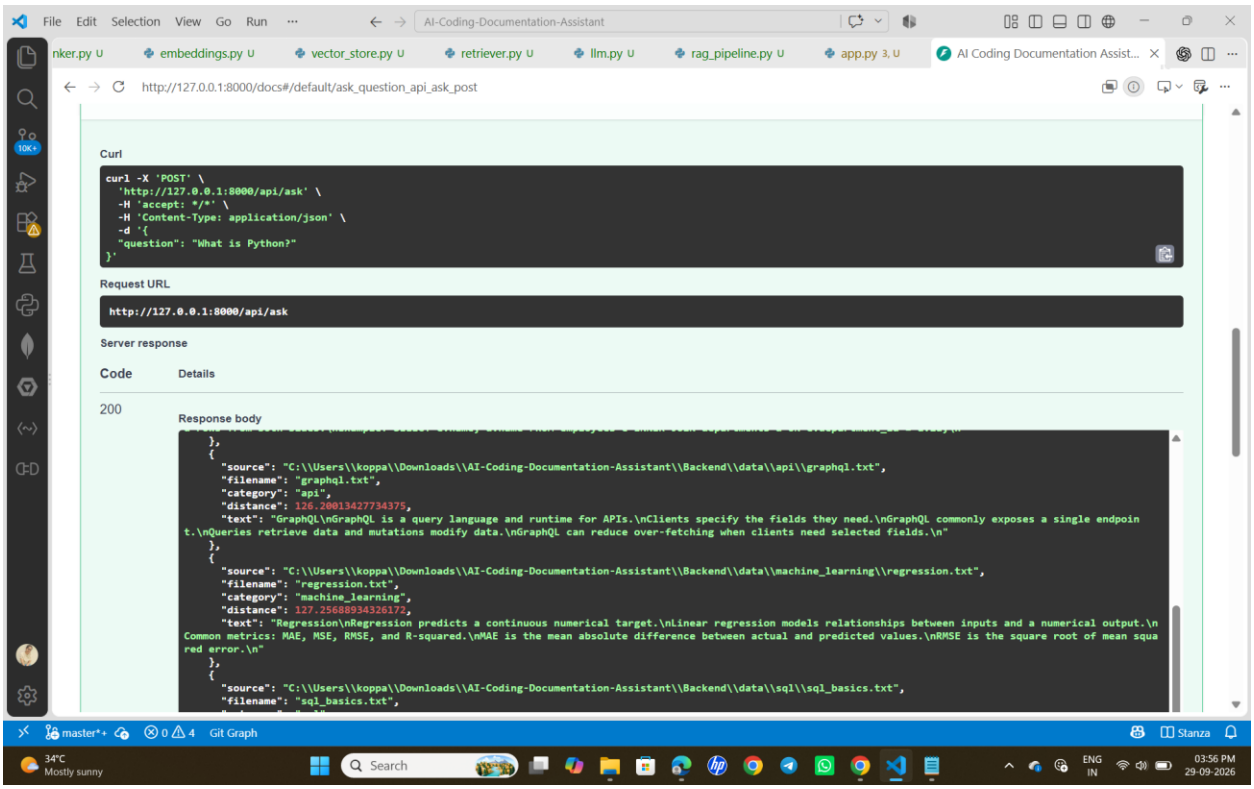
Then:

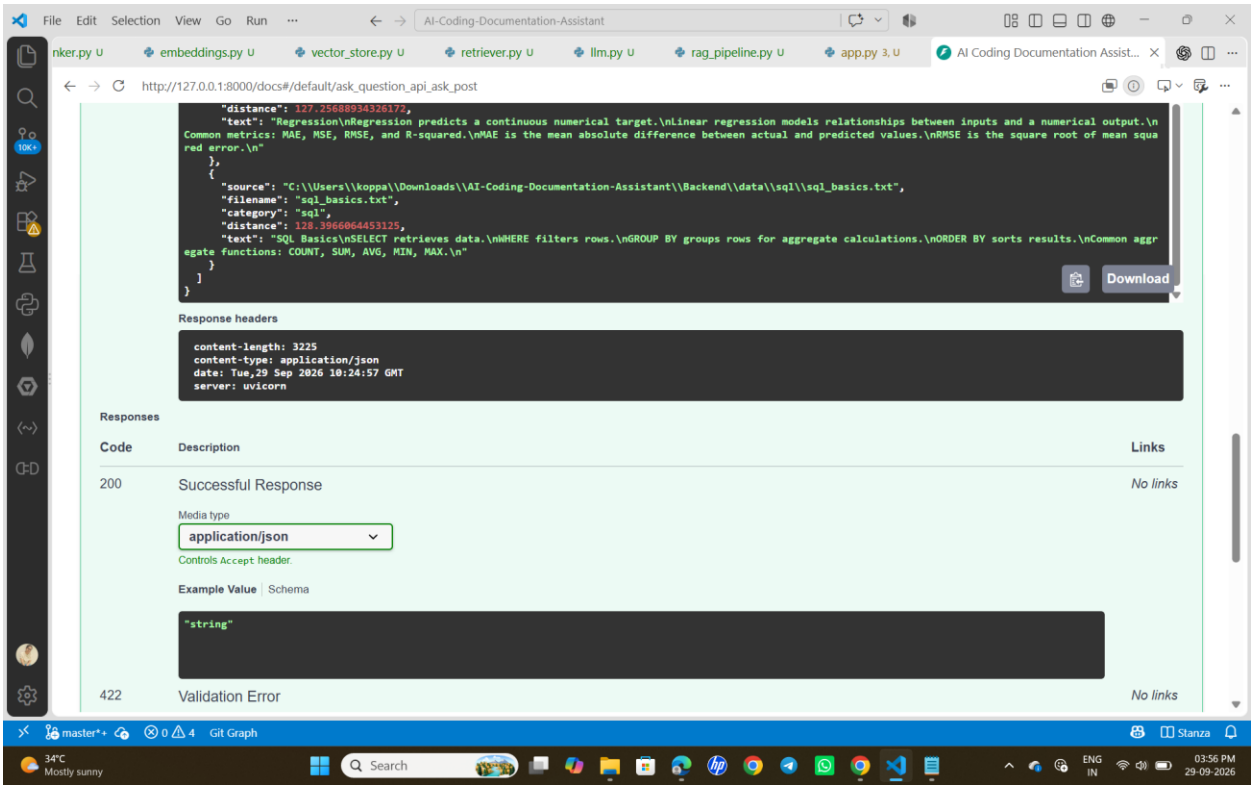
1. Expand **POST** `/api/ask`
2. Click **Try it out**
3. Enter:

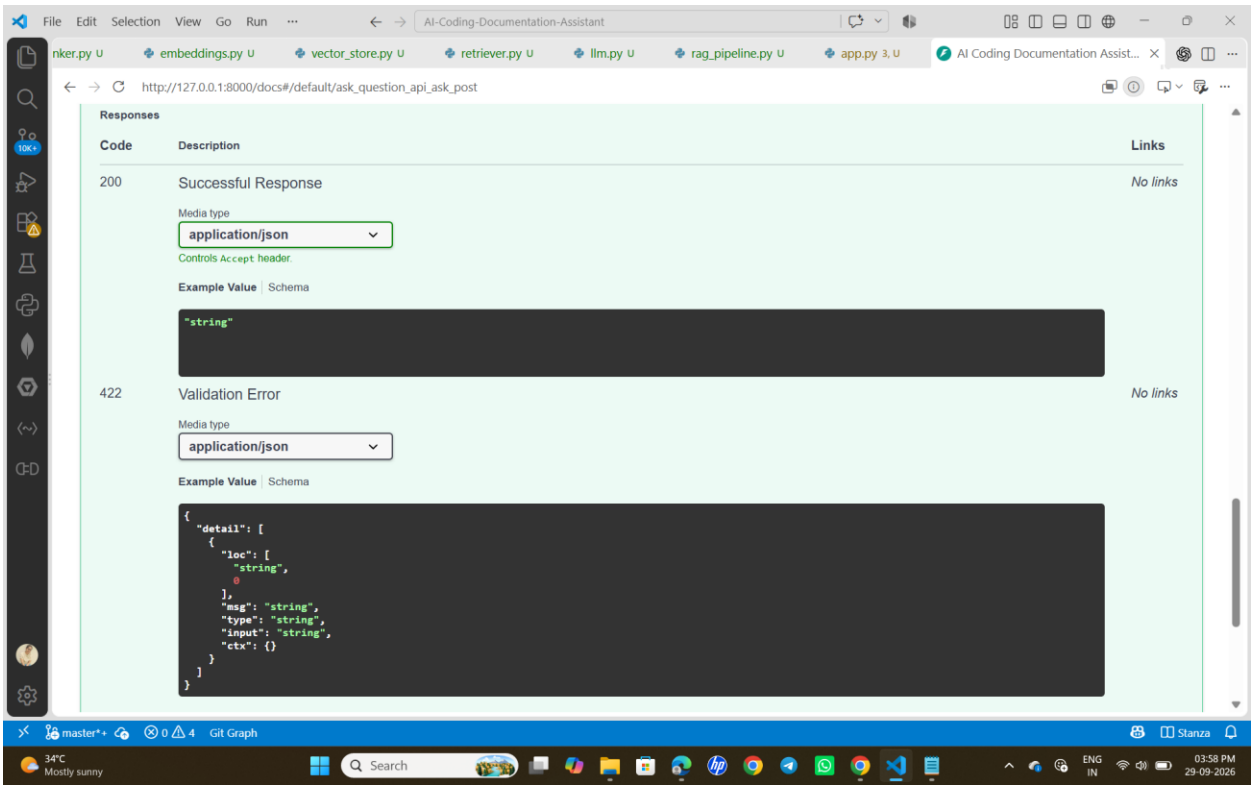
JSON

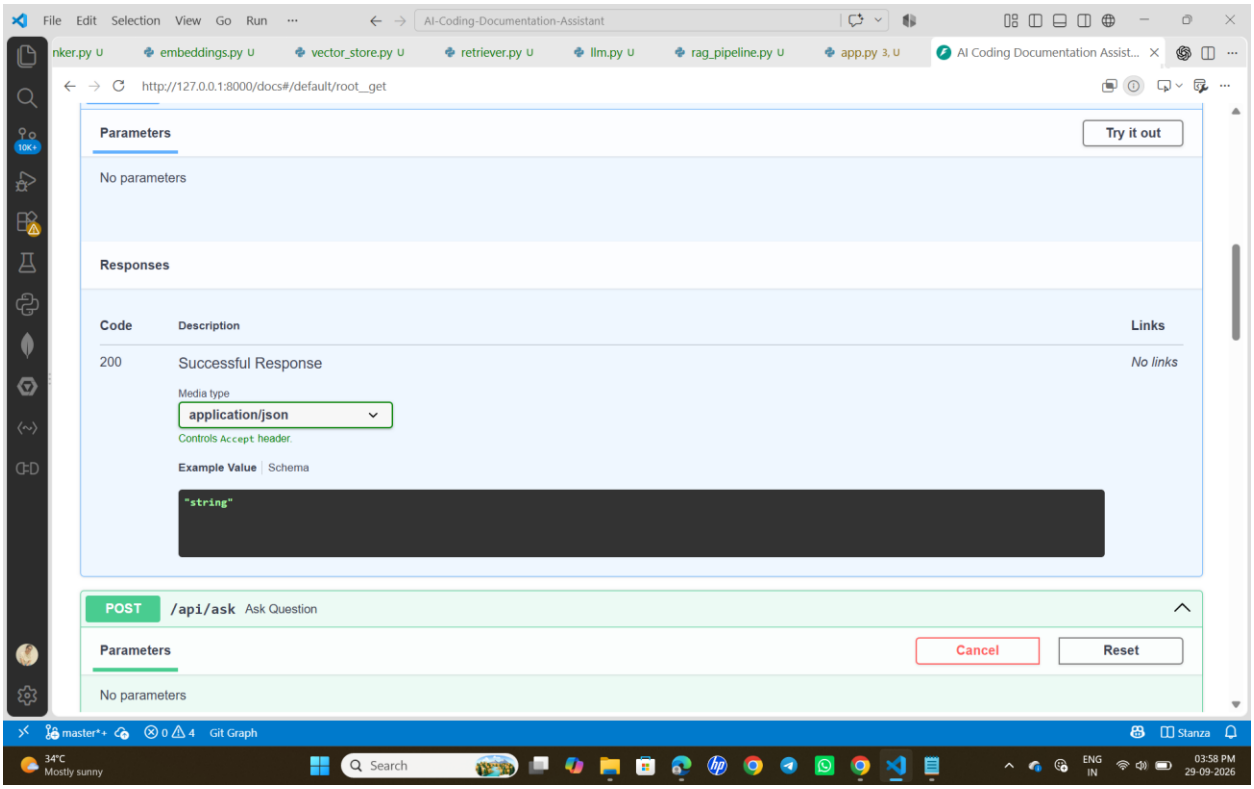
```
{
  "question": "What is Python?"
}
```


File Edit Selection View Go Run
AI-Coding-Documentation-Assistant
nker.py U
embeddings.py U
vector_store.py U
retriever.py U
llm.py U
rag_pipeline.py U
app.py 3, U
AI Coding Documentation Assist...
http://127.0.0.1:8000/docs#/default/ask_question_api_ask_post
POST
/api/ask
Ask Question
Parameters
Cancel
Reset
No parameters
Request body
required
application/json
Edit Value
Schema
{
"question": "What is Python?"
}
Execute
Clear
master*+
0 4
Git Graph
Stanza
34°C
Mostly sunny
Search
ENG
IN
03:55 PM
29-09-2026


File Edit Selection View Go Run
AI-Coding-Documentation-Assistant
nker.py U
embeddings.py U
vector_store.py U
retriever.py U
llm.py U
rag_pipeline.py U
app.py 3, U
AI Coding Documentation Assist...
http://127.0.0.1:8000/docs#/default/ask_question_api_ask_post
Curl
curl -X 'POST' \
'http://127.0.0.1:8000/api/ask' \
-H 'accept: */*' \
-H 'Content-Type: application/json' \
-d '{
"question": "What is Python?"
}'
Request URL
http://127.0.0.1:8000/api/ask
Server response
Code
Details
200
Response body
"source": "C:\\Users\\koppa\\Downloads\\AI-Coding-Documentation-Assistant\\Backend\\data\\api\\graphql.txt",
"filename": "graphql.txt",
"category": "api",
"distance": 126.20013427734375,
"text": "GraphQL\nGraphQL is a query language and runtime for APIs.\nClients specify the fields they need.\nGraphQL commonly exposes a single endpoin
t.\nQueries retrieve data and mutations modify data.\nGraphQL can reduce over-fetching when clients need selected fields.\n"
"source": "C:\\Users\\koppa\\Downloads\\AI-Coding-Documentation-Assistant\\Backend\\data\\machine_learning\\regression.txt",
"filename": "regression.txt",
"category": "machine_learning",
"distance": 127.25688934326172,
"text": "Regression\nRegression predicts a continuous numerical target.\nLinear regression models relationships between inputs and a numerical output.\n
Common metrics: MAE, MSE, RMSE, and R-squared.\nMAE is the mean absolute difference between actual and predicted values.\nRMSE is the square root of mean squa
red error.\n"
"source": "C:\\Users\\koppa\\Downloads\\AI-Coding-Documentation-Assistant\\Backend\\data\\sql\\sql_basics.txt",
"filename": "sql_basics.txt",
master*+
Git Graph
Stanza
34°C
Mostly sunny
Search
ENG
IN
03:56 PM
29-09-2026


File Edit Selection View Go Run
AI-Coding-Documentation-Assistant
nker.py U
embeddings.py U
vector_store.py U
retriever.py U
llm.py U
rag_pipeline.py U
app.py 3, U
AI Coding Documentation Assist...
http://127.0.0.1:8000/docs#/default/ask_question_api_ask_post
"distance": 127.25688934326172,
"text": "Regression\nRegression predicts a continuous numerical target.\nLinear regression models relationships between inputs and a numerical output.\n
Common metrics: MAE, MSE, RMSE, and R-squared.\nMAE is the mean absolute difference between actual and predicted values.\nRMSE is the square root of mean squa
red error.\n"
"source": "C:\\Users\\koppa\\Downloads\\AI-Coding-Documentation-Assistant\\Backend\\data\\sql\\sql_basics.txt",
"filename": "sql_basics.txt",
"category": "sql",
"distance": 128.39660644531125,
"text": "SQL Basics\nSELECT retrieves data.\nWHERE filters rows.\nGROUP BY groups rows for aggregate calculations.\nORDER BY sorts results.\nCommon aggr
egate functions: COUNT, SUM, AVG, MIN, MAX.\n"
Download
Response headers
content-length: 3225
content-type: application/json
date: Tue,29 Sep 2026 10:24:57 GMT
server: uvicorn
Responses
Code
Description
Links
200
Successful Response
No links
Media type
application/json
Controls Accept header.
Example Value
Schema
"string"
422
Validation Error
No links
master*+
Git Graph
Stanza
34°C
Mostly sunny
Search
ENG
IN
03:56 PM
29-09-2026


File Edit Selection View Go Run
AI-Coding-Documentation-Assistant
nker.py U
embeddings.py U
vector_store.py U
retriever.py U
llm.py U
rag_pipeline.py U
app.py 3, U
AI Coding Documentation Assist...
http://127.0.0.1:8000/docs#/default/ask_question_api_ask_post
Responses
Code
Description
Links
200
Successful Response
No links
Media type
application/json
Controls Accept header.
Example Value | Schema
"string"
422
Validation Error
No links
Media type
application/json
Example Value | Schema
{
"detail": [
{
"loc": [
"string",
0
],
"msg": "string",
"type": "string",
"input": "string",
"ctx": {}
}
]
}
master*+
0 4
Git Graph
Stanza
34°C
Mostly sunny
Search
ENG
IN
03:58 PM
29-09-2026


File Edit Selection View Go Run
AI-Coding-Documentation-Assistant
nker.py U
embeddings.py U
vector_store.py U
retriever.py U
llm.py U
rag_pipeline.py U
app.py 3, U
AI Coding Documentation Assist...
http://127.0.0.1:8000/docs#/default/root__get
Parameters
Try it out
No parameters
Responses
Code
Description
Links
200
Successful Response
No links
Media type
application/json
Controls Accept header.
Example Value | Schema
"string"
POST
/api/ask
Ask Question
Parameters
Cancel
Reset
No parameters
master*+
0 4
Git Graph
Stanza
34°C
Mostly sunny
Search
ENG
IN
03:58 PM
29-09-2026

20. Step 16 — Add Document Upload

Create:

```
POST /api/upload
```

The user uploads:

```
Python.pdf
```

Backend:

```
PDF
 ↓
Extract text
 ↓
Clean
 ↓
Chunk
 ↓
Embedding
 ↓
FAISS
```

Then return:

```
{
    "status": "success",
    "message": "Document added successfully",
    "filename": "Python.pdf",
    "chunks": 42
}
```

21. Step 17 — Frontend

Create React application.

From the project root:

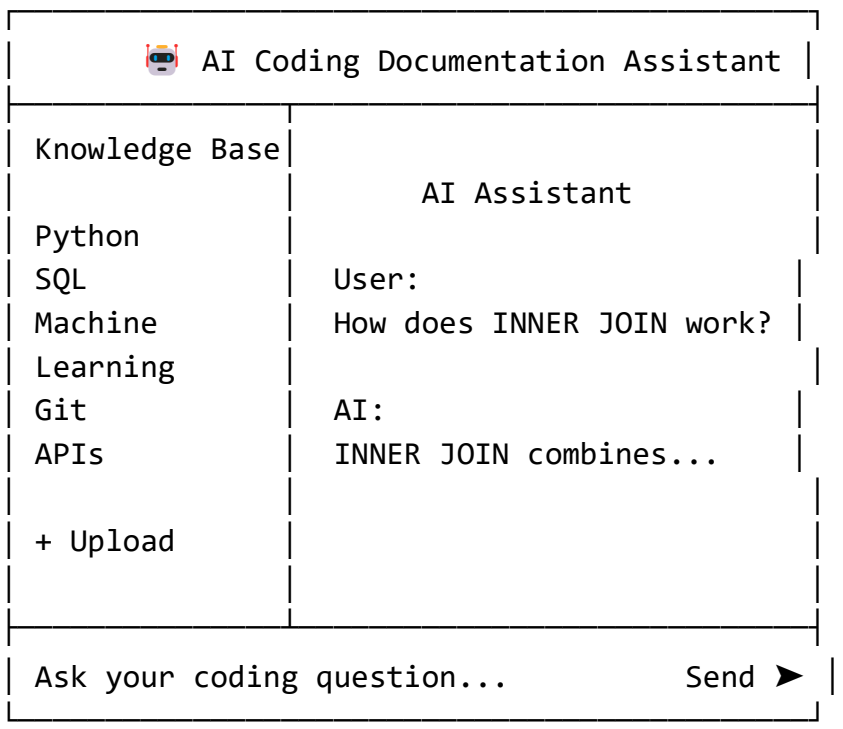
```
npm create vite@latest Frontend -- --template react
```

Then:

```
cd Frontend
npm install
npm install axios
npm run dev
```

22. Frontend UI

Your interface can look like:



23. Chat Example

User:

```
Explain Python decorators.
```

Backend:

```
Question
 ↓
Embedding
 ↓
Vector search
 ↓
Python documentation
 ↓
Gemini
```

Frontend displays:

```
🤖 AI Assistant

A Python decorator is a function that modifies
the behavior of another function without changing
its source code.

Example:

def decorator(func):
    def wrapper():
        print("Before")
        func()
    return wrapper
```

Then:

```
📚 Sources

python_functions.pdf
Page 12
```

24. Step 18 — Source Citations

This will make your project much stronger.

Instead of only returning:

```
Python decorators are...
```

return:

```
Python decorators are...

Sources:
```


python_functions.pdf


python_advanced.pdf

Your backend response can be:

```
{
    "answer": "...",
    "sources": [
        {
            "document": "python_functions.pdf",
            "page": 12
        }
    ]
}
```

25. Step 19 — Handle Unknown Questions

This is extremely important for RAG.

User:

```
Who invented the telephone?
```

Your documentation doesn't contain this.

The assistant should say:

```
I couldn't find this information in the
uploaded coding documentation.
```

It should **not invent an answer**.

Your prompt should explicitly enforce this.

26. Step 20 — Add Categories

Your UI can allow:

```
All Documentation
Python
SQL
Machine Learning
Git
APIs
```

For example:

```
[ Python ]
```

```
Question:
How do Python generators work?
```

Then retrieve primarily from:

```
Python documentation
```

27. Step 21 — Add Code Formatting

For questions involving code:

```
User:
How does a Python list comprehension work?
```

Response:

```
A list comprehension provides a concise way
to create a list.

Example:
```

```
numbers = [1, 2, 3, 4]

squares = [
    x * x
    for x in numbers
]
```

Your frontend should display code inside a proper code block.

28. Step 22 — Add Conversation History

You can maintain:

```
User:
What is INNER JOIN?

AI:
...

User:
Give me an example.

AI:
...
```

The second question can use the previous conversation context.

Architecture:

```
Question
+
Conversation History
+
Retrieved Documentation
        ↓
      Gemini
```

29. Step 23 — Advanced Feature: GitHub Repository

This is the advanced version mentioned in your project idea.

User enters:

https://github.com/user/project

Your system does:

```
GitHub URL
     ↓
Clone Repository
     ↓
Find source files
     ↓
Read .py / .js / .jsx / .java / etc.
     ↓
Split code into chunks
     ↓
Create embeddings
     ↓
Store in vector database
```

Then user can ask:

```
How does authentication work in this project?
```

or:

```
Where is the login API implemented?
```

or:

```
Explain app.py.
```

or:

```
Which file contains the database connection?
```

30. GitHub Repository Structure

For example:

```
github_repository/
│
├── app.py
├── requirements.txt
├── src/
│   ├── auth.py
│   ├── database.py
│   └── routes.py
│
└── README.md
```

Your system converts it into:

```
app.py → chunks
auth.py → chunks
database.py → chunks
routes.py → chunks
README.md → chunks
```

Then embeds them.

31. GitHub Loader

Create:

```
src/github_loader.py
```

Basic flow:

```
from git import Repoimport osdef clone_repository(    repo_url,
destination):    Repo.clone_from(        repo_url,
destination    )    return destination
```

Then scan:

```
.py
.js
.jsx
.ts
.tsx
.java
.cpp
.c
.md
```

Avoid indexing:

```
node_modules/
.git/
venv/
__pycache__/
dist/
build/
```

32. GitHub Question Example

User:

```
How does authentication work?
```

Retriever finds:

```
auth.py
routes.py
database.py
```

Gemini receives:

```
Question:
How does authentication work?

Context:

auth.py
```

```
routes.py
database.py
```

Then answers:

```
Authentication is implemented in auth.py.

The login endpoint is defined in routes.py.

The database user lookup is handled by database.py.
```

And shows:

```
Sources:
```
- auth.py
- routes.py
- database.py

33. Step 24 — Add RAG Evaluation

For your project, don't stop at:

```
"It works."
```

Test it.

Create questions such as:

Python

```
What is a Python decorator?
What is a generator?
What is exception handling?
```

SQL

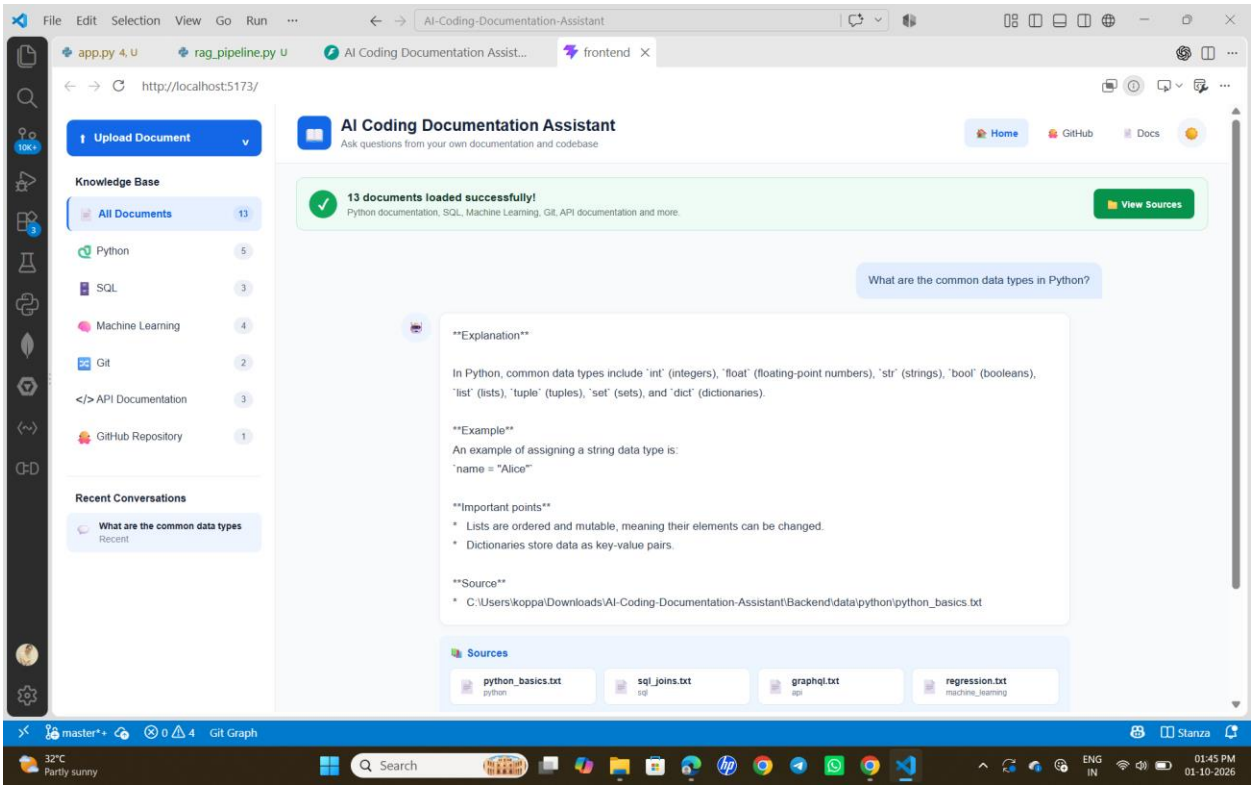
```
What is INNER JOIN?
What is GROUP BY?
What is a subquery?
```

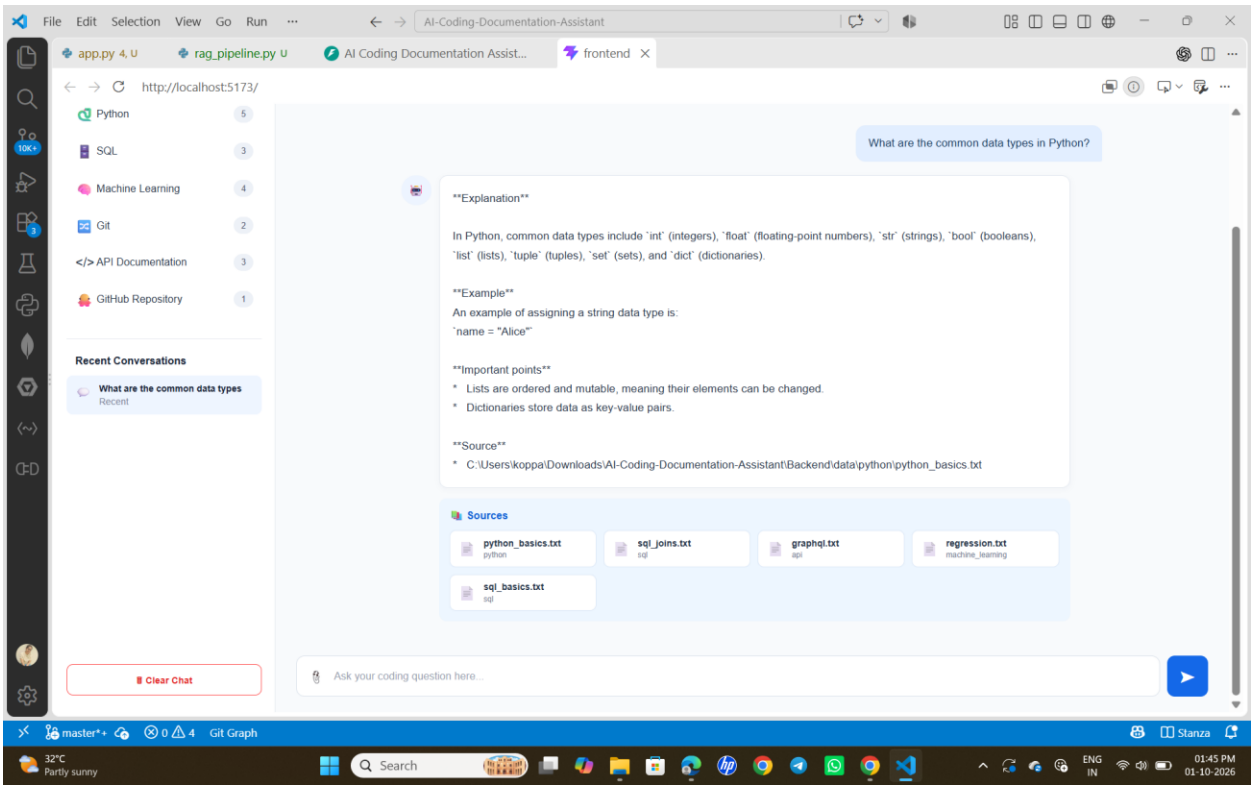
Machine Learning

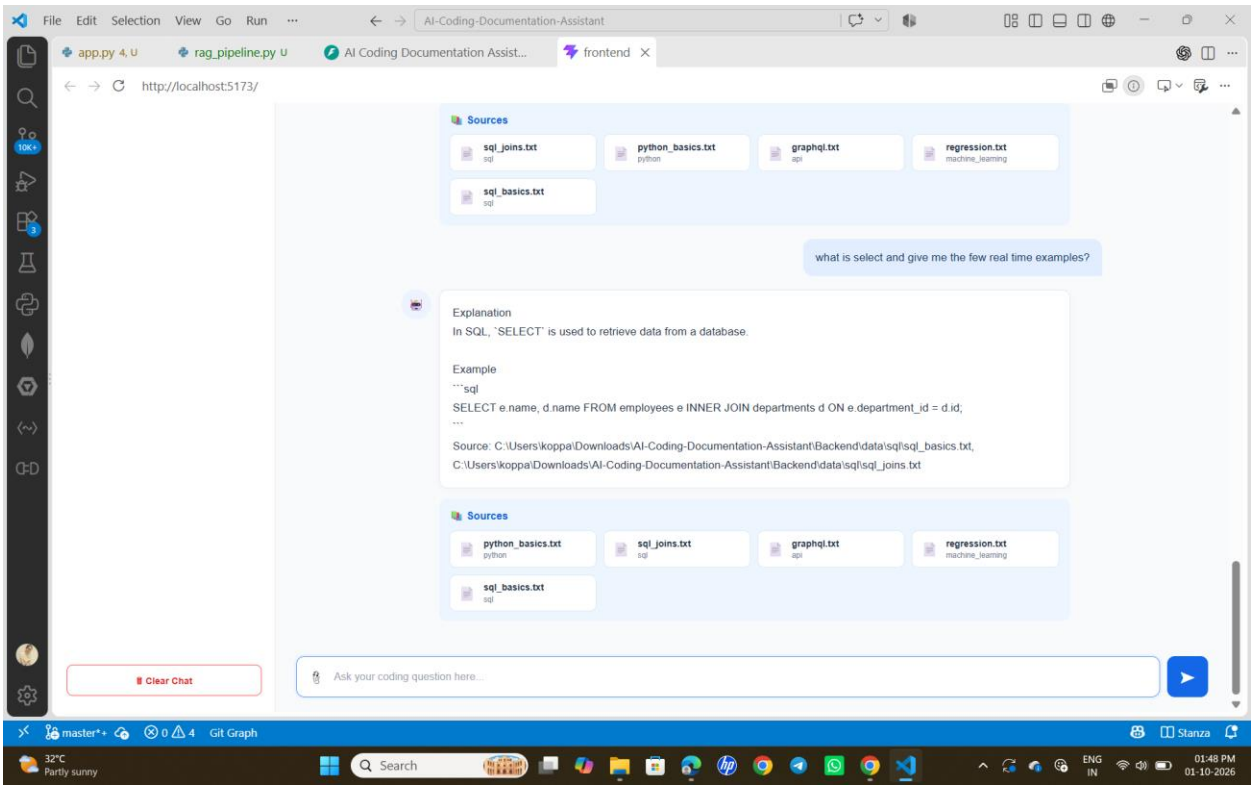
```
What is overfitting?
What is cross-validation?
What is logistic regression?
```

Git

```
What is git merge?
What is git rebase?
How do I create a branch?
```




File Edit Selection View Go Run
AI-Coding-Documentation-Assistant
app.py 4, U
rag_pipeline.py U
AI Coding Documentation Assist...
frontend
http://localhost:5173/
Python 5
SQL 3
Machine Learning 4
Git 2
API Documentation 3
GitHub Repository 1
Recent Conversations
What are the common data types
Recent
Clear Chat
What are the common data types in Python?
Explanation
In Python, common data types include `int` (integers), `float` (floating-point numbers), `str` (strings), `bool` (booleans), `list` (lists), `tuple` (tuples), `set` (sets), and `dict` (dictionaries).
Example
An example of assigning a string data type is:
`name = "Alice"`
Important points
* Lists are ordered and mutable, meaning their elements can be changed.
* Dictionaries store data as key-value pairs.
Source
* C:\Users\koppa\Downloads\AI-Coding-Documentation-Assistant\Backend\data\python\python_basics.txt
Sources
python_basics.txt
python
sql_joins.txt
sql
graphql.txt
api
regression.txt
machine_learning
sql_basics.txt
sql
Ask your coding question here...
master*+
0 4
Git Graph
Stanza
32°C
Partly sunny
Search
ENG
IN
01:45 PM
01-10-2026


File Edit Selection View Go Run
AI-Coding-Documentation-Assistant
app.py 4, U
rag_pipeline.py U
AI Coding Documentation Assist...
frontend
http://localhost:5173/
Sources
sql_joins.txt
sql
python_basics.txt
python
graphql.txt
api
regression.txt
machine_learning
sql_basics.txt
sql
what is select and give me the few real time examples?
Explanation
In SQL, `SELECT` is used to retrieve data from a database.
Example
```sql
SELECT e.name, d.name FROM employees e INNER JOIN departments d ON e.department_id = d.id;
```
Source: C:\Users\koppa\Downloads\AI-Coding-Documentation-Assistant\Backend\data\sql\sql_basics.txt, C:\Users\koppa\Downloads\AI-Coding-Documentation-Assistant\Backend\data\sql\sql_joins.txt
Sources
python_basics.txt
python
sql_joins.txt
sql
graphql.txt
api
regression.txt
machine_learning
sql_basics.txt
sql
Clear Chat
Ask your coding question here...
master*+
0 4
Git Graph
Stanza
32°C
Partly sunny
Search
ENG
IN
01:48 PM
01-10-2026

34. Measure Retrieval Quality

You can evaluate:

```
Retrieval Accuracy
Context Relevance
Answer Correctness
Response Time
```

For example:

```
Question
        ↓
Expected Document
        ↓
Retrieved Documents
        ↓
Did expected document appear?
```

You can calculate:

```
Recall@K
Precision@K
```

This makes the project more academically strong.

35. Step 25 — Add Error Handling

Your backend should handle:

Missing API key

```
GEMINI_API_KEY not found
```

Invalid PDF

```
Unable to read document
```

Empty question

```
Question cannot be empty
```

No relevant documents

```
No relevant documentation found
```

Gemini quota error

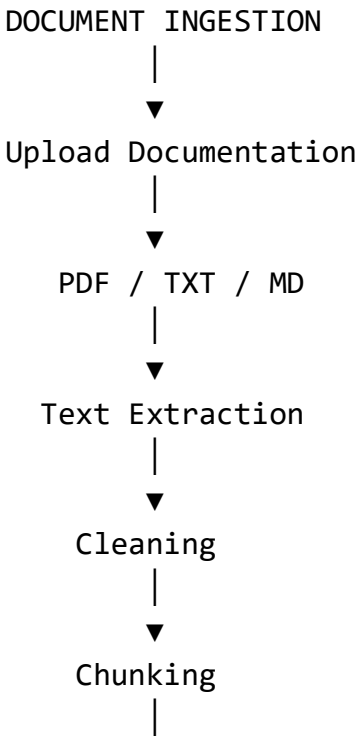
```
Gemini API quota has been exceeded.
Please try again later.
```

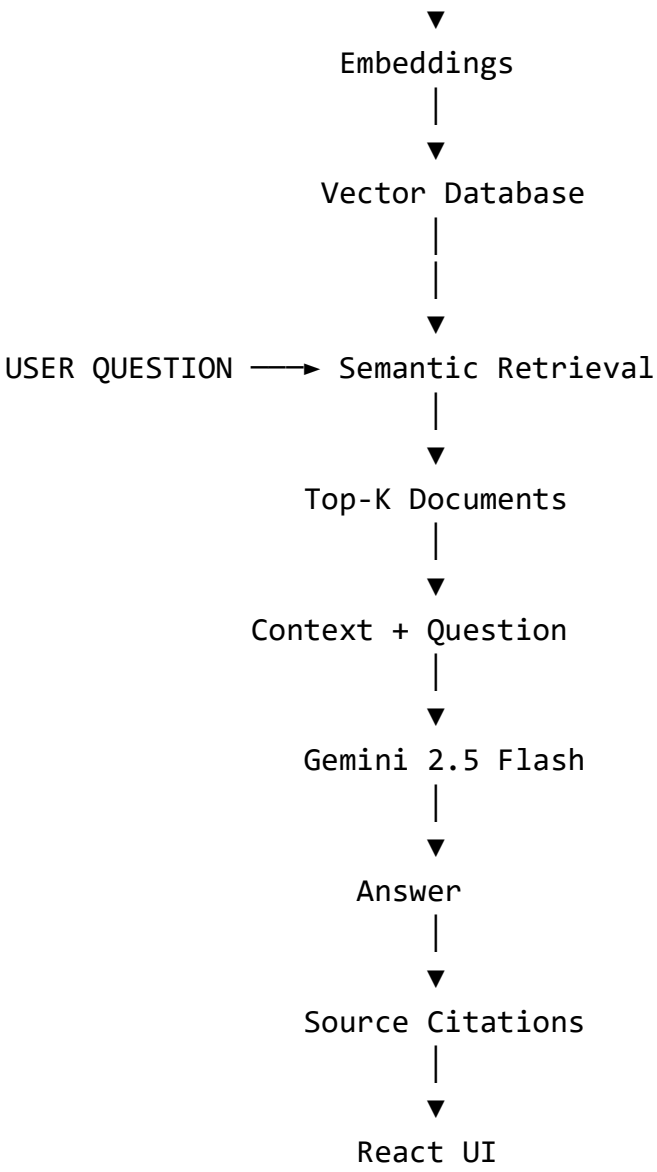
Invalid GitHub URL

```
Invalid repository URL.
```

36. Step 26 — Your Complete RAG Pipeline

At the end, your system should look like this:





37. Recommended Development Order

Don't build everything at once.

Follow this exact order:

Phase 1 — Basic RAG

```
1. Create project
2. Install packages
```

3. Configure Gemini
4. Add PDF
5. Extract text
6. Chunk text
7. Create embeddings
8. Create FAISS database
9. Implement retrieval
10. Connect Gemini

Phase 2 — Backend

11. Create FastAPI
12. /api/health
13. /api/upload
14. /api/ask
15. Error handling
16. Source information

Phase 3 — Frontend

17. Create React app
18. Upload interface
19. Chat interface
20. Display answer
21. Display sources
22. Loading animation
23. Error messages

Phase 4 — Improve RAG

24. Better chunking
25. Metadata filtering
26. Top-K retrieval
27. Conversation history
28. Source citations
29. RAG evaluation

Phase 5 — Advanced

30. GitHub URL input
31. Clone repository
32. Code parsing
33. Code embeddings
34. Repository search
35. Code-aware RAG

38. Final Feature List for Your Resume

Once completed, your project can have:

- ✅ Multi-document knowledge base
- ✅ PDF/TXT/Markdown ingestion
- ✅ Text extraction
- ✅ Text chunking
- ✅ Semantic embeddings
- ✅ FAISS vector search
- ✅ RAG pipeline
- ✅ Gemini 2.5 Flash
- ✅ Context-aware answers
- ✅ Source citations
- ✅ React chat interface
- ✅ Document upload
- ✅ Conversation history
- ✅ Error handling
- ✅ GitHub repository ingestion
- ✅ Codebase question answering

39. Strong Project Description

You can use this in your resume:

AI Coding Documentation Assistant — Developed a RAG-based AI assistant that ingests Python, SQL, Machine Learning, Git, and API documentation, performs semantic retrieval using vector embeddings, and generates context-aware technical answers using Gemini 2.5 Flash. Implemented document chunking, vector search, source citations, and an interactive React chat interface, with advanced support for querying GitHub repositories and their codebases.

40. Short LinkedIn Version

You can post:

🚀 **AI Coding Documentation Assistant**

Built a RAG-based AI assistant that allows users to query Python, SQL, Machine Learning, Git, and API documentation using natural language.

- 🔹 Semantic search & embeddings
- 🔹 FAISS vector database
- 🔹 RAG pipeline
- 🔹 Gemini 2.5 Flash
- 🔹 React + FastAPI
- 🔹 Source-aware responses
- 🔹 GitHub repository/codebase Q&A

This project helped me strengthen my understanding of **RAG, LLMs, embeddings, vector databases, and AI application development.**

#AI #GenerativeAI #RAG #LLM #Python #MachineLearning #Gemini #React #FastAPI #ArtificialIntelligence

🌟 Your immediate next step

Don't start with the React UI or GitHub feature yet.

Build this **minimum working version first**:

```
PDF
 ↓
Text Extraction
 ↓
Chunking
 ↓
Embedding
 ↓
FAISS
 ↓
Question
 ↓
Retrieval
 ↓
Gemini 2.5 Flash
 ↓
Answer
```

Once this works correctly in the terminal, we can connect **FastAPI → React**, and only after that add **GitHub repository ingestion**. This will prevent the kind of import/quota/pipeline errors you were running into in your previous Resume-JD RAG project.